



TRABAJO FIN DE GRADO
INGENIERÍA INFORMÁTICA

Control de navegación cazador-presa para sistemas aéreos no tripulados

Autor

Germán Rodríguez Vidal

Directores

Francisco Barranco Expósito
Álvaro Martínez Novo



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN

Granada, Convocatoria de Junio curso 2025/26

Control de navegación cazador-presa para sistemas aéreos no tripulados

Autor

Germán Rodríguez Vidal

Directores

Francisco Barranco Expósito

Álvaro Martínez Novo



Departamento de

INGENIERÍA DE COMPUTADORES, AUTOMÁTICA Y ROBÓTICA

DEPARTAMENTO DE INGENIERÍA DE COMPUTADORES, AUTOMÁTICA Y
ROBÓTICA

Granada, Convocatoria de Junio, curso 2025/2026

Agradecimientos

Quiero expresar mi más profunda gratitud a todas aquellas personas que me han acompañado a lo largo de mi etapa universitaria, no solo durante la realización de este Trabajo de Fin de Grado, sino también a lo largo de todo proyecto para la consecución de el grado en Ingeniería Informática.

Agradezco especialmente a mi familia, que ha sido un pilar fundamental en mi desarrollo personal y académico. Su apoyo y comprensión han sido indispensables para alcanzar las metas que me había trazado, y poder superar los momentos de duda y dificultad que fueron surgiendo en el transcurso de estos años.

En el plano académico, deseo dar la gracias al espléndido grupo de profesores que me han impartido docencia durante este periodo universitario en la Universidad de Granada. Gracias. En particular, expreso mi reconocimiento a los tutores que me han guiado en la realización de este TFG, los profesores D. Francisco Barranco Expósito y D. Álvaro Martínez Novo. Sin ellos, este proyecto no habría sido posible llevarlo a buen puerto. Su apoyo, dedicación, orientación, y profesionalidad me han impulsado a mantener un alto estándar de calidad, y a superar mis propios límites.

Finalmente, pero no menos importante, quiero agradecer, tanto a mis amigos de fatigas y alegrías, con los que he compartido buenos y malos momentos, como a aquellas nuevas amistades que se han ido forjando a lo largo de estos años universitarios. No hubiera sido lo mismo sin su compañía y apoyo en los momentos más delicados, en esas interminables horas de estudio, pero también de diversión. Todo ello ha contribuido a que esta experiencia haya sido verdaderamente memorable.

Control de navegación cazador-presa para sistemas aéreos no tripulados

Germán Rodríguez Vidal

Palabras clave: Control de navegación, UAV, , Sistemas aéreos no tripulados, Navegación autónoma, Ardupilot, Gazebo, PID, Filtro de Kalman, Detección de Objetos

Resumen

Este Trabajo Fin de Grado propone el desarrollo de métodos de navegación autónoma para sistemas aéreos no tripulados (UAV, por sus siglas en inglés) convencionales en el caso de uso del problema cazador-presa. En concreto, se plantea una estrategia de control de navegación que permita el seguimiento a corta distancia de un UAV presa por parte de un UAV cazador, el cual debe perseguirlo en espacio abierto de forma estable y robusta. Para lograrlo, se integran métodos de control y técnicas de visión por computador que facilitan la detección, localización y seguimiento del objetivo durante la misión.

Los objetivos principales de este proyecto son, en primer lugar, validar un algoritmo de control de navegación capaz de realizar el seguimiento autónomo de un UAV presa a partir de la imagen. En segundo lugar, se persigue implementar los algoritmos desarrollados en una plataforma empotrada para su posterior validación experimental en condiciones de operación real.

Para alcanzar estos objetivos, se ha seleccionado el simulador 3D Gazebo como entorno de experimentación inicial. Esta herramienta permite evaluar los algoritmos de control en escenarios controlados, modificar variables de forma sistemática y repetir pruebas con alta precisión, lo que facilita una comparación rigurosa del rendimiento del sistema ante distintas trayectorias y condiciones dinámicas.

Una vez verificado el funcionamiento del algoritmo en simulación, mediante múltiples escenarios y perfiles de movimiento, se procederá a su validación en un campo de pruebas físico. En esta fase se analizará el comportamiento del sistema frente a situaciones reales, incluyendo perturbaciones e incertidumbres propias del entorno. El sistema se considerará satisfactorio si mantiene el seguimiento del UAV presa durante un tiempo predefinido y a una distancia fija respecto al objetivo.

Hunter-prey navigation control for unmanned aerial systems

Germán Rodríguez Vidal

Keywords: Navigation control, UAV, Unmanned aerial systems, Autonomous navigation, Ardupilot, Gazebo, PID, Kalman filter, Object detection

Abstract

This Bachelor's Thesis proposes the development of autonomous navigation methods for conventional unmanned aerial vehicles (UAVs) in the hunter-prey use case. Specifically, it presents a navigation control strategy that enables short-range tracking of a prey UAV by a hunter UAV, which must pursue it in open space in a stable and robust manner. To achieve this, control methods and computer vision techniques are integrated to facilitate target detection, localization, and tracking throughout the mission.

The main objectives of this project are, first, to validate a navigation control algorithm capable of autonomously tracking a prey UAV from image-based information and, second, to implement the developed algorithms on an embedded platform for subsequent experimental validation under real operating conditions.

To achieve these objectives, the 3D Gazebo simulator has been selected as the initial experimentation environment. This tool makes it possible to evaluate control algorithms in controlled scenarios, modify variables systematically, and repeat tests with high precision, which enables a rigorous comparison of system performance under different trajectories and dynamic conditions.

Once the algorithm has been verified in simulation through multiple scenarios and motion profiles, it will be validated in a physical test field. In this phase, the system behavior will be analyzed under real-world conditions, including disturbances and uncertainties inherent to the environment. The system will be considered satisfactory if it maintains tracking of the prey UAV for a predefined period and at a fixed distance from the target.

Yo, **Germán Rodríguez Vidal**, alumno de la titulación Ingeniería Informática de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI xxxxxxxx, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Germán Rodríguez Vidal

Granada a Junio de 2026 .

D. **Francisco Barranco Expósito** , Profesor del Área de Arquitectura y Tecnología de Computadores del Departamento Ingeniería de Computadores, Automática y Robótica de la Universidad de Granada.

D. **Álvaro Martínez Novo**, Profesor del Área de Arquitectura y Tecnología de Computadores del Departamento Ingeniería de Computadores, Automática y Robótica de la Universidad de Granada.

Informan:

Que el presente trabajo, titulado ***Control de navegación cazador-presa para sistemas aéreos no tripulados***, ha sido realizado bajo su supervisión por **Germán Rodríguez Vidal** , y autorizamos la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expiden y firman el presente informe en Granada a Junio de mes de 2026 .

Los directores:

Francisco Barranco Expósito

Álvaro Martínez Novo

Índice general

Índice de figuras	5
Índice de tablas	8
Índice de extractos de código	9
1. Introducción	10
1.1. Descripción del problema	10
1.2. Motivación	12
1.3. Objetivos	13
1.3.1. Objetivo general	13
1.3.2. Objetivos específicos	13
1.4. Planificación	14
1.4.1. Detalle de las fases	14
1.4.2. Presupuesto	16
2. Estado del Arte	18
2.1. Introducción	18
2.2. Sistemas Aéreos no Tripulados	19
2.2.1. Clasificación por configuración física	19
2.2.2. Aplicaciones actuales	21
2.3. Métodos de Control de Navegación para UAVs	24
2.3.1. Tipos de control de navegación	25
2.3.2. Conceptos de Visual Servoing (IBVS)	26
2.4. <i>Deep Learning</i>	27
2.5. Visión por Computador	28
2.5.1. Métodos de detección de objetos	29
2.5.2. YOLO	30
2.6. Geometría Proyectiva	31
2.6.1. Cámara estenopeica	32
2.6.2. Modelo geométrico de la cámara	33
2.6.3. Usos en navegación visual	34
2.7. Métodos de Control	34
2.7.1. Sistemas de Lazo Abierto vs. Lazo Cerrado	35
2.7.2. Controlador PID	35
2.8. Estimación de Estado y Filtrado	37
2.8.1. Origen y Evolución del Filtro de Kalman	37
2.8.2. Fundamentos y Mecanismo Recursivo	37
2.8.3. Aplicación en Persecución Visual y Alta Velocidad	38
2.8.4. Filtro EMA	38

2.9.	Stack de Software y Hardware	40
2.9.1.	Intercambio de Datos en Robótica	40
2.9.2.	Computación Distribuida Seleccionada	41
2.9.3.	Hardware de Procesamiento Embarcado	46
2.9.4.	Selección de la NVIDIA Jetson Orin Nano	49
2.9.5.	Métodos de Aceleración de Inferencia	50
2.9.6.	Selección de TensorRT para el Despliegue Embarcado	52
2.9.7.	Simulación	53
2.9.8.	Simulador Seleccionado	55
3.	Metodología de Trabajo	57
3.1.	Enfoque Metodológico del Proyecto	57
3.2.	Entorno Experimental y Herramientas	58
3.2.1.	Preparación del Entorno Base	58
3.2.2.	Herramientas y Dependencias Empleadas	59
3.3.	Diseño Metodológico de la Simulación en Gazebo	61
3.3.1.	Modelo del Vehículo Aéreo y Escenario de Prueba	61
3.3.2.	Procedimiento de Lanzamiento y Supervisión	62
3.4.	Diseño del Escenario Multi-UAV	63
3.4.1.	Definición del UAV Cazador	63
3.4.2.	Definición del UAV Objetivo	65
3.4.3.	Inicialiación de la Simulación	66
3.5.	Metodología Para el Componente de Percepción	68
3.5.1.	Detector de UAVs	68
3.5.2.	Dataset Utilizado	69
3.5.3.	Geometría Proyectiva aplicada a la Percepción Visual	71
3.5.4.	Flujo General del Módulo de Percepción	74
3.5.5.	Cálculo y Normalización del Error Visual	75
3.6.	Estimación de Distancia	76
3.7.	Filtrado	78
3.7.1.	Filtro de Kalman	78
3.7.2.	Uso del Filtro de Kalman en el Sistema Propuesto	80
3.7.3.	Filtro EMA aplicado a la Caja de Detección	81
3.8.	Metodología de Control	82
4.	Implementación	84
4.1.	Solución de software	84
4.1.1.	Arquitectura del sistema	84
4.1.2.	Nodos y scripts desarrollados	85
4.2.	Componente de percepción	90
4.2.1.	Entrenamiento de modelos YOLO	90
4.2.2.	Estimación de la distancia mediante el ancho de la caja	94
4.3.	Compensación de actitud y proyección	95
4.4.	Componente de control	98

4.5.	Configuración para pruebas reales	99
4.5.1.	Dron real	99
4.5.2.	Preparación del dron real	101
4.5.3.	Calibración de la cámara	102
4.5.4.	Medición del UAV objetivo y posición de la cámara	103
5.	Pruebas y Validaciones	106
5.1.	Planificación de las Pruebas	106
5.2.	Desarrollo de los Experimentos	107
5.2.1.	Métricas de Evaluación	107
5.2.2.	Selección del Modelo	111
5.2.3.	Optimización del Modelo en Dron Real con TensorRT	113
5.2.4.	Pruebas de Control	115
5.2.5.	Seguimiento del Objetivo en Simulación	129
5.2.6.	Prueba de Estimación de Distancia con Cámara Real	137
5.3.	Análisis de Resultados	143
6.	Conclusiones	146
6.1.	Revisión del cumplimiento de objetivos	146
6.2.	Futuros Trabajos	149
7.	Bibliografía	150

Índice de figuras

1.1. Logo del INTA	11
2.1. Dron de ala fija	20
2.2. Dron multirrotor	20
2.3. Dron VTOL de ala fija	21
2.4. Dron monitoreando cultivos	22
2.5. Dron utilizado en búsqueda y rescate marítimo	22
2.6. Dron utilizado transporte de paquetes	23
2.7. Drones utilizados para operaciones militares	23
2.8. Tipos de algoritmos de Visual Servoing. (Imagen de Elaboración Propia)	25
2.9. Esquema general de una red neuronal multicapa	28
2.10. Ejemplo ilustrativo de detección de objetos mediante cajas delimitadoras	29
2.11. Esquema simplificado del funcionamiento de R-CNN	30
2.12. Esquema simplificado del funcionamiento de Yolo	31
2.13. Funcionamiento simplificado de una cámara estenopeica	33
2.14. Esquema simplificado del modelo de cámara estenopeica	34
2.15. Esquema de un PID	36
2.16. Efecto del filtro EMA sobre una señal ruidosa	39
2.17. Logo de ROS	43
2.18. Arduino Uno	47
2.19. Raspberry Pi	48
2.20. NVIDIA Jetson Orin Nano	48
2.21. Uso de Gazebo con la cámara. (Imagen de Elaboración Propia)	56
3.1. Panel básico de ejecución con varias terminales organizadas en Terminator	66
3.2. Marcos de referencia publicados durante la ejecución inicial con dos UAVs	68
3.3. Vista del dataset final organizado en Roboflow. (Imagen de Elaboración Propia)	70
3.4. Esquema simplificado del modelo de cámara estenopeica	72
3.5. Distorsión de cámara pinhole	74
3.6. Representación del error visual entre el centro de la imagen y el UAV objetivo detectado	75
3.7. Ejemplo de uso del filtro de Kalman sobre la detección visual	79
4.1. Arquitectura general del sistema de seguimiento visual, válida tanto para simulación como para despliegue real. (Imagen de elaboración propia)	84
4.2. Grafo inicial del sistema sin nodos de visión ni control activos	86
4.3. Grafo del sistema con los nodos de visión activados	86

4.4.	Grafo completo del sistema con visión y control activos . . .	87
4.5.	Listado de tópicos ROS obtenido mediante rostopic list . . .	88
4.6.	Evolución de pérdidas y métricas durante el entrenamiento de YOLOv8n. (Imagen de elaboración propia)	91
4.7.	Matriz de confusión obtenida tras el entrenamiento de YOLOv8n. (Imagen de elaboración propia)	92
4.8.	Ejemplos de detección sobre el conjunto de validación tras el entrenamiento. (Imagen de elaboración propia)	93
4.9.	Esquema de compensación de actitud aplicado al plano de imagen	96
4.10.	Vista frontal del dron real utilizado en las pruebas	100
4.11.	Vista posterior del dron real utilizado en las pruebas	101
4.12.	Proceso de calibración de la cámara real mediante tablero de ajedrez	102
4.13.	Montaje del dron cazador con la cámara utilizada en las pruebas reales. (Imagen de elaboración propia)	104
5.1.	Trayectoria circular utilizada para la prueba del PID de <i>yaw</i> . (Imagen de Elaboración Propia)	116
5.2.	Respuesta del PID de <i>yaw</i> durante la trayectoria circular. (Imagen de Elaboración Propia)	117
5.3.	Trayectoria vertical utilizada para la prueba del PID de altura. (Imagen de Elaboración Propia)	118
5.4.	Respuesta del PID de altura durante la trayectoria vertical. (Imagen de Elaboración Propia)	119
5.5.	Trayectoria recta utilizada para la prueba del PID de distancia. (Imagen de Elaboración Propia)	121
5.6.	Respuesta del PID de distancia durante la trayectoria recta. (Imagen de Elaboración Propia)	122
5.7.	Tiempo de establecimiento del PID de <i>yaw</i> . (Imagen de Elaboración Propia)	123
5.8.	Tiempo de establecimiento del PID de altura. (Imagen de Elaboración Propia)	124
5.9.	Tiempo de establecimiento del PID de distancia. (Imagen de Elaboración Propia)	125
5.10.	Trayectoria circular ondulada utilizada para la prueba conjunta de los tres PID. (Imagen de Elaboración Propia)	127
5.11.	Respuesta conjunta de los tres PID durante la trayectoria circular ondulada. (Imagen de Elaboración Propia)	128
5.12.	Trayectoria en zigzag utilizada para la prueba de seguimiento del objetivo. (Imagen de Elaboración Propia)	130
5.13.	Vista del escenario de Gazebo y la cámara durante la prueba de seguimiento en zigzag. (Imagen de Elaboración Propia) . .	131

5.14. Vista del escenario inicial de Gazebo en la prueba de seguimiento con trayectoria circular ondulada. (Imagen de Elaboración Propia)	133
5.15. Trayectoria final por <i>waypoints</i> utilizada para la prueba de seguimiento. (Imagen de Elaboración Propia)	135
5.16. Montaje utilizado para la prueba real de estimación de distancia. (Imagen de Elaboración Propia)	138
5.17. Vista del UAV objetivo a una distancia aproximada de 1 m. (Imagen de Elaboración Propia)	139
5.18. Vista del UAV objetivo a una distancia aproximada de 5.75 m. (Imagen de Elaboración Propia)	140
5.19. Errores de estimación de distancia obtenidos en las cinco pruebas realizadas con la cámara real y el UAV objetivo real. (Imagen de Elaboración Propia)	141
5.20. Error medio de estimación de distancia con la desviación típica sombreada para las cinco pruebas realizadas. (Imagen de Elaboración Propia)	142

Índice de tablas

1.1. Planificación del proyecto	14
1.2. Resumen de costes del proyecto	17
2.3. Comparativa de consumo y rendimiento de módulos NVIDIA Jetson Orin considerados.	49
2.4. Características principales de la NVIDIA Jetson Orin Nano. .	50
4.1. Parámetros finales de los controladores PID utilizados para el seguimiento visual.	99
5.1. Comparativa de modelos YOLO evaluados para la detección visual.	112
5.2. Comparativa de rendimiento antes y después de la optimiza- ción con TensorRT.	114
5.3. Tiempos de establecimiento obtenidos para cada controlador PID.	125
5.4. Métricas EPE/RMSE/TLR obtenidas durante la prueba de seguimiento con trayectoria en zigzag.	131
5.5. Métricas EPE/RMSE/TLR obtenidas durante la prueba de seguimiento con trayectoria circular ondulada.	133
5.6. <i>Waypoints</i> utilizados en la ruta final de seguimiento.	135
5.7. Métricas EPE/RMSE/TLR obtenidas durante la prueba de seguimiento con ruta final por <i>waypoints</i>	136
5.8. Métricas globales del modelo de estimación de distancia con cámara real.	142
5.9. Resumen del grado de cumplimiento de las pruebas realizadas.	144

Índice de extractos de código

3.1. Archivo <code>apm.launch</code> genérico para múltiples drones	62
3.2. Identificador asignado al primer UAV en <code>drone1.params.parm</code>	64
3.3. Fragmento principal de <code>apm.config_d1.yaml</code> para <code>drone1</code> . .	64
3.4. Inclusión de <code>drone1</code> y <code>drone2</code> en <code>runaway.world</code>	65

Introducción

1.1. Descripción del problema

El auge y la proliferación de los vehículos aéreos no tripulados (UAVs), especialmente los cuadricópteros, han transformado múltiples sectores que van desde la logística hasta la defensa, gracias a sus capacidades de despegue y aterrizaje vertical (VTOL), alta maniobrabilidad y coste relativamente bajo. Sin embargo, esta expansión tecnológica ha generado simultáneamente nuevos retos de seguridad debido a su potencial uso hostil, ya que estos sistemas pueden irrumpir en espacios aéreos restringidos como aeropuertos, zonas militares e infraestructuras críticas.

Las contramedidas actuales para neutralizar estas amenazas, que incluyen sistemas de interferencia electrónica y soluciones cinéticas, como láseres o torretas, presentan limitaciones significativas debido a los riesgos de daños colaterales, restricciones regulatorias y altos costes operativos. En este contexto, los cuadricópteros han surgido como una plataforma óptima para la interceptación. A diferencia de los sistemas estacionarios o de superficie, un dron interceptor opera en el mismo dominio aéreo que su objetivo, lo que ofrece un mayor alcance y precisión, a la vez que minimiza las interrupciones involuntarias al tráfico aéreo legal. Además, su despliegue resulta mucho más rentable en comparación con los sistemas tradicionales de grado militar y se presenta como una solución escalable para asegurar espacios aéreos sensibles.

No obstante, la eficacia de un dron cazador depende críticamente de su nivel de autonomía. El enfoque convencional de operación remota basa su éxito en la pericia del piloto humano bajo estrés y en la integridad de un enlace de radiocomunicación bidireccional en tiempo real. Esta dependencia hace que los sistemas sean más detectables y limita su escalabilidad y sus tiempos de respuesta. Además, los vuelve extremadamente vulnerables a la latencia, a técnicas de guerra electrónica y a la inhibición de frecuencias (*jamming* o *spoofing* de GPS), amenazas predominantes que pueden dejar al cazador inoperativo en el momento crítico de la misión.

Para superar estas vulnerabilidades, este Trabajo Fin de Grado aborda el desarrollo de una solución de navegación autónoma para un escenario de seguimiento a corta distancia de tipo cazador-presa. Se propone una estrategia de control basada en la identificación y persecución visual activa, lo que permite al UAV cazador aproximarse a su objetivo de forma precisa. Frente a la vulnerabilidad de las comunicaciones, esta solución garantiza la independencia táctica al procesar la información visual mediante algoritmos de inteligencia artificial directamente en una plataforma embebida a bordo del dron (*on-board*). Esto permite tiempos de reacción mínimos y un seguimiento robusto que no se ve degradado por interferencias, maximizando la eficiencia de la respuesta.

A su vez, para que esta solución mantenga su viabilidad económica frente a alternativas que utilizan sensores costosos, como LiDAR o radar, que erosionan la rentabilidad de los UAVs pequeños, el sistema propuesto está diseñado para operar con un conjunto sensorial mínimo: una cámara monocular y una unidad de medida inercial (IMU).

Esta investigación se enmarca directamente en la necesidad de fortalecer la soberanía tecnológica nacional en materia de seguridad y defensa. En esta línea, el desarrollo está alineado con los intereses del **Instituto Nacional de Técnica Aeroespacial (INTA)**, con sede principal en Torrejón de Ardoz (Madrid), orientados al desarrollo de capacidades avanzadas para la detección, monitorización y neutralización de UAVs que operen de forma hostil. Uno de los objetivos de ese proyecto es precisamente el abordado en este TFG.



Figura 1.1. Logo del INTA. Fuente: [10].

Por ello, el problema específico abordado en este trabajo es demostrar que un sistema de control de navegación visual, basado en modelos de percepción profunda (*Deep Learning*) y algoritmos de control reactivo, permite a un UAV cazador realizar una persecución autónoma, estable y robusta en tiempo real. Los criterios principales a validar durante la investigación son:

- Mantener al UAV objetivo enfocado y centrado en el campo de visión de la cámara de forma continua, minimizando el error de seguimiento.
- Asegurar la capacidad de respuesta y maniobrabilidad del UAV cazador ante trayectorias dinámicas, aceleraciones bruscas o movimientos imprevistos de la presa.

- Garantizar que el conjunto de software pueda ejecutarse manteniendo una tasa de fotogramas por segundo (FPS) óptima en hardware de recursos limitados.

Para llevar a cabo esta validación, el sistema propuesto se probará inicialmente en entornos de simulación 3D, utilizando ROS y Gazebo e integrándolos con ArduPilot mediante simulaciones SITL. Finalmente, tras analizar las métricas obtenidas, se preparará su despliegue en plataformas embebidas reales para su evaluación práctica.

1.2. Motivación

El aumento del uso de sistemas aéreos no tripulados en defensa, junto con la amenaza que representan los UAVs para la seguridad nacional, hace necesaria la creación de sistemas antidrón que funcionen de forma autónoma, eficaz y segura. A medida que los UAVs comerciales son más accesibles y potentes, también se incrementa el riesgo de un uso indebido, desde el espionaje hasta amenazas a infraestructuras críticas, áreas urbanas y eventos con una gran concentración de personas. Esto ha evidenciado la necesidad urgente de disponer de sistemas de neutralización reactivos y muy eficaces.

La principal motivación de este trabajo es dotar a los UAVs cazadores de capacidades de percepción y navegación que les permitan adaptarse rápidamente a maniobras evasivas o a cambios inesperados en las trayectorias de los objetivos. Se trata de ir más allá de las limitaciones de los métodos clásicos de navegación basados en control remoto o en la recepción continua de coordenadas GPS, y de ofrecer una solución basada en modelos de *Deep Learning* y en algoritmos de control reactivo con visión artificial.

Asimismo, el desafío tecnológico de combinar un sistema de detección visual con un control de navegación avanzado en un entorno de simulación, junto con su implementación en hardware real, ofrece una oportunidad valiosa para aplicar los conocimientos adquiridos. Esto implica integrar disciplinas como la inteligencia artificial, la teoría del control, la simulación robótica y el desarrollo de sistemas embebidos. Estas áreas son clave en la ingeniería informática y en la robótica moderna.

Este trabajo no solo busca mejorar la autonomía y precisión de seguimiento de los UAVs cazadores, sino también establecer las bases para desarrollos futuros en navegación autónoma robusta que opere de manera independiente en el exigente campo de la seguridad y defensa.

1.3. Objetivos

1.3.1. Objetivo general

Desarrollar e implementar un sistema robusto de control de navegación autónoma para vehículos aéreos no tripulados (UAVs) capaz de realizar el seguimiento de un objetivo en tiempo real, integrando técnicas de control reactivo, algoritmos de estimación de estado y visión por computador.

1.3.2. Objetivos específicos

Para alcanzar el objetivo general, se proponen los siguientes objetivos específicos:

- **OE1: Evaluar y optimizar modelos de percepción visual para sistemas embebidos.** Entrenar y comparar distintas arquitecturas de detección de objetos de última generación (familia YOLO), evaluando el equilibrio entre precisión (mAP), tiempo de inferencia y tamaño del modelo para seleccionar el modelo óptimo que garantice un seguimiento fluido en tiempo real.
- **OE2: Diseñar e implementar el lazo de control de navegación visual.** Desarrollar el algoritmo encargado de traducir las detecciones 2D de la cámara en comandos de vuelo 3D. Esto implicará el uso de transformaciones geométricas de coordenadas y la sintonización de un controlador reactivo (PID) para mantener a la presa centrada en el campo de visión.
- **OE3: Integrar la arquitectura de software en un entorno de simulación realista.** Implementar el sistema completo utilizando ROS como *middleware* de comunicación, acoplando los nodos de visión y control con el simulador físico 3D Gazebo y ArduPilot mediante simulaciones SITL (*Software-in-the-Loop*).
- **OE4: Validar la robustez del sistema frente a maniobras evasivas.** Diseñar y ejecutar una batería de pruebas automatizadas en las que el UAV objetivo (presa) realice diferentes perfiles de vuelo dinámicos para cuantificar el error de seguimiento y los tiempos de respuesta del cazador.
- **OE5: Desplegar y evaluar la viabilidad computacional en una plataforma embebida real.** Migrar el conjunto de software desarrollado a un hardware de recursos limitados para verificar que el sistema es capaz de mantener la tasa de procesamiento necesaria (FPS) sin comprometer la latencia ni la seguridad del vuelo.

1.4. Planificación

La planificación del proyecto se llevó a cabo utilizando un método en cascada, siguiendo un proceso lineal en el que cada etapa se basa en el éxito de la precedente. Este enfoque es adecuado, ya que cada avance tecnológico debe consolidar la fase anterior para reducir riesgos y fallos.

El tiempo total para la realización del proyecto se estimó en 300 horas, correspondientes a 12 créditos ECTS. Sin embargo, la asignación de este tiempo experimentó algunas alteraciones debido a la necesidad de ajustarse a responsabilidades personales y a los obstáculos que surgieron a lo largo del desarrollo.

En la tabla que se presenta a continuación se muestra la organización general del proyecto, seguida de una explicación detallada de cada etapa.

Fase	Duración	Oct.	Nov.	Dic.	Ene.	Feb.	Mar.	Abr.	May.
Configuración del entorno e integración del controlador	1 mes	X							
Incorporación de los UAVs al simulador	1 mes		X						
Configuración de los controladores	2 meses			X	X				
Pruebas simuladas y representación visual de los resultados	2 meses					X	X		
Implementación del control en UAV real	1 mes							X	
Documentación	3 semanas								X

Tabla 1.1: Planificación del proyecto.

1.4.1. Detalle de las fases

A continuación, se describe brevemente el contenido de cada una de las etapas representadas en la Tabla 1.1.

- 1. Configuración del entorno e integración del controlador (octubre).** Durante esta fase se instaló y configuró el entorno de trabajo, incluyendo ROS, Gazebo, MAVROS y ArduPilot.
- 2. Incorporación de los UAVs al simulador (noviembre).** Se integraron los modelos de los UAVs en Gazebo y se verificó el funcionamiento de los sensores virtuales y de los canales de comunicación. Esta etapa permitió disponer de un entorno de simulación consistente y representativo del escenario de trabajo.
- 3. Configuración de los controladores (diciembre-enero).** Se desarrollaron y ajustaron los controladores necesarios para el seguimiento

visual del objetivo, así como los nodos ROS encargados de intercambiar información entre percepción, estimación y actuación. Además, se realizaron pruebas iniciales en condiciones controladas para comprobar la estabilidad del lazo de control.

4. **Pruebas simuladas y representación visual de los resultados (febrero–marzo).** Se ejecutaron vuelos de prueba en distintos escenarios simulados para evaluar métricas como el error de seguimiento, la capacidad de respuesta y la estabilidad del sistema frente a maniobras dinámicas de la presa. Los datos obtenidos se procesaron y representaron mediante gráficas que facilitaron su análisis e interpretación.
5. **Implementación del control en UAV real (abril).** El sistema desarrollado se adaptó para su despliegue en una plataforma embebida asociada al UAV real. En esta fase se realizaron ajustes de integración y las primeras verificaciones experimentales orientadas a validar el funcionamiento del sistema fuera del entorno puramente simulado.
6. **Documentación (mayo, 3 semanas).** Finalmente, se elaboró la memoria del proyecto, recopilando el desarrollo realizado, los resultados obtenidos, las conclusiones principales y las posibles líneas de mejora y trabajo futuro.

Si bien la planificación inicial resultó adecuada, durante el desarrollo del proyecto surgieron desviaciones inevitables respecto al cronograma previsto. En particular, la fase de configuración del entorno requirió más esfuerzo del estimado debido a incompatibilidades entre distintas versiones de ROS y a dificultades en la integración con MAVROS. Asimismo, la comunicación entre los UAVs presentó problemas adicionales que obligaron a realizar ajustes sucesivos en la arquitectura de software y provocaron retrasos puntuales en algunas de las fases posteriores.

Las pruebas en simulación se desarrollaron de manera satisfactoria. No obstante, la incorporación de nuevas implementaciones y ajustes de diseño prolongó esta fase, ya que durante su ejecución se detectaron pequeños errores que fue necesario corregir de forma iterativa. Aun así, el desarrollo de scripts y automatizaciones para el procesamiento y la representación de los datos contribuyó a agilizar el análisis de resultados y a hacer más eficiente el flujo de trabajo.

En conjunto, la metodología en cascada permitió mantener una estructura de trabajo clara y ordenada a lo largo del proyecto. No obstante, su carácter secuencial dificultó en determinados momentos la adaptación ágil ante incidencias imprevistas, especialmente durante las fases de integración

de módulos y depuración del entorno experimental. A pesar de estas limitaciones, el proyecto avanzó conforme a los plazos generales previstos, y los resultados obtenidos en simulación respaldan la viabilidad del enfoque de navegación autónoma propuesto como base para su posterior validación en una plataforma real.

1.4.2. Presupuesto

El desarrollo del proyecto ha implicado distintos costes asociados al uso de equipamiento informático para la ejecución de simulaciones, a la adquisición de materiales necesarios para la implementación del sistema embebido en el UAV real y al coste estimado de la dedicación personal. A continuación, se detallan los conceptos principales que conforman el presupuesto global del proyecto.

Equipo y herramientas

Durante las fases iniciales del trabajo fue necesario disponer de un equipo informático con capacidad suficiente para ejecutar simulaciones con una carga moderada tanto de procesamiento como gráfica. Para ello se utilizó un ordenador personal cuyas prestaciones (procesador de alto rendimiento, tarjeta gráfica dedicada y memoria RAM suficiente) permitieron ejecutar de manera adecuada los entornos ROS, Gazebo y el resto de herramientas de software empleadas en el desarrollo.

El coste estimado del equipo informático utilizado se fija, de forma aproximada, en **1.400 €**.

Gastos de implementación del sistema embebido

La implementación del sistema embebido en el UAV real supuso un coste total de **2.693,10 €**. Este importe incluye la adquisición de componentes estructurales, como el chasis y el tren de aterrizaje; elementos de propulsión, entre ellos motores, variadores electrónicos de velocidad (ESC) y hélices; el sistema de control y navegación, compuesto por la controladora Pixhawk Cube Orange, el módulo RTK y la estación base; los dispositivos de comunicación, como la telemetría RFD y la emisora RC; el sistema de procesamiento embebido basado en una Jetson Orin Nano; los elementos de alimentación, como baterías y convertidores DC-DC; así como cableado, conectores y diverso material auxiliar de montaje.

Horas de trabajo

La dedicación al proyecto se estima en 300 horas, correspondientes a los 12 créditos ECTS asignados. Para realizar una valoración económica orientativa del trabajo desarrollado, se considera una tarifa de 20 € por hora, representativa de un perfil júnior de ingeniería [2]. Bajo esta hipótesis, el coste asociado a la dedicación personal asciende a:

$$300 \text{ horas} \times 20 \text{ €} / \text{ hora} = 6.000 \text{ €}$$

Resumen de costes

En la Tabla 1.2 se recoge el resumen de los costes estimados del proyecto.

Concepto	Coste (€)
Ordenador personal para simulaciones	1.400,00
Materiales para la implementación del sistema embebido	2.693,10
Horas de trabajo	6.000,00
Total estimado del proyecto	10.093,10

Tabla 1.2: Resumen de costes del proyecto.

Por tanto, el coste total estimado del proyecto asciende a **10.093,10 €**, considerando tanto los recursos materiales empleados como la valoración económica aproximada del tiempo de trabajo dedicado a su desarrollo.

Estado del Arte

2.1. Introducción

Esta sección tiene como objetivo establecer el marco teórico y tecnológico que sustenta el desarrollo de este proyecto. Dado que el control de navegación para el seguimiento a cortas distancias de vehículos aéreos no tripulados (UAV) constituye un campo multidisciplinar, resulta necesario revisar las principales tecnologías empleadas para abordar este problema [27] y fundamentar la solución propuesta.

En primer lugar, se revisan los Sistemas Aéreos no Tripulados, analizando su clasificación actual y cómo su evolución ha permitido que su utilización se extienda globalmente, pasando de aplicaciones militares a complejas tareas de investigación y servicios civiles [6, 7].

El bloque de percepción visual introduce, en primer lugar, los fundamentos del *Deep Learning*, para después enmarcarlos dentro de la visión por computador y de la detección de objetos. En este contexto, se analiza la familia YOLO como uno de los enfoques más extendidos para percepción visual en tiempo real sobre plataformas embarcadas [41].

El núcleo estratégico del proyecto se desarrolla en el Control de Navegación Visual. Se analizan los distintos tipos de navegación aérea, profundizando en el paradigma IBVS (Image-Based Visual Servoing) [25]. Basándose en la literatura técnica consultada, se justifica el uso de IBVS como una solución más eficiente y menos costosa en términos de procesamiento y complejidad técnica que otros enfoques [27, 28]. Asimismo, se exploran diversos métodos de control, fundamentando la elección del controlador PID como una herramienta clásica y eficaz para el seguimiento de objetivos dinámicos.

Dada la naturaleza ruidosa de las detecciones visuales, la sección de Estimación de Estado y Filtrado detalla el uso del Filtro de Kalman [42] y el filtro EMA (Exponential Moving Average). Estas herramientas son críticas para mitigar el ruido de los sensores y predecir la posición del objetivo en caso de oclusiones temporales.

Para conectar el plano de la imagen con el mundo físico, se presentan los fundamentos de la Geometría Proyectiva [54]. En este apartado se explica el funcionamiento de la proyección de la cámara y el uso de transformaciones

de coordenadas, esenciales para traducir los errores en píxeles a comandos de movimiento en el sistema de referencia del dron [48].

Finalmente, se describe el Stack de Software y Hardware, detallando los frameworks de comunicación robótica (ROS/Mavros) y las herramientas de simulación (Gazebo) que permiten la validación segura de los algoritmos desarrollados.

2.2. Sistemas Aéreos no Tripulados

Los Sistemas Aéreos no Tripulados (UAS, por sus siglas en inglés, *Unmanned Aerial Systems*) han experimentado una evolución notable en las últimas décadas. Lo que comenzó como una tecnología asociada principalmente al ámbito militar se ha transformado en una plataforma versátil con aplicaciones en ingeniería civil, agricultura de precisión, logística, inspección y vigilancia. Esta expansión ha sido posible gracias a la miniaturización de sensores y al incremento de la capacidad de cómputo embarcado, lo que ha permitido desplegar drones en escenarios complejos donde la autonomía constituye un requisito crítico.

Sin embargo, el diseño de sistemas de navegación autónoma que sean robustos, eficientes y capaces de adaptarse a entornos dinámicos y no estructurados sigue siendo uno de los principales retos actuales.

2.2.1. Clasificación por configuración física

Atendiendo a los requisitos de maniobrabilidad y a la naturaleza de la misión, los UAS pueden clasificarse principalmente según su estructura [6]:

- **Ala fija (*Fixed-wing*).** Presentan una arquitectura similar a la de la aviación convencional. Destacan por su elevada autonomía y eficiencia en vuelos de larga distancia. No obstante, su incapacidad para realizar vuelo estacionario (*hover*) los hace poco adecuados para misiones de interceptación que requieran cambios bruscos de dirección o seguimiento cercano a baja velocidad. La Figura 2.1 muestra un ejemplo representativo de esta configuración.



Figura 2.1. Dron de ala fija. Fuente: [3].

- **Multirrotores (*Multi-rotors*).** Constituyen la plataforma de referencia para este Trabajo de Fin de Grado. Su configuración, habitualmente basada en cuatro motores, permite un control directo. Su capacidad de despegue y aterrizaje vertical (VTOL), junto con su elevada agilidad, resulta esencial para mantener a un objetivo móvil dentro del campo de visión de la cámara de forma continua. La Figura 2.2 presenta un ejemplo de esta configuración.



Figura 2.2. Dron multirrotor. Fuente: [5].

- **Híbridos (VTOL *Fixed-wing*).** Tratan de combinar la autonomía del ala fija con la versatilidad del multirrotor, aunque su mayor complejidad mecánica y de control suele traducirse en un incremento del peso, el coste y la dificultad de integración. La Figura 2.3 presenta un ejemplo de plataforma VTOL de ala fija.



Figura 2.3. Dron VTOL de ala fija. Fuente: [4].

2.2.2. Aplicaciones actuales

Entre sus distintas configuraciones, el dron multirrotor se ha consolidado como la plataforma más extendida debido a su maniobrabilidad, su facilidad de despliegue y su capacidad de vuelo estacionario, cualidades especialmente valiosas en operaciones que requieren cercanía al entorno y una respuesta rápida ante cambios del escenario.

Tradicionalmente, muchas de estas operaciones han dependido de un control humano directo, en el que un operador supervisa la trayectoria, corrige desviaciones y decide cada maniobra relevante. Este enfoque puede resultar suficiente en escenarios simples o muy controlados, sin embargo, pierde eficacia cuando el entorno es dinámico, cuando aparecen obstáculos imprevistos o cuando el objetivo modifica su trayectoria de forma continua.

En este contexto, la navegación autónoma surge como una solución tecnológica necesaria no solo para mejorar el rendimiento del sistema, sino también para reducir la carga de trabajo del operador humano. En misiones prolongadas, repetitivas o de alta exigencia, la dependencia constante del pilotaje manual incrementa el riesgo de fatiga, errores de supervisión y pérdida de eficacia operativa. La incorporación de capacidades autónomas permite automatizar tareas como el seguimiento de trayectorias, la corrección del movimiento y la respuesta ante cambios del entorno, disminuyendo la intervención continua del operador. Además, este enfoque contribuye a optimizar recursos y a abaratar costes operativos, al reducir la necesidad de control manual permanente y favorecer misiones más estables, seguras y sostenibles en el tiempo.

A partir de esta idea, la navegación autónoma adquiere especial relevancia en varias situaciones de uso:

Utilización en agricultura: En el ámbito de la agricultura de precisión, la navegación autónoma permite recorrer parcelas extensas, mantener trayectorias estables y adaptar el vuelo a las características del terreno o del cultivo [17]. Esto facilita tareas como la supervisión del estado de las plantaciones, la detección de incidencias y la recopilación sistemática de información, reduciendo al mismo tiempo la intervención manual continua del operador.



Figura 2.4. Dron utilizado en agricultura. Fuente: [16].

Apoyo en emergencias y búsqueda de objetivos: En escenarios de emergencia, catástrofe o difícil acceso, la navegación autónoma permite explorar áreas amplias, evitar obstáculos y centrar la misión en la localización de objetivos de interés, como personas, embarcaciones, vehículos o puntos críticos del terreno. Esta capacidad resulta valiosa en operaciones de apoyo en emergencias, donde el sistema debe reaccionar, adaptarse a cambios del entorno y mantener una búsqueda continua [19].

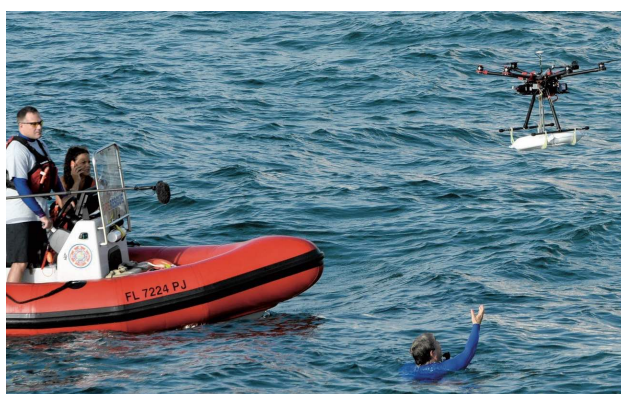


Figura 2.5. Dron utilizado en búsqueda y rescate marítimo. Fuente: [18].

Transporte autónomo de mercancías: En entornos urbanos, industriales o logísticos [21], los UAV pueden convertirse en una alternativa eficiente para

el envío de pequeñas cargas, repuestos o material de primera necesidad. En este tipo de operaciones, la navegación autónoma permite reajustar la ruta ante restricciones del entorno, variaciones en el trayecto o condiciones externas no previstas, manteniendo al mismo tiempo un vuelo estable y seguro.



Figura 2.6. Dron utilizado para transporte de paquetes. Fuente: [20].

Operaciones militares y de defensa: Este es el caso de uso más próximo al enfoque de este Trabajo de Fin de Grado. En misiones de reconocimiento, vigilancia, defensa, interceptación [24, 23] y seguimiento de UAV enemigos, la capacidad de reaccionar ante amenazas dinámicas resulta fundamental. En estos escenarios, el sistema no solo debe desplazarse, sino también detectar, seguir y mantener el objetivo dentro del campo de visión, corrigiendo su trayectoria en tiempo real incluso ante maniobras evasivas.



Figura 2.7. Drones utilizados para operaciones militares. Fuente: [22].

En consecuencia, los métodos basados exclusivamente en pilotaje humano o en trayectorias rígidamente prefijadas resultan insuficientes cuando

el sistema debe desenvolverse en entornos no estructurados y frente a objetivos dinámicos. Además, este enfoque incrementa la dependencia de pilotos con un alto nivel de entrenamiento, capaces de mantener decisiones rápidas y precisas durante situaciones de elevada exigencia operativa. Por ello, este Trabajo de Fin de Grado se orienta hacia estrategias de percepción visual y control de navegación que permitan reducir esa dependencia, automatizar tareas críticas de seguimiento e intercepción y dotar al UAV cazador de una respuesta robusta, precisa y adaptable en tiempo real.

2.3. Métodos de Control de Navegación para UAVs

Para alcanzar este nivel de autonomía, el control del vehículo no puede entenderse como una acción aislada, sino como parte de un sistema integrado de Guiado, Navegación y Control, habitualmente denominado GNC (*Guidance, Navigation and Control*). Este marco organiza el comportamiento inteligente de la aeronave en tres niveles jerárquicos y complementarios: el guiado determina la estrategia de la misión y la trayectoria a seguir; la navegación estima el estado del UAV y su posición relativa respecto al entorno; y el control transforma esta información en comandos específicos de actitud y empuje. En conjunto, el sistema GNC permite cerrar el lazo entre la percepción sensorial y la respuesta dinámica del dron, un factor crítico cuando el escenario de operación cambia en tiempo real y el objetivo se mueve de forma continua.

Dentro de este esquema, el control de navegación actúa como el núcleo de decisión táctica. Su función principal es procesar la información proveniente del módulo de navegación para generar comandos de movimiento que permitan cumplir objetivos complejos, como la persecución de una presa. En este tipo de misiones, el éxito no depende únicamente de una detección correcta, sino de la capacidad del algoritmo para orquestar un movimiento fluido que mantenga al objetivo dentro del campo de visión sin comprometer la estabilidad del vuelo.

Dada la complejidad de este desafío, la literatura científica ha evolucionado hacia diversos paradigmas de control, diferenciados principalmente por la fuente de los datos y la arquitectura del procesamiento 2.8. Estos enfoques se pueden clasificar en tres grandes categorías: los métodos basados en localización global, los enfoques fundamentados en aprendizaje profundo (*Deep Learning*) y los sistemas de control basado en imagen (*Visual Servoing*). Cada una de estas metodologías presenta un compromiso distinto entre precisión, coste computacional y robustez, factores que determinan su idoneidad para operar en sistemas empotrados de recursos limitados. A continuación, se describen los fundamentos y aplicaciones de estos paradigmas

para situar las soluciones propuestas en la literatura y preparar el marco metodológico del trabajo.

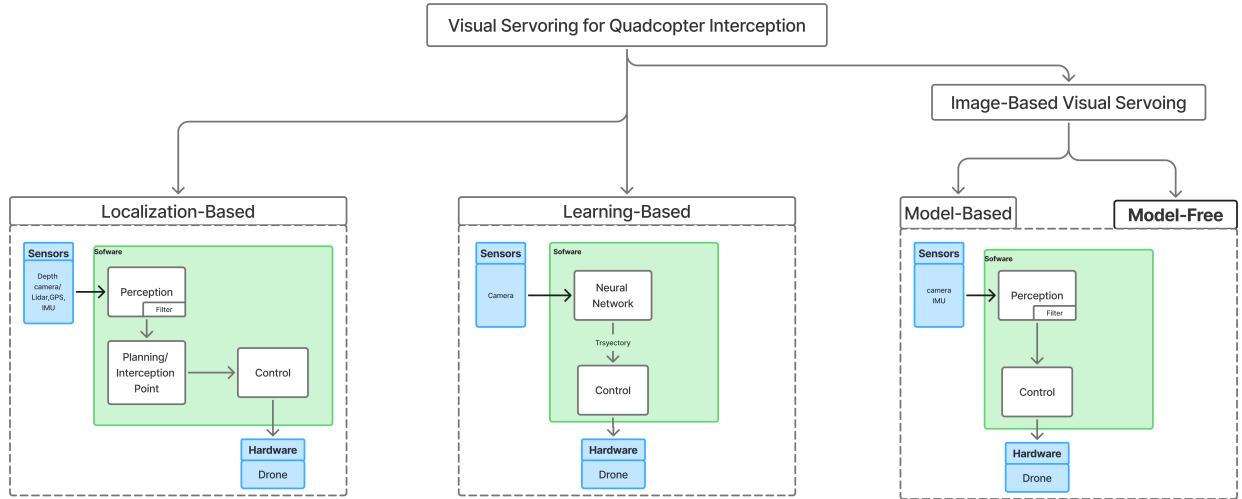


Figura 2.8. Diferencias entre los tipos de algoritmos de Visual Servoing. (Imagen de Elaboración Propia)

2.3.1. Tipos de control de navegación

Los **métodos basados en localización**, encuadrados habitualmente en el paradigma *Position-Based Visual Servoing* (PBVS), tienen como objetivo reconstruir la posición y la orientación del blanco en el espacio cartesiano a partir de la información visual [26]. Esta estimación tridimensional permite definir trayectorias de interceptación con un mayor grado de precisión, algo especialmente útil en misiones de seguimiento e interceptación con cuadricópteros. Dentro de esta familia, algunos trabajos recurren a controladores predictivos no lineales que optimizan la acción de control en un horizonte temporal, teniendo en cuenta tanto la evolución prevista del objetivo como las restricciones dinámicas del UAV. Otros enfoques emplean lazos de control sobre velocidades angulares para hacer coincidir la línea de visión con el vector de velocidad del vehículo, o modelan la trayectoria del blanco en tres dimensiones para escoger puntos de interceptación más convenientes. De igual modo, también se han propuesto arquitecturas de doble lazo PID que aprovechan los estados tridimensionales estimados del objetivo para seguir blancos terrestres o aéreos[31].

En definitiva, la principal fortaleza de PBVS radica en su capacidad para prever el movimiento del objetivo y determinar puntos de interceptación.

No obstante, su rendimiento queda condicionado por una estimación fiable de la profundidad y por una calibración rigurosa de la cámara. Asimismo, una parte importante de los trabajos revisados complementa la información visual con sensores adicionales, como radares o cámaras estéreo y de profundidad, con el fin de mejorar la reconstrucción espacial y aumentar la robustez del sistema.

Los **métodos basados en aprendizaje** delegan una parte importante del control en modelos entrenados con datos, normalmente redes neuronales capaces de inferir comandos de navegación de forma directa a partir de la información visual. Dentro de esta familia destacan enfoques de aprendizaje supervisado y, especialmente, técnicas de aprendizaje por refuerzo, que han mostrado una notable capacidad de adaptación frente al ruido sensorial y ante entornos dinámicos o poco estructurados. Esta flexibilidad resulta especialmente atractiva en tareas de interceptación, donde el dron debe reaccionar en tiempo real a cambios imprevistos de la escena o de la trayectoria del objetivo [33, 34]. No obstante, su aplicación práctica sigue presentando dificultades relevantes: requieren grandes volúmenes de datos y tiempos de entrenamiento elevados, suelen quedar condicionados por los escenarios utilizados durante el aprendizaje y continúan arrastrando problemas importantes de transferencia entre simulación y mundo real. A ello se suma la cuestión de la generalización, ya que un modelo entrenado en condiciones concretas puede perder fiabilidad cuando cambian el entorno, la dinámica del vehículo o las perturbaciones externas [35].

Por su parte, los **métodos basados en imagen**, agrupados bajo el paradigma *Image-Based Visual Servoing* (IBVS), utilizan directamente el error medido en el plano imagen para generar la acción de control. En lugar de reconstruir por completo la geometría tridimensional del objetivo, estos enfoques trabajan con características visuales como el desplazamiento del blanco respecto al centro de la imagen o la variación de su tamaño aparente. Esta decisión reduce la dependencia respecto a estimaciones geométricas completas y suele favorecer soluciones más ligeras y mejor adaptadas a plataformas embarcadas con recursos limitados [26, 27, 28].

2.3.2. Conceptos de Visual Servoing (IBVS)

Dentro de IBVS pueden distinguirse, a su vez, dos enfoques. El primero corresponde a los métodos basados en modelo, que incorporan de forma explícita la dinámica del cuadricóptero y la cinemática visual en el diseño del controlador. Estos métodos permiten respuestas más predictivas, un seguimiento más preciso y, en algunos casos, garantías formales de estabilidad, pero a cambio requieren un modelado más detallado del sistema y un mayor esfuerzo de ajuste [48, 27, 28]. El segundo enfoque es el de los métodos libres

de modelo, en los que el controlador abstrae gran parte de la dinámica interna del UAV y delega la estabilización básica en el autopiloto o en los lazos de bajo nivel ya disponibles. Esta aproximación reduce la carga computacional y simplifica la integración práctica, aunque puede ofrecer una menor capacidad de respuesta ante maniobras extremadamente agresivas o escenarios muy exigentes [29, 30, 32].

Por tanto, la elección entre PBVS, enfoques basados en aprendizaje e IBVS no depende únicamente de cuál resulte más sofisticado desde un punto de vista teórico, sino de cuál se adapta mejor al contexto de aplicación. PBVS aporta una representación espacial rica, pero exige una percepción 3D precisa; los métodos basados en aprendizaje ofrecen flexibilidad, aunque todavía presentan dificultades importantes de entrenamiento y transferencia al entorno real; e IBVS proporciona una relación especialmente favorable entre simplicidad, latencia y viabilidad en tiempo real. En problemas donde la percepción ya suministra detecciones en el plano imagen, los enfoques IBVS libres de modelo constituyen una opción especialmente coherente, ya que permiten conectar de forma directa la localización visual del objetivo con la generación de consignas de control sin introducir una complejidad geométrica o computacional excesiva. Esta observación explica que buena parte de la literatura combine percepción visual, estimación ligera y control reactivo, y sirve además como punto de partida para la metodología desarrollada posteriormente.

2.4. *Deep Learning*

El *Deep Learning* es una rama del aprendizaje automático basada en redes neuronales artificiales con múltiples capas, capaces de aprender representaciones jerárquicas directamente a partir de los datos. A diferencia de otros enfoques más tradicionales, reduce la necesidad de diseñar manualmente las características de entrada, ya que el propio modelo ajusta internamente niveles sucesivos de abstracción durante el proceso de entrenamiento [14].

El desarrollo de esta disciplina ha estado impulsado por la disponibilidad de grandes volúmenes de datos, el aumento de la capacidad de cómputo y la mejora de los métodos de entrenamiento. Gracias a ello, el *Deep Learning* se ha consolidado como un enfoque general para abordar tareas de clasificación, regresión, predicción y generación de datos en contextos muy diversos. Esta flexibilidad explica su relevancia dentro del aprendizaje automático actual y justifica su presencia como base conceptual en numerosos sistemas inteligentes [14].

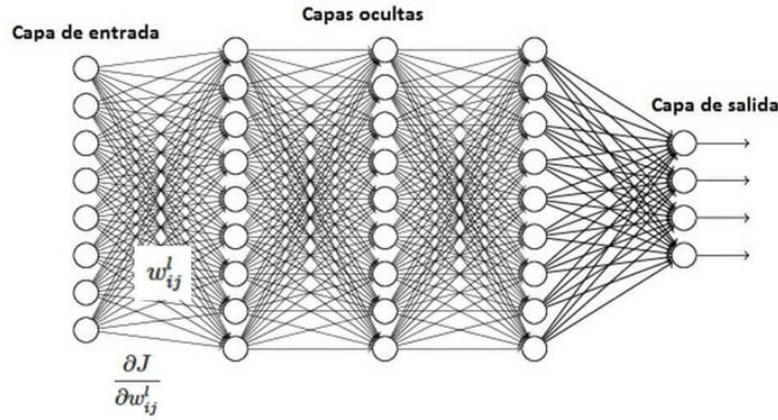


Figura 2.9. Esquema general de una red neuronal multicapa. Fuente: [8].

2.5. Visión por Computador

La visión por computador, también conocida como visión artificial o visión computarizada, es una rama de la inteligencia artificial que agrupa técnicas y herramientas orientadas a adquirir, procesar e interpretar información visual procedente del entorno. A partir de imágenes o secuencias de vídeo, estos sistemas pueden extraer características relevantes que facilitan la detección, la clasificación y el análisis de objetos, escenas y movimientos.

Este campo integra conocimientos de procesamiento digital de imágenes, estadística, aprendizaje automático y robótica con el propósito de desarrollar sistemas capaces de comprender el contenido visual y actuar a partir de él, de forma autónoma o asistida. En este sentido, la visión por computador busca dotar a las máquinas de capacidades de percepción útiles para interpretar su entorno y apoyar procesos posteriores de decisión y control.

Sus orígenes suelen situarse en la década de 1960, cuando comenzaron a explorarse métodos para que los ordenadores reconocieran formas sencillas en imágenes bidimensionales. En aquella etapa inicial, las aplicaciones se centraban en tareas relativamente básicas, como la detección de bordes o el reconocimiento de patrones geométricos, debido tanto a las limitaciones del hardware disponible como a la escasez de datos con los que entrenar y validar los modelos.

En las últimas décadas, el incremento de la capacidad de cómputo, la disponibilidad de grandes conjuntos de datos y el desarrollo de arquitecturas basadas en redes neuronales convolucionales (CNN) han impulsado notablemente la evolución del área. Gracias a ello, hoy es posible abordar

tareas complejas como la detección de objetos en tiempo real, la segmentación semántica, el reconocimiento facial o el seguimiento automático, y han surgido detectores especialmente influyentes como Faster R-CNN y la familia YOLO [37].

Entre sus aplicaciones destacan la conducción autónoma, la videovigilancia, la robótica, la inspección automática y la interacción persona-máquina. En el contexto de este trabajo, la visión por computador se empleará como base perceptiva para la detección y el seguimiento visual del dron objetivo, lo que enlaza de forma natural con la detección de objetos tratada en el apartado siguiente.

2.5.1. Métodos de detección de objetos

El objetivo fundamental de la detección de objetos consiste en localizar y clasificar regiones de interés dentro de una imagen, normalmente representadas mediante cajas delimitadoras. La detección debe resolver simultáneamente qué objetos aparecen y dónde se encuentran. Por ello, se trata de un problema más exigente desde el punto de vista computacional y metodológico, ya que combina percepción semántica y localización espacial en una misma tarea.

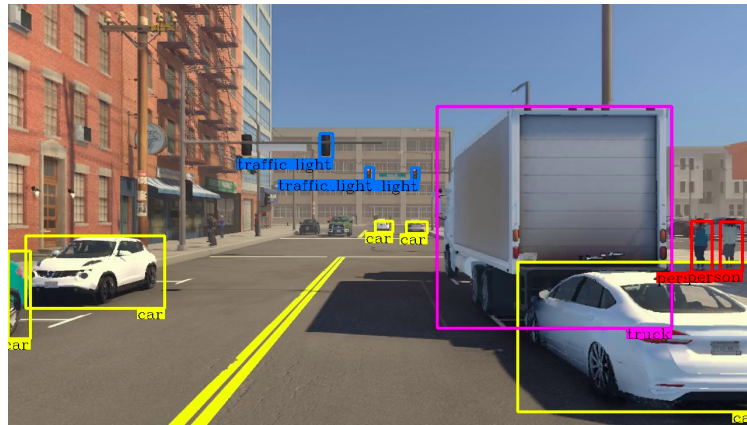


Figura 2.10. Ejemplo ilustrativo de detección de objetos mediante cajas delimitadoras. Fuente: [15].

Antes de la consolidación de los enfoques basados en redes profundas, una parte importante de los métodos de detección se apoyaba en estrategias de ventana deslizante. En ese marco, modelos como *Deformable Parts Model* (DPM) recorrían la imagen evaluando un gran número de subregiones candidatas, lo que permitía detectar objetos a distintas escalas y posiciones, pero a costa de una carga computacional muy elevada. Aunque estos enfoques marcaron una etapa importante en la evolución de la detección de

objetos, su coste de inferencia y su limitada capacidad de adaptación a escenas complejas hicieron evidente la necesidad de arquitecturas más eficientes [38].

La transición hacia el *Deep Learning* se consolidó con la familia R-CNN, que introdujo un esquema de dos etapas. En su formulación original, R-CNN generaba primero un conjunto de regiones potencialmente relevantes mediante búsqueda selectiva, después extraía características con una red convolucional para cada una de esas regiones y, finalmente, realizaba la clasificación. Este planteamiento supuso un avance decisivo en precisión, pero seguía penalizado por el hecho de procesar cada región de manera separada y por depender de un mecanismo externo de propuesta de regiones. Faster R-CNN mejoró este pipeline al integrar la generación de propuestas dentro de la propia red mediante una *Region Proposal Network* (RPN). Con ello, el sistema gana coherencia y velocidad respecto a R-CNN, aunque sigue siendo un detector de dos etapas y, por tanto, mantiene una latencia relativamente mayor que otras alternativas [37, 36].

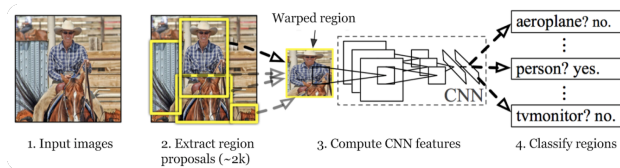


Figura 2.11. Esquema simplificado del funcionamiento de R-CNN, [37].

Frente a esta familia de detectores, los métodos de una sola etapa reformulan la detección como un problema unificado de predicción directa. En lugar de separar explícitamente la propuesta de regiones y la clasificación, una única red procesa la imagen completa y estima de forma simultánea las categorías, las puntuaciones de confianza y las coordenadas de las cajas delimitadoras. Esta simplificación reduce la complejidad del pipeline y favorece tiempos de inferencia mucho menores, una propiedad especialmente relevante en plataformas embarcadas y en escenarios dinámicos como la intercepción aire-aire, donde la frecuencia de actualización del detector resulta tan importante como su precisión.

2.5.2. YOLO

La familia YOLO (*You Only Look Once*) constituye el ejemplo más influyente de detector de una sola etapa basado en regresión unificada. Su aportación principal consiste en reformular la detección de objetos como un problema de predicción directa sobre la imagen completa, evitando la separación explícita entre propuesta de regiones y clasificación. De este modo, la

red estima de forma conjunta la localización de las cajas delimitadoras y la probabilidad de las clases presentes en la escena.

Desde un punto de vista operativo, estos detectores combinan tres ideas fundamentales: una partición implícita de la escena en regiones de predicción, la estimación directa de cajas delimitadoras y puntuaciones de confianza mediante regresión, y una etapa final de depuración basada en métricas de solapamiento como la intersección sobre unión (IoU) y en procedimientos como *Non-Maximum Suppression* (NMS). Esta formulación permite generar detecciones con latencias reducidas y con una frecuencia de actualización compatible con aplicaciones de tiempo real.

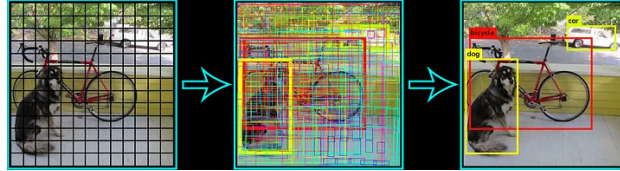


Figura 2.12. Esquema simplificado del funcionamiento de Yolo, [40].

La literatura reciente ha consolidado a YOLO como una de las familias más extendidas en visión embarcada gracias a su equilibrio entre precisión, velocidad y facilidad de despliegue. En problemas de detección aérea, seguimiento e intercepción, este compromiso resulta especialmente valioso porque la utilidad del detector depende no solo de su exactitud espacial, sino también de su capacidad para suministrar observaciones consistentes a alta frecuencia. Además, al predecir directamente magnitudes continuas asociadas a la localización del blanco, estos modelos proporcionan una base adecuada para etapas posteriores de seguimiento visual, estimación geométrica e inferencia de distancia relativa [39].

2.6. Geometría Projectiva

Una vez presentadas las técnicas de percepción visual y detección de objetos, resulta necesario introducir el marco geométrico que permite interpretar esa información en términos espaciales y relacionarla con el movimiento del dron. En este contexto, la geometría proyectiva constituye la base teórica que conecta los puntos del espacio tridimensional con su representación en el plano imagen. En navegación visual, esta disciplina resulta esencial porque la cámara no proporciona directamente una posición 3D del objetivo, sino una proyección bidimensional condicionada por la perspectiva, la orientación de la cámara y sus parámetros internos [54, 56, 55]. Antes de introducir el modelo geométrico de la cámara, conviene partir de la cámara estenopeica

o modelo *pinhole*, ya que constituye la aproximación más simple y utilizada para explicar cómo se forma una imagen.

2.6.1. Cámara estenopeica

La cámara estenopeica, también conocida como modelo *pinhole*, es una idealización geométrica utilizada para describir cómo se forma una imagen a partir de la luz procedente del entorno. En este modelo, los rayos luminosos atraviesan un único orificio y se proyectan sobre un plano imagen, lo que permite relacionar de forma sencilla el espacio tridimensional con su representación bidimensional.

Sus antecedentes históricos se asocian a la cámara oscura y a las primeras observaciones sobre la propagación rectilínea de la luz. Desde la Antigüedad se estudiaron fenómenos de proyección similares, y con el tiempo estos principios dieron lugar a dispositivos estenopeicos cada vez más elaborados, primero con fines de observación y más adelante también con aplicaciones fotográficas y didácticas [51].

Desde un punto de vista funcional, su funcionamiento se basa en que solo una pequeña fracción de la luz procedente de cada punto de la escena atraviesa el orificio, lo que genera una imagen invertida sobre el plano de proyección. Así, el sistema no mide directamente la profundidad ni las coordenadas espaciales del entorno, sino su proyección en la imagen. Esta idea resulta fundamental para comprender cómo la posición, el tamaño aparente y la perspectiva de los objetos quedan codificados visualmente.

Tradicionalmente, la cámara estenopeica se construía de forma muy simple, mediante una caja opaca, un pequeño estenopo y una superficie fotosensible, como papel o película fotográfica, por lo que las exposiciones solían ser largas y el control de entrada de luz era manual. En la actualidad, ese mismo principio puede aplicarse también sobre dispositivos con sensores digitales, como los CCD, y sigue utilizándose con fines educativos, artísticos y experimentales. De este modo, puede compararse una versión histórica basada en captura analógica con variantes modernas apoyadas en tecnología digital [51].

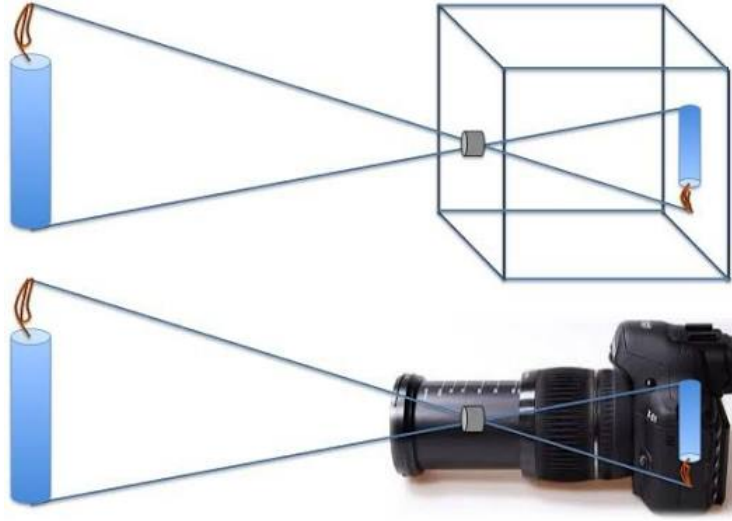


Figura 2.13. Funcionamiento simplificado de una cámara estenopeica. Fuente: [51].

2.6.2. Modelo geométrico de la cámara

A partir de esta idealización, la proyección de un punto del mundo puede representarse de forma compacta mediante la relación entre parámetros extrínsecos, parámetros intrínsecos y coordenadas del punto observado. En forma homogénea, esta idea suele resumirse como:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \cdot [R|t] \cdot \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} \quad (2.1)$$

donde la matriz intrínseca recoge parámetros como la distancia focal y el punto principal, mientras que la transformación extrínseca describe la posición y orientación de la cámara respecto al entorno. Aunque se trata de un modelo idealizado, resulta suficientemente expresivo para describir la mayor parte de los fenómenos geométricos relevantes en visión por computador [50].

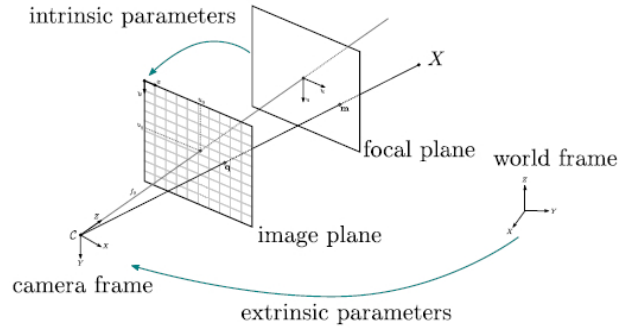


Figura 2.14. Esquema simplificado del modelo de cámara estenopeica. Fuente: [52].

2.6.3. Usos en navegación visual

En la literatura sobre *visual servoing* y seguimiento de UAV, la geometría proyectiva se emplea principalmente con tres objetivos. En primer lugar, permite calibrar el sistema de visión y caracterizar propiedades como el campo de visión, el punto principal o las distorsiones introducidas por la óptica. En segundo lugar, proporciona la base para relacionar variables medidas en píxeles con magnitudes útiles para estimación y control, como la dirección de visión, la escala aparente del objetivo o su posición relativa. En tercer lugar, sirve de soporte para compensar el efecto de la actitud del vehículo y para diseñar controladores que actúan a partir del error visual observado [48, 27].

Por ello, incluso en enfoques IBVS que evitan una reconstrucción tridimensional completa, la geometría proyectiva sigue siendo una herramienta conceptual relevante. Más que un bloque aislado de reconstrucción 3D, actúa como un lenguaje común que conecta calibración, percepción y control dentro de los sistemas de navegación guiados por cámara [32].

2.7. Métodos de Control

En robótica aérea, el control puede definirse como el conjunto de algoritmos encargados de manipular las variables de entrada del sistema, como la velocidad de los motores o las consignas de actitud, para lograr que las variables de salida del vehículo, entre ellas su posición, velocidad y orientación, sigan un comportamiento deseado. Su papel es fundamental para mantener la estabilidad frente a perturbaciones externas y para garantizar que el UAV cumpla de forma autónoma los objetivos de la misión.

2.7.1. Sistemas de Lazo Abierto vs. Lazo Cerrado

La clasificación más básica de los sistemas de control depende de la existencia o no de retroalimentación entre la salida y la acción de control [59, 60].

- **Control de lazo abierto (*open-loop*)**. La señal de control se genera sin tener en cuenta el estado real de la salida. En un UAV, esto equivaldría a enviar una orden de movimiento sin verificar si el vehículo ha respondido realmente como se esperaba. Este enfoque resulta poco adecuado para navegación autónoma, ya que no corrige errores ni compensa perturbaciones como el viento o pequeñas variaciones dinámicas del sistema.
- **Control de lazo cerrado (*closed-loop*)**. El sistema mide continuamente la salida, la compara con una referencia deseada y calcula un error a partir de esa diferencia. A continuación, el controlador actualiza la acción de mando para reducir dicho error. Este es el paradigma predominante en navegación autónoma basada en visión, ya que permite cerrar el lazo entre la percepción visual y la respuesta dinámica del dron. En este contexto, la referencia puede asociarse, por ejemplo, a mantener el objetivo centrado en la imagen, mientras que la salida medida corresponde a la posición observada del blanco en el plano imagen.

2.7.2. Controlador PID

Dentro de los mecanismos de control por retroalimentación, el controlador PID (*Proportional-Integral-Derivative*) destaca como una de las soluciones clásicas más utilizadas por su sencillez, su bajo coste computacional y su eficacia práctica en numerosos sistemas dinámicos. Su idea central consiste en calcular la acción de control a partir de la evolución temporal del error entre una referencia deseada y la salida medida del sistema [59, 60].

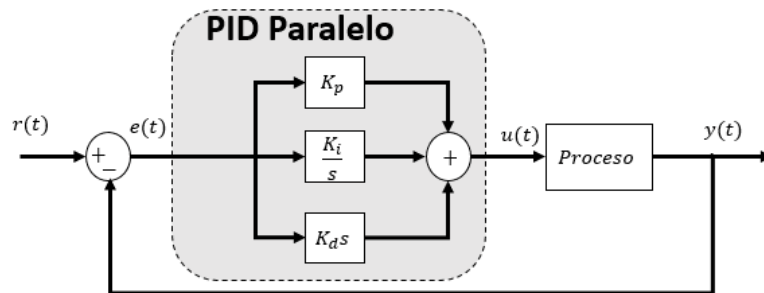


Figura 2.15. Esquema del funcionamiento de un PID., [58].

Si se define el error como:

$$e(t) = r(t) - y(t) \quad (2.2)$$

donde $r(t)$ es la referencia deseada y $y(t)$ es la salida observada, entonces la señal de control generada por un PID puede expresarse como:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (2.3)$$

Esta formulación combina tres contribuciones complementarias:

- **Término proporcional (K_p).** Proporciona una respuesta inmediata en función del error actual. Si el objetivo está alejado de la referencia, la acción de control aumenta de forma proporcional para corregir esa desviación con rapidez.
- **Término integral (K_i).** Acumula el error a lo largo del tiempo. Su función principal es eliminar el error residual o estático, de modo que el sistema no se quede permanentemente cerca de la referencia sin llegar a alcanzarla completamente.
- **Término derivativo (K_d).** Actúa sobre la velocidad de cambio del error. Su efecto puede interpretarse como un amortiguamiento que anticipa la tendencia del sistema y ayuda a reducir oscilaciones o respuestas demasiado bruscas.

Desde un punto de vista funcional, el término proporcional aporta rapidez de reacción, el integral mejora la precisión final y el derivativo contribuye a suavizar la dinámica transitoria. La combinación adecuada de estas tres acciones permite construir un controlador que responda con suficiente rapidez sin comprometer la estabilidad del sistema.

Por estas razones, el PID sigue siendo una elección frecuente en plataformas embebidas que deben compartir recursos con módulos de percepción e inferencia en tiempo real. Frente a enfoques de control más complejos, ofrece una implementación ligera y una integración sencilla con los lazos internos del autopiloto, lo que explica su amplia presencia en aplicaciones de seguimiento visual de UAV.

2.8. Estimación de Estado y Filtrado

En sistemas de navegación visual, las observaciones extraídas por el detector suelen estar afectadas por ruido, retardos y pérdidas puntuales. Por ello, la literatura incorpora con frecuencia técnicas de estimación y filtrado que aportan coherencia temporal a la información perceptiva antes de utilizarla en el lazo de control. Entre las soluciones más extendidas destacan el filtro de Kalman y los métodos de suavizado recursivo, como el EMA, por su equilibrio entre coste computacional y capacidad para atenuar fluctuaciones de la señal.

2.8.1. Origen y Evolución del Filtro de Kalman

El filtro de Kalman es un método de estimación recursiva formulado por Rudolf E. Kalman en 1960 para inferir el estado de sistemas dinámicos lineales a partir de medidas afectadas por ruido. Su interés radica en que combina la predicción proporcionada por un modelo dinámico con la información de los sensores, obteniendo estimaciones más estables y consistentes que las medidas directas por sí solas [42, 43, 44].

Con el tiempo, esta formulación se ha extendido a variantes no lineales, como el filtro de Kalman extendido (EKF), muy utilizadas en robótica móvil y aérea. En navegación visual con UAV, además, el filtrado suele integrarse con modelos dinámicos y estrategias predictivas para compensar retardos de percepción, suavizar detecciones y mantener la coherencia temporal del seguimiento.

2.8.2. Fundamentos y Mecanismo Recursivo

Desde un punto de vista operativo, el filtro funciona de manera recursiva y en tiempo real. En cada instante combina una etapa de predicción, basada en el modelo del sistema, con una etapa de actualización, en la que incorpora la nueva medida disponible. Esta estructura lo hace especialmente apropiado para plataformas embebidas, donde interesa limitar el coste computacional y no almacenar todo el historial de observaciones.

El sistema se representa mediante un modelo de espacio de estados:

$$\mathbf{x}_k = \mathbf{F}_k \mathbf{x}_{k-1} + \mathbf{B}_k \mathbf{u}_k + \mathbf{w}_k \quad (2.4)$$

$$\mathbf{z}_k = \mathbf{H}_k \mathbf{x}_k + \mathbf{v}_k \quad (2.5)$$

En estas ecuaciones, $\mathbf{x}_k$ representa el estado del sistema, $\mathbf{z}_k$ la observación disponible, $\mathbf{F}_k$ la dinámica del modelo y $\mathbf{H}_k$ la relación entre el estado y la medida. Los términos $\mathbf{w}_k$ y $\mathbf{v}_k$ modelan, respectivamente, el ruido del proceso y el ruido de medida.

2.8.3. Aplicación en Persecución Visual y Alta Velocidad

En sistemas de persecución visual, el filtrado de Kalman aporta continuidad temporal a las detecciones, atenúa el ruido de la medida y permite anticipar a corto plazo el movimiento del objetivo. Estas propiedades resultan especialmente relevantes cuando la información visual llega con retardos, presenta oclusiones parciales o debe combinarse con la dinámica del interceptor dentro del lazo de control.

En la literatura reciente, este tipo de estimación se integra con estrategias de navegación visual para fusionar la información procedente de la cámara con el modelo dinámico del vehículo, lo que contribuye a mantener trayectorias de seguimiento más estables y a reducir el efecto de fluctuaciones rápidas en la percepción. De este modo, el filtrado no sustituye al controlador, sino que mejora la calidad de la señal sobre la que este actúa.

Entre sus aportaciones más relevantes en este contexto pueden destacarse las siguientes:

- **Continuidad temporal:** ayuda a mantener una estimación coherente del objetivo incluso ante pérdidas puntuales de detección.
- **Fusión entre modelo y medida:** combina la información visual con la predicción dinámica del sistema para reducir la sensibilidad al ruido.
- **Predicción a corto plazo:** facilita anticipar la evolución inmediata del blanco, algo especialmente útil en maniobras rápidas o agresivas.

2.8.4. Filtro EMA

El filtro EMA (*Exponential Moving Average*), también denominado suavizado exponencial, es un método recursivo de suavizado utilizado para reducir las fluctuaciones rápidas de una señal sin necesidad de almacenar un gran número de muestras previas. Su interés principal reside en que requiere muy poca memoria y un coste computacional muy bajo, por lo que resulta especialmente adecuado para sistemas embebidos y aplicaciones de control en tiempo real [45, 46, 47].

Desde un punto de vista funcional, el EMA actúa como un filtro paso bajo: atenúa las variaciones de alta frecuencia asociadas al ruido y conserva la tendencia general de la señal. Por este motivo, se utiliza de forma habitual cuando se desea reducir la sensibilidad de una señal frente a pequeñas oscilaciones entre muestras consecutivas.

La formulación discreta del filtro EMA es la siguiente:

$$\hat{x}_k = \alpha x_k + (1 - \alpha)\hat{x}_{k-1} \quad (2.6)$$

donde x_k representa la medida actual, $\hat{x}_k$ la salida filtrada y $\hat{x}_{k-1}$ el valor filtrado del instante anterior. El parámetro α , con $0 < \alpha \leq 1$, determina el peso relativo entre la muestra nueva y el historial previo.

Esta expresión también puede escribirse como:

$$\hat{x}_k = \hat{x}_{k-1} + \alpha (x_k - \hat{x}_{k-1}) \quad (2.7)$$

Esta segunda forma resulta útil para interpretar el funcionamiento del filtro: la nueva salida se obtiene corrigiendo el valor filtrado anterior en función de la diferencia entre la medida actual y la estimación previa. Si α toma valores pequeños, el suavizado es mayor y la respuesta del filtro es más lenta; si α se aproxima a 1, la salida sigue más de cerca la medida y el efecto de suavizado disminuye.

La Figura 2.16 ilustra este comportamiento sobre una señal ruidosa. En ella puede apreciarse cómo la salida filtrada atenúa las oscilaciones rápidas de la señal original y conserva su tendencia global.

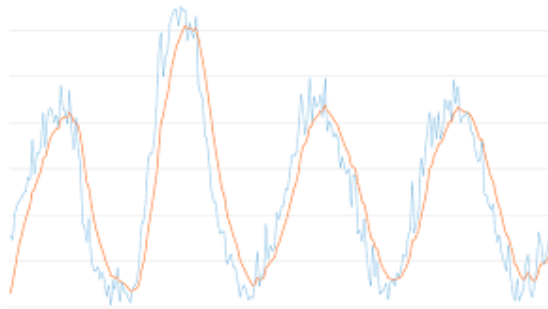


Figura 2.16. Efecto del filtro EMA sobre una señal ruidosa. Fuente: [47].

En consecuencia, el EMA constituye una técnica de suavizado sencilla, ligera y eficiente para señales discretas. Su bajo coste computacional y su

formulación recursiva hacen que resulte especialmente útil en aplicaciones donde se requiere atenuar ruido de alta frecuencia sin introducir una complejidad elevada.

2.9. Stack de Software y Hardware

La sofisticación de los sistemas robóticos modernos, especialmente en el ámbito de las aeronaves no tripuladas que operan en entornos dinámicos, exige infraestructuras de software que permitan la ejecución paralela de procesos heterogéneos. En este contexto, resulta imperativo emplear marcos de trabajo que actúen como capas de abstracción o middleware, garantizando la interoperabilidad entre los diferentes módulos de percepción, planificación y control mediante intercambios de datos deterministas y eficientes. Estas plataformas aseguran una comunicación estandarizada entre procesos, facilitando la integración de algoritmos complejos de navegación autónoma.

2.9.1. Intercambio de Datos en Robótica

La coordinación de un sistema robótico distribuido requiere mecanismos capaces de mover datos entre procesos de forma ordenada, predecible y compatible con las necesidades temporales de cada módulo. En el caso de los UAVs, esta comunicación resulta especialmente crítica, ya que los bloques de percepción, estimación, planificación y control deben compartir información de manera continua para mantener una respuesta coherente durante el vuelo. Dentro de este marco se revisan tres alternativas representativas: DDS, MQTT y ROS, cada una con un enfoque distinto respecto a la organización de la red, la fiabilidad de los mensajes y su adecuación al control en tiempo real.

DDS como Middleware Orientado a Datos

Data Distribution Service (DDS) es un estándar definido por la *Object Management Group* para sistemas distribuidos en los que la fiabilidad, el determinismo y la disponibilidad son requisitos de elevada importancia. Su diseño se apoya en una arquitectura descentralizada de publicación-suscripción, donde los productores y consumidores de información pueden descubrirse e intercambiar datos sin depender de un servidor central. Esta ausencia de un punto único de fallo mejora la tolerancia del sistema ante caídas de nodos o problemas puntuales de red, una propiedad especialmente valiosa en robótica móvil, vehículos autónomos y aplicaciones aeroespaciales.

Uno de los elementos más relevantes de DDS es la configuración de políticas de Calidad de Servicio (QoS). A través de ellas es posible ajustar parámetros como la latencia tolerada, la prioridad de los mensajes, la persistencia de la información y la fiabilidad de la entrega. Esta capacidad proporciona un control muy fino sobre el comportamiento temporal de la comunicación; sin embargo, también incrementa la complejidad de diseño. La selección incorrecta de las políticas QoS puede afectar al rendimiento global, por lo que su uso exige conocimiento técnico avanzado y una etapa de configuración cuidadosa.

MQTT como Protocolo Ligero basado en Broker

Message Queuing Telemetry Transport (MQTT) responde a una filosofía distinta. Se trata de un protocolo de mensajería ligero, concebido para redes con ancho de banda limitado y dispositivos con recursos computacionales reducidos, condiciones habituales en aplicaciones del Internet de las Cosas (IoT). A diferencia de los esquemas completamente descentralizados, MQTT introduce un *broker* central que recibe los mensajes de los publicadores y los distribuye a los suscriptores correspondientes. Esta solución simplifica la topología de comunicación y facilita la integración de dispositivos heterogéneos.

Aunque MQTT permite trabajar con diferentes niveles de fiabilidad en la entrega de datos, su diseño no está orientado a cerrar lazos de control de baja latencia ni a sincronizar estados cinemáticos con la precisión temporal que requiere un UAV durante maniobras rápidas. Por este motivo, su uso en plataformas aéreas resulta más adecuado para tareas auxiliares, como telemetría, monitorización remota o transmisión de información no crítica. En cambio, para algoritmos de seguimiento visual o interceptación, donde el controlador debe reaccionar de forma continua ante cambios del objetivo, MQTT no ofrece las garantías temporales necesarias.

2.9.2. Computación Distribuida Seleccionada

La implementación del sistema de navegación autónoma para la interceptación de aeronaves no tripuladas requiere de una infraestructura de software capaz de gestionar procesos concurrentes con un alto grado de desacoplamiento. En este sentido, se ha seleccionado el paradigma de Robot Operating System (ROS) como el middleware central, integrando componentes críticos como MAVROS para la interfaz con el hardware de vuelo.

La decisión de emplear ROS frente a otras alternativas técnicas se sustenta en la versatilidad de su ecosistema y la modularidad de su arquitectura. Si

bien soluciones como Data Distribution Service (DDS) ofrecen una gestión granular de las políticas de Calidad de Servicio (QoS) y un control estricto sobre el determinismo de la red, su implementación exige una especialización técnica que puede ralentizar las fases iniciales de prototipado. No obstante, la evolución hacia ROS 2 permite heredar las garantías de comunicación de DDS de forma subyacente, manteniendo una interfaz de programación de alto nivel que simplifica la integración de módulos complejos de percepción y control.

Esta arquitectura modular resulta fundamental para la naturaleza del proyecto, permitiendo que el nodo de proyección geométrica pueda intercambiar información de forma eficiente con el bucle de control de guiado. A diferencia de protocolos como MQTT, que destacan por su ligereza en aplicaciones de telemetría y entornos de Internet de las Cosas (IoT), estos carecen de las capacidades necesarias para el control de actuadores en tiempo real y la sincronización de estados cinemáticos exigida en maniobras de persecución.

El ecosistema de ROS proporciona, además, una serie de bibliotecas y herramientas de grabación de datos que agilizan el ciclo de validación experimental. Esta flexibilidad facilita el escalado del sistema, permitiendo la incorporación de nuevas capacidades de visión artificial o técnicas de filtrado estocástico sin comprometer la cohesión del diseño original. ROS se presenta como la opción más completa y robusta para implementar un sistema de navegación vectorial distribuido, garantizando tanto la eficiencia computacional como la cohesión arquitectónica del conjunto.

ROS como Núcleo de la Arquitectura Modular

En este proyecto, Robot Operating System (ROS) se adopta como el elemento vertebrador de la arquitectura distribuida, ya que permite coordinar procesos concurrentes sin imponer una estructura monolítica al sistema. Su función principal es actuar como middleware entre los algoritmos de alto nivel y los elementos asociados al vehículo, proporcionando una capa de abstracción que simplifica la comunicación, la supervisión y la integración de los distintos módulos. Esta capacidad resulta especialmente adecuada para navegación autónoma, donde percepción, estimación y control deben operar de forma simultánea y con dependencias bien definidas.

La organización interna de ROS se apoya en el concepto de nodo. Cada nodo representa una unidad lógica de ejecución encargada de una tarea específica, como procesar imágenes, calcular referencias de guiado, publicar estados del UAV o enviar comandos de control. Al tratarse de procesos independientes, pueden implementarse en distintos lenguajes de programación y evolucionar de manera separada, lo que incrementa la modularidad y facilita

la depuración. El intercambio continuo de información entre ellos se realiza principalmente mediante tópicos, canales asíncronos que siguen un modelo de publicación–suscripción.

Junto a los tópicos, ROS ofrece mecanismos adicionales como servicios y acciones. Los servicios son apropiados para interacciones puntuales de tipo petición–respuesta, por ejemplo al consultar un estado concreto del vehículo o modificar una configuración. Las acciones, en cambio, están pensadas para procesos de mayor duración que requieren seguimiento, retroalimentación o cancelación. No obstante, la arquitectura desarrollada en este trabajo se apoya principalmente en tópicos, ya que los flujos de navegación visual, telemetría y control necesitan actualizaciones continuas y una comunicación más adecuada para operación en tiempo real.



Figura 2.17. Robot Operating System. Fuente: [1].

Herramientas de Terminal para ROS

La operación de ROS Noetic se apoya de forma habitual en la línea de comandos, tanto durante la puesta en marcha del sistema como en las fases de diagnóstico y depuración. En lugar de constituir comandos aislados, estas utilidades forman parte del flujo de trabajo necesario para iniciar la infraestructura de comunicación, lanzar nodos, consultar el estado de ejecución y comprobar el contenido de los mensajes intercambiados. La Tabla siguiente resume las instrucciones utilizadas con mayor frecuencia durante el desarrollo del proyecto.

Comando	Función dentro del flujo de trabajo
<code>roscore</code>	Inicializa el maestro de ROS y los servicios básicos que permiten registrar nodos y habilitar el descubrimiento entre procesos.

<code>roslaunch nombre_paquete nombre_nodo</code>	Ejecuta un nodo concreto incluido en un paquete previamente compilado, lo que resulta útil para pruebas individuales o depuración localizada.
<code>roslaunch nombre_paquete archivo.launch</code>	Lanza de forma coordinada varios nodos, parámetros y configuraciones definidos en un archivo <code>.launch</code> , facilitando el arranque de escenarios más complejos.
<code>rostopic list</code>	Muestra los nodos activos registrados en el sistema y permite comprobar rápidamente qué procesos se encuentran en ejecución.
<code>rostopic list</code>	Enumera los tópicos disponibles en la sesión actual, ayudando a identificar los canales de comunicación publicados por los distintos módulos.
<code>rostopic echo nombre_topico</code>	Visualiza en tiempo real los mensajes emitidos en un tópico específico, por lo que resulta esencial para verificar datos, formatos y posibles anomalías de comunicación.
<code>rosparam list</code>	Consulta el servidor de parámetros de ROS y permite revisar las variables de configuración cargadas durante la ejecución.

MAVROS como Interfaz entre ROS y el Autopiloto

MAVROS se emplea como la capa de interoperabilidad encargada de conectar la arquitectura ROS con MAVLink, protocolo de comunicación utilizado de forma habitual por autopilotos como PX4 y ArduPilot. Su función no se limita a transferir paquetes entre ambos entornos, sino que proporciona una traducción estructurada entre los mensajes, servicios y parámetros propios de ROS y las órdenes, estados y configuraciones que interpreta el controlador de vuelo. Gracias a esta abstracción, los módulos de percepción, estimación y control pueden operar sobre una interfaz uniforme sin depender de los detalles específicos del autopiloto utilizado.

Desde una perspectiva operativa, MAVROS establece un canal bidireccional entre los nodos desarrollados en ROS y el UAV, tanto en simulación como en una plataforma física. A través de esta conexión se recibe telemetría relativa a la posición, la velocidad, la actitud, el estado de vuelo y el modo de operación del vehículo. En sentido inverso, la misma interfaz permite enviar consignas de movimiento, comandos de misión y modificaciones de determi-

nados parámetros. Esta comunicación continua resulta crítica en navegación autónoma, ya que el lazo de control necesita actualizarse con información reciente del vehículo y generar nuevas referencias durante la ejecución.

Dentro del sistema propuesto, MAVROS actúa como punto de enlace entre la navegación visual y el sistema de vuelo. Los algoritmos de seguimiento obtienen datos organizados para estimar el comportamiento del UAV, mientras que las correcciones calculadas por el controlador se remiten al autopiloto mediante una interfaz normalizada. Esta separación entre la lógica de alto nivel y el hardware concreto aumenta la reutilización del software, facilita el mantenimiento y reduce la dependencia de una configuración aérea específica.

Otro elemento clave es su integración con el sistema de transformaciones espaciales de ROS, gestionado mediante `tf`. Esta funcionalidad permite coordinar marcos de referencia como `map`, `odom` y `base_link`, necesarios para interpretar la navegación del vehículo y relacionar las observaciones visuales con el movimiento del UAV. En escenarios con varios drones, la gestión de estos marcos requiere especial cuidado: si diferentes vehículos publican referencias con nombres comunes, pueden aparecer ambigüedades o solapamientos. Por ello, durante la integración fue necesario separar correctamente los datos publicados por cada UAV dentro del entorno de simulación.

En síntesis, MAVROS proporciona mucho más que una pasarela de comunicación con el autopiloto. Su incorporación permite desacoplar percepción, control y ejecución de vuelo dentro de una arquitectura distribuida, manteniendo al mismo tiempo modularidad, escalabilidad y portabilidad entre pruebas simuladas y aplicaciones sobre plataformas reales.

Justificación de la Arquitectura Adoptada

La selección de una arquitectura basada en ROS, MAVROS y MAVLink responde a criterios técnicos directamente relacionados con las necesidades del proyecto. En lugar de presentar estas ventajas como elementos aislados, la Tabla siguiente resume el papel que desempeña cada criterio dentro del diseño global del sistema.

Criterio	Aportación al sistema
Modularidad	La división en nodos independientes permite desarrollar, probar y mantener cada bloque de forma progresiva, reduciendo el acoplamiento entre percepción, estimación y control.

Escalabilidad	La estructura distribuida facilita incorporar nuevos sensores, módulos de control o múltiples UAV sin modificar de manera sustancial la arquitectura base.
Portabilidad	El uso de estándares consolidados como ROS y MAVLink permite trasladar la lógica desarrollada desde simulación hacia plataformas reales con cambios limitados.
Flexibilidad	La integración de MAVROS permite trabajar con distintos autopilotos y configuraciones de vehículo bajo una interfaz homogénea, tanto en pruebas simuladas como en escenarios reales.

En conjunto, esta arquitectura satisface los requisitos técnicos del proyecto y, además, favorece un proceso de desarrollo más ordenado y sostenible. La combinación de ROS, MAVROS y herramientas de supervisión como RViz ofrece un entorno adecuado para diseñar, observar y validar el comportamiento del sistema. Asimismo, deja abierta la posibilidad de incorporar sensores más avanzados, nuevos algoritmos de percepción y estrategias cooperativas entre múltiples UAV en entornos distribuidos.

2.9.3. Hardware de Procesamiento Embarcado

Además del software de percepción y control, la selección del hardware de procesamiento resulta determinante en un sistema UAV embarcado. El procesador debe ejecutar el detector visual, recibir imágenes de la cámara, publicar resultados en ROS, comunicarse con MAVROS y mantener suficiente margen temporal para que los controladores reaccionen sin introducir retardos excesivos. Por ello, no basta con escoger una plataforma capaz de ejecutar código general, sino que es necesario valorar su capacidad real para inferencia de redes neuronales, su consumo energético y su integración con herramientas de aceleración.

Entre las alternativas consideradas se encuentran placas basadas en microcontrolador, ordenadores monoplaca de propósito general y plataformas de computación embebida con GPU. En concreto, se analizan Arduino, Raspberry Pi y NVIDIA Jetson, ya que representan tres niveles de capacidad de procesamiento habituales en proyectos de robótica: control electrónico básico, computación Linux de bajo coste e inferencia acelerada.

Arduino

Arduino es una plataforma de prototipado electrónico basada en microcontroladores, ampliamente utilizada para leer sensores, generar señales de control, activar actuadores y construir sistemas embebidos sencillos. Su

principal fortaleza es la simplicidad: ofrece bajo consumo, arranque rápido, facilidad de programación y una integración directa con entradas y salidas digitales o analógicas. Por ello, resulta especialmente útil para tareas de bajo nivel, como capturar medidas simples, accionar periféricos o servir como placa auxiliar dentro de un sistema robótico.



Figura 2.18. Arduino Uno. Fuente de referencia visual: [62].

Sin embargo, su arquitectura no está pensada para ejecutar algoritmos de visión por computador ni modelos de *Deep Learning*. La memoria disponible, la frecuencia de trabajo y la ausencia de sistema operativo completo limitan su uso en tareas que requieren capturar vídeo, procesar imágenes en tiempo real y ejecutar un detector como YOLO. En el contexto de este proyecto, Arduino podría emplearse como apoyo para electrónica auxiliar, pero no como computador principal de percepción y navegación visual.

Raspberry Pi

Raspberry Pi es un ordenador monoplaca de bajo coste que ejecuta sistemas operativos Linux y permite integrar cámaras, redes, almacenamiento y librerías de programación de forma más flexible que un microcontrolador. Esta plataforma resulta atractiva en prototipos educativos y robóticos porque combina tamaño reducido, amplia comunidad, facilidad de conexión con periféricos y capacidad suficiente para ejecutar tareas generales de control, adquisición de datos y visión clásica.



Figura 2.19. Raspberry Pi. Fuente de referencia visual: [63].

Su desventaja principal aparece cuando la carga de trabajo se desplaza hacia inferencia de redes neuronales en tiempo real. Aunque una Raspberry Pi puede ejecutar modelos ligeros o apoyarse en aceleradores externos, no integra una GPU NVIDIA compatible con CUDA y TensorRT. Esto reduce el margen disponible para ejecutar YOLO de forma estable mientras se mantienen activos ROS, MAVROS, filtrado y control. Por tanto, aunque es una alternativa compacta y económica, su rendimiento resulta menos adecuado para el objetivo de seguimiento visual en tiempo real planteado en este trabajo.

NVIDIA Jetson

NVIDIA Jetson es una familia de plataformas de computación embebida orientadas a inteligencia artificial. A diferencia de Arduino y Raspberry Pi, estas placas integran CPU y GPU en un mismo módulo, lo que permite ejecutar modelos de *Deep Learning* optimizados mediante CUDA y TensorRT. Esta característica resulta especialmente relevante en sistemas autónomos que deben procesar imágenes localmente, reducir la latencia y evitar depender de un ordenador externo o de comunicaciones con la nube.

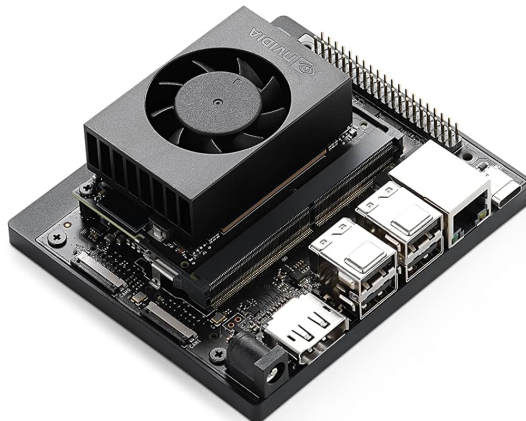


Figura 2.20. NVIDIA Jetson Orin Nano. Fuente de referencia visual: [64].

Como contrapartida, la familia Jetson requiere una configuración software más específica, especialmente cuando se trabaja con versiones concretas de CUDA, TensorRT, controladores y dependencias de ROS. No obstante, estas desventajas son asumibles cuando el requisito principal es ejecutar percepción visual acelerada en tiempo real dentro del propio UAV.

2.9.4. Selección de la NVIDIA Jetson Orin Nano

Dado que la percepción del sistema se basa en YOLO y debe ejecutarse en tiempo real, la familia NVIDIA Jetson resulta la opción más adecuada frente a plataformas sin aceleración GPU específica. Dentro de esta familia existen distintos niveles de rendimiento y consumo. La Tabla 2.3 compara tres alternativas representativas de la generación Orin [61].

Cuadro 2.3: Comparativa de consumo y rendimiento de módulos NVIDIA Jetson Orin considerados.

Modelo	Potencia configurable	Rendimiento IA máximo	Adecuación al UAV
NVIDIA Jetson Orin Nano	7–25 W	Hasta 67 TOPS	Mejor equilibrio entre inferencia acelerada, consumo moderado y facilidad de integración en una plataforma aérea ligera.
NVIDIA Jetson Orin NX	10–40 W	Hasta 157 TOPS	Mayor rendimiento para modelos más exigentes, pero con incremento de consumo, coste y necesidades de disipación térmica.
NVIDIA Jetson AGX Orin	15–60 W	Hasta 275 TOPS	Muy alta capacidad de cómputo, sobredimensionada para el objetivo del proyecto y menos adecuada para maximizar autonomía en un UAV multirroto.

Finalmente, se selecciona la **NVIDIA Jetson Orin Nano** como plataforma de procesamiento embarcado. Aunque los modelos Orin NX y AGX Orin ofrecen un rendimiento máximo superior, también incrementan el consumo, el coste y las necesidades de disipación térmica. En un UAV multirrotor, estos factores afectan directamente a la autonomía, al peso total y a la estabilidad térmica durante el vuelo. La Orin Nano proporciona suficiente capacidad para ejecutar el detector optimizado, manteniendo un rango de potencia más contenido y, por tanto, una mejor eficiencia energética para el escenario planteado.

Para completar la selección, la Tabla 2.4 resume las características principales de la NVIDIA Jetson Orin Nano consideradas más relevantes para este proyecto: capacidad de inferencia, recursos de CPU/GPU, memoria, almacenamiento, conectividad para cámara y consumo energético [61].

Cuadro 2.4: Características principales de la NVIDIA Jetson Orin Nano.

Característica	Especificación relevante
Rendimiento IA	Hasta 67 TOPS.
GPU	Arquitectura NVIDIA Ampere con 1024 núcleos CUDA y 32 núcleos Tensor.
CPU	Arm Cortex-A78AE v8.2 de 64 bits con 6 núcleos, 1.5 MB de caché L2 y 4 MB de caché L3.
Memoria	4 GB LPDDR5 de 128 bits, con ancho de banda de 102 GB/s.
Almacenamiento	Soporte para tarjeta SD y almacenamiento externo NVMe.
Vídeo y cámara	Decodificación H.265 hasta 1x 4K60, 2x 4K30, 5x 1080p60 u 11x 1080p30; dos conectores de cámara MIPI CSI-2 de 22 pines.
Conectividad e I/O	PCIe M.2, 4x USB 3.2 Gen2, USB-C, 1x GbE, DisplayPort 1.2 y cabecera de expansión de 40 pines.
Consumo	Potencia configurable entre 7 W y 25 W.
Dimensiones	103 mm x 90.5 mm x 34.77 mm.

2.9.5. Métodos de Aceleración de Inferencia

La ejecución de modelos de detección en una plataforma embarcada exige reducir al máximo el tiempo de inferencia sin comprometer la precisión necesaria para mantener el seguimiento visual. En este proyecto, el detector no actúa de forma aislada: sus salidas alimentan al filtro de Kalman, al suavizado temporal y a los controladores del UAV. Por tanto, cualquier

reducción de latencia en la red neuronal tiene un efecto directo sobre la estabilidad del lazo de control y sobre la capacidad de reacción ante maniobras rápidas del objetivo.

A partir del modelo YOLO entrenado, existen varias estrategias de aceleración aplicables antes de su despliegue en una Jetson. Entre las alternativas más relevantes se encuentran la cuantización a INT8, la ejecución mediante ONNX Runtime y la generación de un motor optimizado con TensorRT. Todas persiguen disminuir el coste computacional del detector, pero presentan compromisos distintos entre portabilidad, velocidad, complejidad de integración y posible pérdida de precisión.

Cuantización INT8

La cuantización INT8 consiste en representar las operaciones internas de la red neuronal mediante enteros de 8 bits en lugar de utilizar precisión de coma flotante. En comparación con FP16, esta estrategia reduce todavía más el tamaño de las operaciones y puede aumentar de forma significativa la velocidad de inferencia, especialmente en hardware preparado para ejecutar operaciones enteras de baja precisión. En una plataforma Jetson, este enfoque podría permitir tiempos de inferencia muy reducidos, del orden de pocos milisegundos, siempre que el modelo y la arquitectura aprovechen correctamente este tipo de cálculo.

Sin embargo, INT8 introduce un riesgo importante: al reducir de forma agresiva la precisión numérica, el detector puede perder sensibilidad ante objetos pequeños, cambios de escala o variaciones de iluminación. Para evitar esta degradación se requiere un proceso de calibración, en el que se proporciona al optimizador un conjunto representativo de imágenes reales o simuladas para ajustar los rangos numéricos de activaciones y pesos. En el caso de este proyecto, una mala calibración podría provocar cajas menos estables o incluso pérdidas puntuales del UAV objetivo, afectando directamente al cálculo del error visual. Por este motivo, INT8 se considera una alternativa interesante para trabajos futuros, pero no la opción más adecuada para una primera integración robusta.

Ejecución mediante ONNX Runtime

Otra posibilidad consiste en convertir el modelo entrenado a ONNX (*Open Neural Network Exchange*) y ejecutarlo mediante ONNX Runtime. Este enfoque tiene como principal ventaja la portabilidad: un modelo ONNX puede ejecutarse sobre distintas plataformas y proveedores de hardware, incluyendo CPU, GPU y aceleradores heterogéneos, sin depender de un único fabricante [67]. Esta característica resulta útil cuando se busca mantener el

mismo modelo entre equipos de desarrollo, estaciones de entrenamiento y distintos sistemas embarcados.

No obstante, esa generalidad también limita su rendimiento máximo en hardware NVIDIA. Aunque ONNX Runtime puede acelerar la inferencia respecto a una ejecución directa en PyTorch, no siempre explota con el mismo nivel de especialización los núcleos y librerías específicas de una Jetson Orin Nano. En consecuencia, podría ser una solución adecuada si la prioridad fuese la compatibilidad multiplataforma, pero no necesariamente la mejor elección cuando el objetivo principal es minimizar la latencia en una GPU NVIDIA concreta.

TensorRT y FP16

TensorRT es un SDK desarrollado por NVIDIA para optimizar y acelerar la inferencia de modelos de *Deep Learning* sobre GPU de NVIDIA. A diferencia del entrenamiento, donde la red neuronal ajusta sus pesos a partir de datos, la inferencia corresponde a la fase en la que un modelo ya entrenado procesa nuevas entradas y genera predicciones. Por tanto, en un sistema de navegación visual en tiempo real, TensorRT no se utiliza para entrenar el detector, sino para ejecutar de forma más eficiente el modelo desplegado sobre el hardware embarcado [65, 66].

Su funcionamiento se basa en transformar un modelo previamente entrenado en un motor de ejecución optimizado para la arquitectura concreta de la GPU. Durante este proceso se aplican técnicas como la fusión de capas, la selección de kernels más eficientes, la gestión optimizada de memoria y el uso de precisión reducida cuando el modelo y el hardware lo permiten. En este trabajo se emplea FP16, o media precisión de coma flotante, que reduce el coste computacional frente a FP32 sin exigir una calibración específica como ocurre con INT8.

2.9.6. Selección de TensorRT para el Despliegue Embarcado

Entre las alternativas consideradas, TensorRT con FP16 se selecciona como la opción más adecuada para el despliegue del detector en la Jetson Orin Nano. La elección responde a un compromiso entre rendimiento, estabilidad y complejidad de integración. Frente a ONNX Runtime, TensorRT ofrece una optimización más específica para la GPU NVIDIA utilizada. Frente a INT8, FP16 evita el proceso adicional de calibración y reduce el riesgo de degradar la precisión del detector, manteniendo al mismo tiempo una mejora notable de velocidad respecto a la ejecución original.

En el contexto de este proyecto, esta decisión resulta especialmente rele-

vante porque la plataforma NVIDIA Jetson Orin Nano integra CPU y GPU en un formato compacto y de bajo consumo, adecuado para aplicaciones embarcadas, pero limitado por potencia, temperatura y capacidad de cómputo frente a un ordenador de sobremesa. Al convertir el modelo de detección a un motor optimizado, TensorRT permite aprovechar mejor los recursos de la Jetson y mantener una tasa de fotogramas más estable durante la ejecución del nodo de percepción.

Además, al reducir el tiempo necesario para obtener cada predicción, se mejora la capacidad del sistema para reaccionar ante movimientos rápidos del objetivo, reducir retrasos acumulados en el lazo de control y sostener el seguimiento visual en condiciones compatibles con la operación en tiempo real. De esta forma, TensorRT no solo incrementa los frames por segundo máximos teóricos del detector, sino que también aporta margen temporal para que los nodos de ROS, MAVROS, filtrado y control puedan ejecutarse de manera concurrente sin saturar la plataforma embarcada.

2.9.7. Simulación

La simulación constituye una etapa esencial en el desarrollo de sistemas robóticos autónomos y, en particular, en aplicaciones con vehículos aéreos no tripulados. Antes de trasladar un algoritmo a una plataforma física, es necesario disponer de un entorno controlado en el que sea posible estudiar el comportamiento dinámico del UAV, comprobar la estabilidad del controlador y analizar la respuesta del sistema ante perturbaciones, cambios de trayectoria o variaciones en las condiciones de operación. De este modo, el simulador actúa como un punto intermedio entre el diseño teórico y la validación experimental sobre hardware real.

Los simuladores tridimensionales permiten reproducir, con distintos grados de fidelidad, elementos propios del entorno físico, como la gravedad, las colisiones, la respuesta cinemática, determinadas aproximaciones aerodinámicas y la lectura de sensores virtuales. Esta capacidad facilita la repetición de ensayos bajo condiciones equivalentes, el ajuste iterativo de parámetros y la evaluación de la solución en escenarios que serían costosos o arriesgados de reproducir directamente con un dron real. Por ello, la simulación no solo reduce riesgos materiales, sino que también acelera la depuración de los módulos de percepción, comunicación y control.

La elección del simulador resulta especialmente relevante cuando la arquitectura se apoya en herramientas como ROS, MAVROS y autopilotos compatibles con MAVLink, ya que la utilidad del entorno no depende únicamente de su calidad visual o de su motor físico, sino también de su capacidad para integrarse de forma estable con el resto del sistema. En este contexto, se han considerado distintas alternativas utilizadas habitualmente

en robótica y navegación autónoma, entre las que destacan AirSim, Webots y Gazebo.

AirSim

AirSim es un simulador orientado a vehículos autónomos, desarrollado por Microsoft sobre el motor gráfico Unreal Engine. Su principal ventaja reside en la elevada calidad visual de los escenarios y en la posibilidad de emular sensores habituales en plataformas aéreas, como cámaras RGB, LIDAR, IMU o GPS. Estas características lo convierten en una herramienta atractiva para investigaciones centradas en visión por computador, aprendizaje por refuerzo o navegación basada principalmente en percepción.

Sin embargo, en el marco de este proyecto presenta algunas limitaciones relevantes. Aunque puede conectarse con ROS, su integración no resulta tan directa como en otras plataformas más extendidas dentro de la robótica experimental, y la comunicación con autopilotos como PX4 o ArduPilot suele requerir una configuración adicional. A ello se añade la carga computacional asociada a Unreal Engine, que puede dificultar la ejecución simultánea de varios UAV y ralentizar el ciclo de pruebas cuando se pretende evaluar una arquitectura distribuida completa.

Webots

Webots, desarrollado por Cyberbotics, es una solución de código abierto ampliamente empleada en docencia e investigación. Ofrece una interfaz accesible, compatibilidad con distintos lenguajes de programación y soporte para una variedad amplia de sensores, actuadores y modelos robóticos. Además, dispone de integración con ROS, especialmente en entornos vinculados con ROS 2, por lo que constituye una plataforma versátil para el prototipado y la validación de aplicaciones de robótica general.

No obstante, su idoneidad disminuye cuando el objetivo es reproducir un sistema UAV completo gobernado por autopilotos reales. La conexión con plataformas como PX4 o ArduPilot no alcanza el mismo grado de madurez que en otros entornos más utilizados para simulación aérea con SITL, y su motor físico, aunque suficiente para numerosos casos, puede resultar menos adecuado para analizar maniobras exigentes, control fino de trayectoria o escenarios multivehículo con mayor realismo dinámico.

Gazebo

Gazebo se ha consolidado como uno de los simuladores más utilizados en robótica por su equilibrio entre realismo físico, flexibilidad y facilidad de integración con el ecosistema ROS. Su arquitectura basada en plugins, la posi-

bilidad de construir mundos modificables y la compatibilidad con múltiples sensores y modelos robóticos permiten ensayar algoritmos de navegación, percepción y control en condiciones reproducibles.

En proyectos centrados en UAV, esta combinación de fidelidad e interoperabilidad resulta especialmente valiosa. Gazebo puede conectarse con ROS mediante paquetes específicos y, a través de MAVROS y MAVLink, enlazarse con autopilotos como PX4 o ArduPilot. De este modo, los nodos de control pueden intercambiar consignas y telemetría con el vehículo simulado siguiendo una estructura cercana a la que se emplearía en una plataforma real, lo que facilita la automatización de experimentos y la reproducción de escenarios con varios agentes.

2.9.8. Simulador Seleccionado

Atendiendo a las necesidades del proyecto, Gazebo ha sido el simulador seleccionado para el desarrollo y la validación del sistema de navegación propuesto. La decisión se basa en su amplia implantación en robótica, su integración natural con ROS y su capacidad para representar entornos físicos configurables sin separar la simulación del resto de la arquitectura software. Mediante paquetes como `gazebo_ros` y `gazebo_plugins`, el simulador permite incorporar modelos de sensores, publicar información en tópicos ROS y conectar la dinámica del entorno con los nodos encargados de la percepción y el control.

Uno de los aspectos más relevantes de esta elección es la integración con MAVROS, que permite enlazar la simulación con un autopiloto virtual basado en PX4 o ArduPilot a través del protocolo MAVLink. Así, las consignas generadas por los nodos de control se envían al UAV virtual de forma análoga a como se enviarían a un dron real, mientras que la telemetría producida por el autopiloto se redistribuye al resto de módulos ROS. Esta continuidad entre percepción, control, autopiloto y simulación resulta fundamental para validar el comportamiento global del sistema sin abandonar un entorno seguro, repetible y controlado.

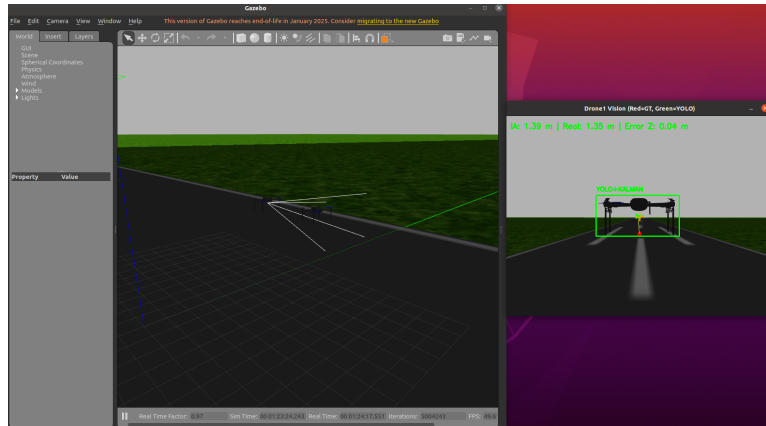


Figura 2.21. Uso de Gazebo con la cámara. (Imagen de Elaboración Propia)

En la práctica, Gazebo ha desempeñado un papel fundamental durante las fases de diseño, prueba y ajuste del sistema, ya que ha permitido analizar la respuesta del UAV ante cambios de trayectoria, maniobras de seguimiento y variaciones del entorno de forma repetible y sin riesgo para la plataforma física. Además, su compatibilidad con ROS, MAVROS, PX4 y ArduPilot facilita que los resultados obtenidos en simulación puedan trasladarse posteriormente a una configuración real con cambios limitados en la arquitectura de comunicación.

Aun así, la simulación no elimina por completo el denominado *sim2real gap*, es decir, las diferencias inevitables entre el entorno virtual y el comportamiento real del vehículo. Factores como las latencias, el ruido sensorial, las condiciones aerodinámicas o ciertas dinámicas no modeladas obligan a interpretar Gazebo como una etapa crítica de validación previa, pero no como un sustituto definitivo de las pruebas experimentales sobre hardware real.

Metodología de Trabajo

3.1. Enfoque Metodológico del Proyecto

Este capítulo describe el procedimiento seguido para desarrollar y validar el sistema propuesto. Su objetivo no es detallar todavía la programación concreta de cada nodo, script, sino establecer la lógica de trabajo empleada: preparación del entorno experimental, diseño de la simulación, definición del escenario multi-UAV, planteamiento de la percepción visual, estimación del estado y criterios de validación.

Las pruebas del sistema se plantean siguiendo una estrategia progresiva: primero se realizan en simulación y, una vez comprobado que el comportamiento es correcto, se considera su despliegue sobre un dron real. Esta decisión responde a una práctica habitual en robótica, donde los recursos hardware suelen ser limitados, costosos y sensibles a fallos durante las primeras etapas de desarrollo. Por ello, antes de utilizar una plataforma física, resulta necesario verificar en un entorno simulado que la comunicación, la percepción, el filtrado y el control funcionan de forma coherente y segura.

La metodología seguida se plantea de forma incremental. En primer lugar, se prepara un entorno reproducible basado en ROS, Gazebo, MAVROS y ArduPilot, cuya relación con el resto de módulos del sistema se sintetiza en el esquema general de la Figura 4.1. A continuación, se configura una simulación con dos UAVs diferenciados para poder evaluar tareas de seguimiento visual. Sobre esta base se incorporan los módulos de percepción y estimación, encargados de obtener una medida visual del objetivo y suavizarla antes de alimentar el controlador. Finalmente, el sistema se valida inicialmente mediante pruebas en simulación, observando la comunicación entre nodos, la coherencia de los marcos de referencia y la respuesta del UAV ante las consignas generadas, como paso previo a una posible validación posterior con hardware real.

De este modo, la metodología actúa como guía del proceso experimental: define qué herramientas se emplean, por qué se utilizan y cómo se encadenan las fases de trabajo. La implementación, en cambio, recoge cómo se materializa cada una de esas decisiones en archivos, modelos entrenados, tópicos,

parámetros y comandos concretos.

3.2. Entorno Experimental y Herramientas

En esta sección se describe el entorno base utilizado para el desarrollo del proyecto. ROS Noetic [9] se adopta como middleware principal por su capacidad para organizar el sistema en nodos independientes y facilitar la comunicación entre percepción, estimación, control y simulación. La preparación del entorno se documenta con el fin de garantizar la reproducibilidad del trabajo; no obstante, la explicación de los módulos software concretos desarrollados se reserva para el capítulo de implementación.

3.2.1. Preparación del Entorno Base

Para el desarrollo e integración de paquetes propios en ROS Noetic, resulta necesario disponer de un espacio de trabajo organizado, compatible con la estructura habitual de ROS y preparado para compilar los distintos módulos del proyecto. En este caso, se ha utilizado `catkin tools` en lugar del método tradicional basado en `catkin make`, ya que ofrece una gestión más flexible del proceso de compilación y facilita el control del entorno de desarrollo.

1. **Crear el directorio:** En primer lugar, se crea el directorio principal del espacio de trabajo, denominado `drone_ws`, junto con la carpeta `src`, destinada a almacenar el código fuente de los paquetes desarrollados o incorporados al sistema. Posteriormente, se inicializa el espacio de trabajo mediante las siguientes instrucciones:

```
mkdir -p ~/drone_ws/src
cd ~/drone_ws
catkin init
```

Con este procedimiento se establece la estructura básica del entorno de desarrollo y se deja preparada para su uso con `catkin tools`.

2. **Inicializar el espacio de trabajo:** Una vez creada e inicializada la estructura del workspace, se realiza una primera compilación con el objetivo de generar las carpetas `build` y `devel`, necesarias para construir los paquetes y configurar correctamente sus dependencias. Para ello, se emplea el siguiente comando:

```
catkin build
```

Durante esta etapa se generan los archivos asociados al proceso de compilación y se prepara el entorno que permitirá ejecutar los nodos, librerías y recursos vinculados al proyecto.

3. **Configurar el espacio de trabajo en el entorno:** Finalmente, para que ROS pueda localizar automáticamente los paquetes incluidos en `drone_ws` al abrir una nueva terminal, se añade el archivo de configuración del workspace al archivo `.bashrc` del usuario. Esta configuración se realiza mediante:

```
echo "source ~/drone_ws/devel/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

De este modo, cada nueva sesión de terminal cargará el entorno de ROS Noetic junto con los paquetes del espacio de trabajo `drone_ws`, evitando tener que ejecutar manualmente el comando `source` en cada ocasión.

3.2.2. Herramientas y Dependencias Empleadas

La correcta ejecución del sistema de navegación y seguimiento visual exige la integración de diversas bibliotecas especializadas que extienden las funcionalidades base de ROS Noetic. Para este proyecto, el entorno requiere herramientas que gestionen desde la comunicación con el hardware de vuelo hasta el procesamiento avanzado de imágenes y uso de matrices de transformación.

1. **Instalar MavROS y MavLink:** La instalación de *MavROS* y *MavLink* permite comunicar *ROS* con autopilotos como *ArduPilot* o *PX4*. Con ello se habilita el intercambio de telemetría, órdenes de navegación y datos de sensores dentro del sistema. El procedimiento se apoya en `wstool`, `rosdep` y `catkin build`, siguiendo la guía de *Intelligent Quads* [11]:

```
cd ~/drone_ws
wstool init ~/drone_ws/src

rosinstall_generator --upstream mavros | tee /tmp/mavros.rosinstall
rosinstall_generator mavlink | tee -a /tmp/mavros.rosinstall
wstool merge -t src /tmp/mavros.rosinstall
wstool update -t src
rosdep install --from-paths src --ignore-src --rosdistro \
'echo $ROS_DISTRO' -y

catkin build
```

2. **Clonar el paquete *iq_sim* de *ROS*:** El paquete *iq_sim* [13], desarrollado por *Intelligent Quads*, proporciona modelos y escenarios de simulación para UAVs en *ROS* y *Gazebo*. En este proyecto se utiliza como base para generar un entorno de prueba controlado, incluyendo datos de cámara simulada mediante mensajes `sensor_msgs/Image`. Su descarga se realiza con los siguientes comandos:

```
cd ~/drone_ws/src
git clone https://github.com/Intelligent-Quads/iq_sim.git
```

3. **Instalar *ArduPilot* y *MAVProxy*:** *ArduPilot* se emplea como plataforma de control de vuelo para reproducir en simulación el comportamiento de un autopiloto real y representar esquemas de navegación convencionales. Por su parte, *MAVProxy* funciona como estación de control en terminal y permite interactuar con el UAV mediante *MAVLink*. La instalación se realiza con las siguientes instrucciones:

```
cd ~
sudo apt install git
git clone https://github.com/ArduPilot/ardupilot.git
cd ardupilot
Tools/environment_install/install-prereqs-ubuntu.sh -y

. ~/.profile

sudo pip install future pymavlink MAVProxy

export PATH=$PATH:$HOME/ardupilot/Tools/autotest
export PATH=/usr/lib/ccache:$PATH
. ~/.bashrc
```

4. **Instalar el plugin de *Gazebo* para *APM*:** El plugin *ardupilot_gazebo* permite enlazar *Gazebo* con *ArduPilot* para simular el vuelo del UAV con mayor realismo físico. Esta integración facilita el intercambio de datos entre el simulador y el autopiloto durante las pruebas. Su instalación se basa en la guía de *Intelligent Quads* [12]:

```
cd ~
git clone https://github.com/khancyr/ardupilot_gazebo.git
cd ardupilot_gazebo
mkdir -p build
cd build
cmake ..
```

```
make -j4
sudo make install
```

```
echo 'source "$(pkg-config --variable=prefix gazebo) \
/share/gazebo/setup.sh"' >> ~/.bashrc
echo 'export GAZEBO_MODEL_PATH=~/.ardupilot_gazebo/models'\
>> ~/.bashrc
. ~/.bashrc
```

5. **Instalación de herramientas para el procesamiento de imagen y visión:** Para tratar las imágenes obtenidas en la simulación se incorporan *cv_bridge*, *vision_opencv* y *OpenCV*. Estas herramientas permiten convertir mensajes de *ROS* a estructuras de imagen procesables y aplicar operaciones básicas de visión artificial.

```
sudo apt-get install ros-noetic-cv-bridge \
ros-noetic-vision-opencv libopencv-dev
```

6. **Instalación de librerías para cálculo matricial y álgebra lineal:** Para las operaciones matemáticas del sistema se instala *Eigen*, una biblioteca ligera y eficiente para trabajar con matrices, vectores y transformaciones geométricas.

```
sudo apt-get install libeigen-dev
```

Siguiendo estos pasos se obtiene un workspace provisto con todos los paquetes necesarios para el desarrollo satisfactorio del proyecto.

3.3. Diseño Metodológico de la Simulación en Gazebo

En esta sección se presenta el planteamiento seguido para preparar y ejecutar la simulación de los UAVs mediante *Gazebo*. Este simulador físico permite representar entornos de prueba realistas e interactuar con nodos de *ROS*, ofreciendo un marco seguro y repetible para evaluar algoritmos de navegación, analizar la respuesta dinámica del vehículo y comprobar la robustez del sistema ante distintas condiciones de operación.

3.3.1. Modelo del Vehículo Aéreo y Escenario de Prueba

La simulación se construye a partir del paquete *iq_sim* [13], desarrollado por *Intelligent Quads*, ya que proporciona modelos de UAVs, sensores

virtuales y mundos de prueba preparados para trabajar con *ROS* y *Gazebo*. El modelo empleado recoge los elementos necesarios para representar el comportamiento del dron dentro del simulador:

- **Estructura física:** define la geometría, masa, enlaces y articulaciones del vehículo, permitiendo reproducir su respuesta dinámica durante el vuelo.
- **Sensores integrados:** incorpora sensores simulados, como *IMU*, *GPS* y barómetro, cuyas medidas se publican en tópicos de *ROS* para ser utilizadas por los nodos de control.
- **Plugins de control:** emplea complementos de *Gazebo*, como `gazebo_ros_imu` o `gazebo_ros_control`, que facilitan la comunicación entre la física simulada y los procesos de control ejecutados en *ROS*.

Además del modelo del UAV, se utiliza un archivo de mundo `.world` para definir el escenario de pruebas. En este caso, se opta por un entorno aéreo abierto y sin obstáculos fijos, de modo que las trayectorias puedan desarrollarse con mayor libertad y sea posible observar el comportamiento del sistema en condiciones controladas.

3.3.2. Procedimiento de Lanzamiento y Supervisión

La ejecución de la simulación se organiza mediante archivos `.launch`, encargados de cargar el modelo del dron, iniciar el entorno de *Gazebo* y activar los nodos necesarios para la comunicación con el sistema de control. Estos archivos se ajustan para incluir la carga automática del UAV, la inicialización de *MAVROS* como puente de comunicación y la conexión con *ArduPilot* a través de puertos UDP simulados.

Una vez iniciado el entorno, se activa el controlador correspondiente. Durante la ejecución se supervisa la respuesta del sistema mediante la visualización 3D en *Gazebo*, la consulta de tópicos con herramientas como `rqt_graph` y `rostopic echo`, y la inspección del estado de los nodos mediante `rostopic list`. Esta supervisión permite comprobar que la simulación se ejecuta correctamente y que los módulos de percepción, comunicación y control intercambian información de forma adecuada.

Para facilitar el lanzamiento de varias instancias de *MAVROS*, se emplea un archivo `apm.launch` genérico parametrizado mediante argumentos. De esta forma, cada UAV puede reutilizar la misma plantilla modificando únicamente el identificador del dron, el puerto UDP, el espacio de nombres y el archivo de configuración correspondiente.

```
1 <launch>
2 <!-- APM launch generico para multiples drones -->
```



```

3
4 <arg name="drone_id" default="1" />
5 <arg name="fcu_url" default="udp://127.0.0.1:14551@14555" />
6 <arg name="gcs_url" default="" />
7 <arg name="tgt_system" default="1" />
8 <arg name="tgt_component" default="1" />
9 <arg name="log_output" default="screen" />
10 <arg name="respawn_mavros" default="true"/>
11 <arg name="mavros_ns" default="/" />
12 <arg name="config_yaml" default="mavros/launch/apm_config_d$(
    arg drone_id).yaml" />
13
14 <include file="$(find iq_sim)/launch/mavros_node.launch">
15   <arg name="pluginlists_yaml" value="$(find mavros)/launch/
    apm_pluginlists.yaml" />
16   <arg name="config_yaml" value="$(arg config_yaml)" />
17   <arg name="mavros_ns" value="$(arg mavros_ns)" />
18   <arg name="fcu_url" value="$(arg fcu_url)" />
19   <arg name="gcs_url" value="$(arg gcs_url)" />
20   <arg name="respawn_mavros" value="$(arg respawn_mavros)" />
21   <arg name="tgt_system" value="$(arg tgt_system)" />
22   <arg name="tgt_component" value="$(arg tgt_component)" />
23   <arg name="log_output" value="$(arg log_output)" />
24 </include>
25 </launch>

```

Listing 3.1: Archivo `apm.launch` genérico para múltiples drones

3.4. Diseño del Escenario Multi-UAV

En esta sección se aborda la configuración de los dos UAVs empleados en el entorno simulado. La separación entre ambos vehículos permite definir de forma clara sus parámetros de comunicación, identificadores, espacios de nombres y archivos de lanzamiento, evitando conflictos entre tópicos, nodos y conexiones con el autopiloto. Esta organización resulta especialmente importante cuando se trabaja con varios drones dentro de una misma instancia de *Gazebo*, ya que facilita el seguimiento individual de cada plataforma y prepara el sistema para ejecutar pruebas coordinadas de navegación y control.

3.4.1. Definición del UAV Cazador

Para el primer UAV se tomó como base el modelo de dron incluido en *iq_sim* y se creó una variante propia denominada `drone1`, ubicada dentro de la ruta relativa `iq_sim/models`. Esta separación permite modificar el modelo sin alterar la plantilla original y facilita su identificación dentro del entorno de simulación. El modelo queda registrado mediante el archivo

`model.config`, donde se declara el nombre `drone1` y se enlaza con el archivo principal `model.sdf`.

Además del modelo físico, se generó una carpeta `config` asociada a `drone1`. En ella se incluyó un archivo `.yaml` con la información de la cámara, necesario para mantener organizada su configuración dentro de *ROS*. También se añadió el archivo `drone1_params.parm`, donde se define el identificador del vehículo para que *ArduPilot* pueda reconocerlo de forma independiente dentro de la simulación.

1 SYSID_THISMAV 1

Listing 3.2: Identificador asignado al primer UAV en `drone1_params.parm`

Por último, se creó el archivo `apm_config.d1.yaml` dentro de la ruta relativa `mavros/launch`. Este archivo es leído por `apm.launch` mediante el argumento `config.yaml` y permite lanzar el primer dron con marcos de referencia propios. En particular, se asigna el cuerpo del UAV al marco `drone1/base_link`, evitando solapamientos con otros vehículos simulados.

```
1 # global_position
2 global_position:
3   frame_id: "map"
4   child_frame_id: "drone1/base_link"
5   rot_covariance: 99999.0
6   gps_uere: 1.0
7   use_relative_alt: true
8   tf:
9     send: false
10    frame_id: "map"
11    global_frame_id: "earth"
12    child_frame_id: "drone1/base_link"
13
14 # imu_pub
15 imu:
16   frame_id: "drone1/base_link"
17   linear_acceleration_stdev: 0.0003
18   angular_velocity_stdev: 0.0003490659
19   orientation_stdev: 1.0
20   magnetic_stdev: 0.0
21
22 # local_position
23 local_position:
24   frame_id: "map"
25   tf:
26     send: true
27     frame_id: "map"
28     child_frame_id: "drone1/base_link"
29     send_fcu: false
```

Listing 3.3: Fragmento principal de `apm_config.d1.yaml` para `drone1`

3.4.2. Definición del UAV Objetivo

Para el segundo UAV se siguió el mismo procedimiento aplicado al UAV cazador, reutilizando la estructura de `drone1` como base y creando una nueva instancia denominada `drone2`. Esta decisión permitió mantener una organización homogénea entre ambos vehículos y reducir la duplicación de configuraciones, modificando únicamente los parámetros necesarios para que el UAV objetivo pudiera ejecutarse de forma independiente dentro de la simulación.

La principal diferencia funcional respecto al UAV cazador es que en `drone2` se eliminó la cámara, ya que el flujo de imagen solo debía generarse desde el vehículo perseguidor. El UAV objetivo conserva, por tanto, la estructura física básica del modelo, pero no incorpora el sensor visual utilizado por el módulo de percepción.

Los cambios realizados respecto a la configuración de `drone1` fueron los siguientes:

- **Nombre del modelo:** se cambió el identificador del modelo de `drone1` a `drone2` dentro del archivo `model.config`, manteniendo `model.sdf` como archivo de descripción física.
- **Identificador de ArduPilot:** se creó el archivo `drone2_params.parm` y se asignó el valor `SYSID_THISMAV 2`, para que el autopiloto pudiera distinguir el UAV objetivo del UAV cazador.
- **Configuración de MAVROS:** se generó el archivo `apm_config_d2.yaml` siguiendo la misma estructura que `apm_config_d1.yaml`, pero sustituyendo las referencias a `drone1/base_link` por `drone2/base_link` en los apartados de posición global, IMU y posición local.

Tras definir los modelos y sus configuraciones individuales, ambos UAVs fueron añadidos al archivo de mundo `runaway.world`. En este archivo se indica a *Gazebo* qué modelos deben cargarse en la escena y la posición inicial de cada uno. De esta forma, `drone1` se sitúa en el origen del entorno, mientras que `drone2` se inicializa desplazado 10 metros en el eje *x*, evitando que ambos vehículos aparezcan superpuestos al comenzar la simulación.

```
1 <model name="drone1">
2   <pose>0 0 0 0 0 0</pose>
3   <include>
4     <uri>model://drone1</uri>
5   </include>
6 </model>
7
8 <model name="drone2">
```

```

9   <pose>10 0 0 0 0 0</pose>
10  <include>
11    <uri>model://drone2</uri>
12  </include>
13 </model>

```

Listing 3.4: Inclusión de drone1 y drone2 en runaway.world

3.4.3. Inicialiación de la Simulación

Una vez definidos los modelos `drone1` y `drone2`, sus identificadores de *ArduPilot*, los archivos de configuración de *MAVROS* y su inclusión en el mundo de *Gazebo*, la simulación básica se ejecuta mediante varias terminales simultáneas. Para organizar este proceso se empleó *Terminator*, una herramienta que permite dividir una misma ventana en varios paneles de terminal. Esta característica resulta especialmente útil en entornos ROS, ya que facilita observar al mismo tiempo la salida de *Gazebo*, las dos instancias SITL de *ArduPilot* y los dos nodos de *MAVROS*, evitando cambiar continuamente entre ventanas independientes.

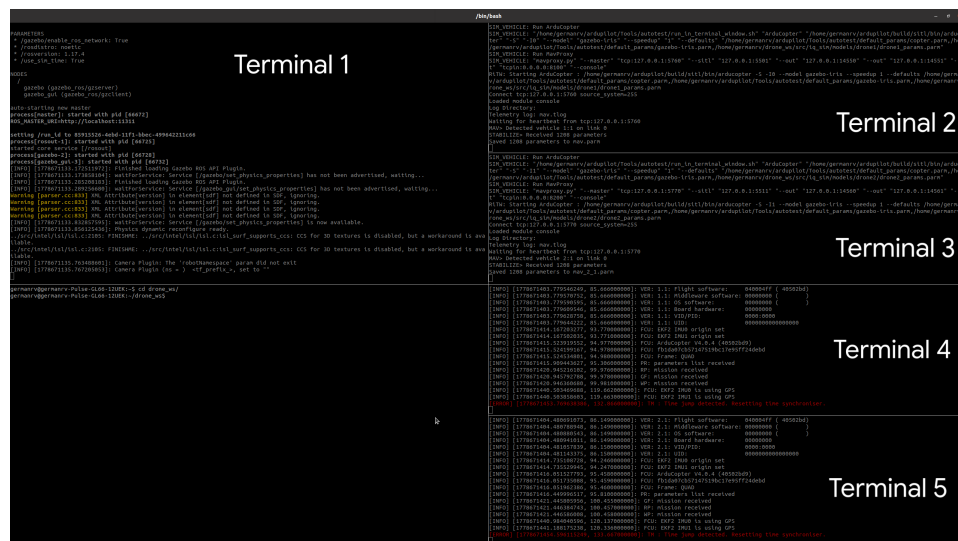


Figura 3.1. Panel básico de ejecución con varias terminales organizadas en *Terminator*. (Imagen de Elaboración Propia)

En la Figura 3.1 se muestra la disposición básica utilizada para lanzar el sistema. Cada panel corresponde a una tarea concreta y se mantiene visible durante toda la prueba, lo que permite detectar rápidamente errores de conexión, problemas en el arranque del autopiloto o fallos en la publicación de mensajes. La ejecución se organiza de la siguiente forma:

Terminal 1: lanzamiento del mundo de Gazebo

```

roslaunch iq_sim runway.launch

# Terminal 2: instancia SITL del primer UAV
sim_vehicle.py -v ArduCopter -f gazebo-iris --console -I0 \
  --out=tcpin:0.0.0.0:8100 \
  --add-param-file=$HOME/drone_ws/src/iq_sim/models/drone1/drone1_params.parm

# Terminal 3: instancia SITL del segundo UAV
sim_vehicle.py -v ArduCopter -f gazebo-iris --console -I1 \
  --out=tcpin:0.0.0.0:8200 \
  --add-param-file=$HOME/drone_ws/src/iq_sim/models/drone2/drone2_params.parm

# Terminal 4: conexion MAVROS del primer UAV
roslaunch iq_sim apm.launch drone_id:=1 \
  fcu_url:=udp://127.0.0.1:14551@14555 \
  mavros_ns:=/drone1 tgt_system:=1

# Terminal 5: conexion MAVROS del segundo UAV
roslaunch iq_sim apm.launch drone_id:=2 \
  fcu_url:=udp://127.0.0.1:14561@14565 \
  mavros_ns:=/drone2 tgt_system:=2

```

En la primera terminal se lanza el entorno de *Gazebo* mediante `runway.launch`, que carga el mundo preparado previamente y permite visualizar los modelos incorporados en `runaway.world`. Las terminales segunda y tercera arrancan dos instancias independientes de *ArduPilot* en modo SITL. La opción `-I0` se asocia al primer UAV y la opción `-I1` al segundo, mientras que los archivos `drone1_params.parm` y `drone2_params.parm` asignan los identificadores `SYSID_THISMAV 1` y `SYSID_THISMAV 2`, respectivamente. De este modo, cada autopiloto virtual queda vinculado al dron configurado en las subsecciones anteriores.

Las terminales cuarta y quinta ejecutan el archivo `apm.launch` para crear los dos puentes *MAVROS*. En cada caso se indica el identificador del dron mediante `drone_id`, el puerto UDP correspondiente mediante `fcu_url`, el espacio de nombres `mavros_ns` y el sistema objetivo mediante `tgt_system`. Esta separación permite que los tópicos, servicios y marcos de referencia de `drone1` y `drone2` se mantengan diferenciados dentro de ROS, coherentemente con los archivos `apm_config_d1.yaml` y `apm_config_d2.yaml` descritos anteriormente.

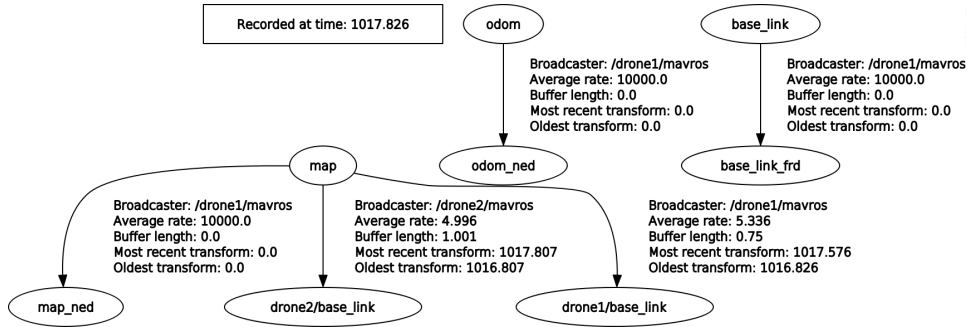


Figura 3.2. Marcos de referencia publicados durante la ejecución inicial con dos UAVs. (Imagen de Elaboración Propia)

La Figura 3.2 muestra los marcos de referencia publicados tras ejecutar los comandos anteriores. Esta comprobación permite verificar que cada UAV dispone de su propio *frame* asociado, como `drone1/base_link` y `drone2/base_link`, enlazado con el marco global `map`. De esta forma se confirma que la configuración definida en `apm_config_d1.yaml` y `apm_config_d2.yaml` separa correctamente las transformaciones de ambos vehículos y evita ambigüedades al incorporar posteriormente los nodos de percepción y control propuestos.

3.5. Metodología Para el Componente de Percepción

Una vez definido el entorno de simulación, se establece la base metodológica que permite transformar las observaciones de la cámara en información útil para el control. El objetivo principal del módulo de percepción es localizar el UAV objetivo en la imagen capturada por el UAV cazador y convertir esa detección en variables numéricas que puedan ser utilizadas por los módulos de filtrado y control.

La percepción se plantea como una etapa de detección visual basada en imágenes. La cámara simulada en *Gazebo* proporciona el flujo de entrada, que posteriormente es procesado por un detector de objetos.

3.5.1. Detector de UAVs

Como se indicó en la sección de estado del arte, los detectores basados en redes convolucionales pueden organizarse, de forma general, en modelos de dos etapas y modelos de una etapa. Los métodos de dos etapas, como la familia R-CNN, primero generan regiones candidatas en la imagen y posteriormente clasifican cada una de ellas. Este planteamiento puede ofrecer buenos resultados de detección, pero introduce una mayor complejidad y un

mayor tiempo de procesamiento, algo poco conveniente cuando la salida del detector debe alimentar un lazo de control en tiempo real [36, 37].

Por este motivo, en este trabajo se opta por un detector de la familia YOLO. A diferencia de los enfoques de dos etapas, YOLO realiza la detección en una única pasada sobre la imagen, proporcionando directamente la caja delimitadora del UAV objetivo. Esta salida resulta suficiente para la metodología propuesta, ya que a partir de ella se obtiene el centro del objetivo, el error visual respecto al centro de la cámara. Así, YOLO ofrece un compromiso adecuado entre rapidez, simplicidad de integración y precisión para una tarea de seguimiento visual de UAVs.

3.5.2. Dataset Utilizado

El conjunto de datos empleado para entrenar y evaluar el módulo de detección se construyó de forma progresiva, con el objetivo de adaptar el modelo a las distintas condiciones visuales presentes en el proyecto. Como punto de partida se realizó un *fork* del dataset público *Synthetic Drone*, disponible en Roboflow Universe: <https://universe.roboflow.com/drone-spegy/synthetic-drone>. Las imágenes de este primer dataset estaban anotadas originalmente en formato de segmentación, por lo que fue necesario revisar el conjunto y etiquetarlo manualmente mediante cajas delimitadoras adaptadas a la tarea de detección de UAVs. Este primer conjunto proporcionó una base inicial de imágenes sintéticas de drones, útil para comenzar el entrenamiento y comprobar la viabilidad del detector en una fase temprana.

Para realizar el etiquetado y organizar las distintas versiones del conjunto de datos se utilizó *Roboflow*, debido a su facilidad de uso y a la disponibilidad de herramientas de anotación asistidas por IA. Estas herramientas permitieron acelerar el etiquetado de múltiples imágenes, ya que generaban propuestas iniciales de cajas delimitadoras que posteriormente podían revisarse y corregirse manualmente. Además, la plataforma facilitó la gestión del dataset, la revisión visual de las anotaciones y la exportación en un formato compatible con el entrenamiento de modelos YOLO.

Posteriormente, se añadieron imágenes obtenidas directamente desde el entorno de simulación. Esta ampliación permitió aproximar el dataset a las condiciones específicas del sistema desarrollado, ya que las pruebas del controlador y de la percepción visual se realizan inicialmente en Gazebo. La incorporación de imágenes procedentes de la simulación resultó importante para que el modelo no dependiera únicamente de ejemplos genéricos, sino que también aprendiera características visuales cercanas a las que encontraría durante los ensayos simulados del UAV.

En una etapa final, el dataset se enriqueció con imágenes de drones reales. La recogida de estas muestras se realizó con el apoyo del Ejército del Aire, que facilitó el acceso a sus infraestructuras para la toma de datos en un entorno controlado. Estas muestras permitieron introducir mayor variabilidad visual en cuanto a iluminación, fondo, escala, orientación y apariencia del objetivo. Todas las imágenes reales incorporadas en esta última fase fueron etiquetadas manualmente, revisando la posición del dron en cada imagen y ajustando las cajas delimitadoras para asegurar una anotación coherente. Este proceso fue necesario para mejorar la calidad del entrenamiento y reducir la diferencia entre las imágenes sintéticas, las capturas simuladas y las condiciones reales de operación.

Como resultado, se obtuvo un dataset final de carácter mixto, compuesto por imágenes sintéticas, imágenes procedentes de la simulación e imágenes reales etiquetadas manualmente. Esta combinación permite entrenar un detector más robusto, capaz de reconocer el objetivo tanto en el entorno virtual como en situaciones más próximas a un escenario real. El conjunto final utilizado en este trabajo se encuentra organizado en Roboflow en el siguiente enlace: <https://app.roboflow.com/germanrv-uyz9j/synthetic-drone-d2ius/browse>.



Figura 3.3. Vista del dataset final organizado en Roboflow. (Imagen de Elaboración Propia)

El procedimiento seguido para llegar al dataset final puede resumirse en cuatro fases: primero, selección y *fork* del dataset sintético de partida;

segundo, incorporación de capturas obtenidas en simulación para adaptar el entrenamiento al entorno Gazebo; tercero, inclusión de imágenes reales de drones para aumentar la diversidad del conjunto; y cuarto, etiquetado manual y revisión de las nuevas muestras antes de entrenar los modelos YOLO comparados en la sección siguiente.

3.5.3. Geometría Projectiva aplicada a la Percepción Visual

La geometría proyectiva se incorpora en esta metodología para justificar cómo se interpretan las medidas obtenidas por el detector visual. En este trabajo, YOLO no proporciona directamente la posición tridimensional del UAV objetivo, sino una caja delimitadora en la imagen. A partir de esa caja se calculan el centro del objetivo y el error respecto al centro del fotograma. Por tanto, esta subsección no se plantea como un bloque independiente de reconstrucción 3D, sino como la base geométrica que permite entender por qué esas magnitudes visuales son válidas para el seguimiento.

Relación entre cámara, detección y error visual

El punto de partida es el modelo de cámara estenopeica (*pinhole camera model*). Este modelo idealiza la formación de la imagen suponiendo que los rayos de luz pasan por un único punto, denominado centro óptico o centro de proyección, antes de alcanzar el plano imagen. Aunque una cámara real incluye lentes y pequeñas deformaciones, esta aproximación permite describir de forma sencilla cómo un punto del espacio se transforma en una posición dentro de la imagen [56, 55].

En visión por computador suele utilizarse un plano imagen virtual situado delante del centro óptico. Esta convención evita invertir los signos de la imagen y facilita la definición del error visual que se presenta en las subsecciones siguientes. Así, cuando el centro de la caja detectada se desplaza respecto al centro del fotograma, dicho desplazamiento puede interpretarse como una desviación angular aproximada del objetivo respecto al eje óptico de la cámara [56, 54].

Si un punto del entorno se expresa en el sistema de referencia de la cámara como $P_c = (X_c, Y_c, Z_c)$, su proyección ideal sobre el plano imagen métrico viene dada por:

$$x = f \frac{X_c}{Z_c}, \quad y = f \frac{Y_c}{Z_c} \quad (3.1)$$

En esta expresión, X_c e Y_c representan la posición lateral y vertical del

punto respecto al eje óptico, Z_c representa la profundidad y f es la distancia focal. Esta relación conecta directamente con lo realizado en el módulo de percepción: si el UAV objetivo se desplaza lateralmente o verticalmente, cambia el centro de la caja detectada; si se acerca o se aleja, cambia su tamaño aparente. Por esta razón, el error visual permite alimentar un seguimiento reactivo sin reconstruir completamente la posición 3D del objetivo [55, 54].

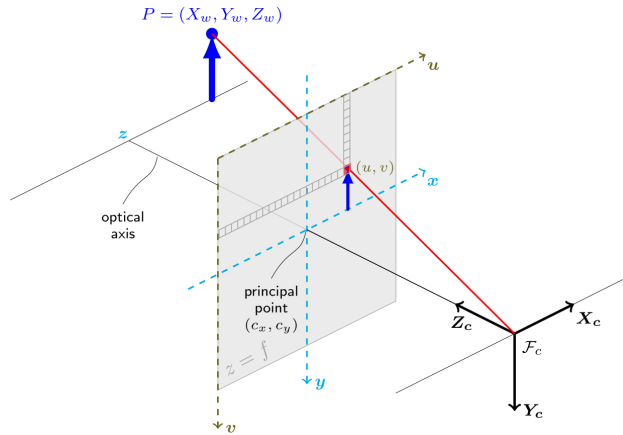


Figura 3.4. Esquema simplificado del modelo de cámara estenopeica. Fuente: [50].

Paso a coordenadas de píxel

Para utilizar esta relación dentro del sistema desarrollado, la proyección geométrica debe expresarse en coordenadas de imagen. Primero se obtienen las coordenadas normalizadas:

$$x_n = \frac{X_c}{Z_c}, \quad y_n = \frac{Y_c}{Z_c} \quad (3.2)$$

Estas coordenadas recogen la dirección del punto observado respecto a la cámara. Después, se transforman a coordenadas de píxel mediante los parámetros intrínsecos:

$$u = f_x x_n + c_x, \quad v = f_y y_n + c_y \quad (3.3)$$

Aquí, f_x y f_y son las distancias focales expresadas en píxeles, mientras que c_x y c_y definen el punto principal de la cámara. Esta conversión es la

que permite trabajar con las mismas magnitudes que entrega el detector: coordenadas de píxel de la caja delimitadora. Además, explica por qué en la metodología se normaliza el error visual respecto a $W/2$ y $H/2$: de esta forma, la señal de control depende menos de la resolución concreta de la cámara utilizada [54, 50].

La formulación completa de la proyección puede escribirse en coordenadas homogéneas como:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \cdot [R|t] \cdot \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} \quad (3.4)$$

En esta expresión, (X_w, Y_w, Z_w) son las coordenadas del punto en el sistema de referencia del mundo, $[R|t]$ representa la posición y orientación de la cámara respecto a ese mundo, K agrupa los parámetros intrínsecos y s es un factor de escala. La matriz intrínseca se expresa como:

$$K = \begin{bmatrix} f_x & \gamma & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (3.5)$$

Desde el punto de vista metodológico, esta formulación ayuda a separar dos aspectos: por un lado, la geometría de la cámara y sus parámetros internos; por otro, la posición relativa entre cámara y objetivo. En el sistema propuesto no se utiliza esta ecuación para calcular una pose 3D completa en cada instante, sino para fundamentar el uso de medidas 2D normalizadas como entrada al filtrado y al control [54, 56].

Efecto del FOV, la resolución y la distorsión

Otro aspecto importante para el seguimiento es el campo de visión o *Field of View* (FOV), que define la amplitud angular de la escena capturada por la cámara. Si d_x y d_y representan el tamaño del sensor en horizontal y vertical, el FOV puede aproximarse mediante:

$$FOV_x = 2 \arctan \left(\frac{d_x}{2f} \right), \quad FOV_y = 2 \arctan \left(\frac{d_y}{2f} \right) \quad (3.6)$$

Este parámetro afecta directamente al comportamiento observado. Un FOV amplio facilita mantener al UAV objetivo dentro de la imagen durante

maniobras rápidas, aunque reduce el tamaño aparente del objeto. Un FOV más estrecho ofrece más detalle sobre el objetivo, pero aumenta el riesgo de perderlo si se desplaza fuera del encuadre. Por ello, en este trabajo el error visual se trata como una medida relativa dentro del fotograma, en lugar de depender únicamente de valores absolutos en píxeles [55, 54].

Además, una cámara real no proyecta la escena de manera perfectamente ideal. La lente puede introducir distorsión radial y tangencial. La distorsión radial desplaza los puntos según su distancia al centro de la imagen, mientras que la distorsión tangencial aparece cuando lente y sensor no están perfectamente alineados. Estas deformaciones son relevantes porque afectan precisamente a las variables usadas por el sistema: centro de la caja, error visual y área aparente [50, 56].

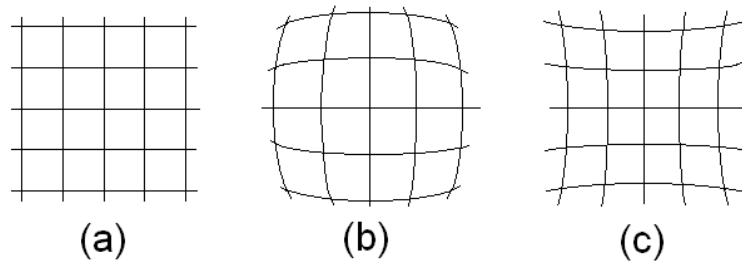


Figura 3.5. Imágenes de una pantalla de objeto rectangular mostradas con (a) ninguna distorsión; (b) distorsión radial; (c) distorsión tangencial. Fuente: [57].

Desde el punto de vista del sistema propuesto, esto implica que una misma posición real del objetivo podría producir un error visual ligeramente diferente según la zona de la imagen donde aparezca. Por este motivo, conviene trabajar con coordenadas normalizadas o con puntos corregidos mediante calibración antes de usar la medida en las etapas posteriores [50, 54].

3.5.4. Flujo General del Módulo de Percepción

El flujo metodológico del módulo comienza con la captura de una imagen desde la cámara del UAV cazador. Sobre dicha imagen se ejecuta el detector, que devuelve una caja delimitadora o *bounding box* asociada al UAV objetivo. Esta caja constituye la salida básica del detector y permite calcular tanto la posición del objetivo en el plano imagen como su tamaño aparente.

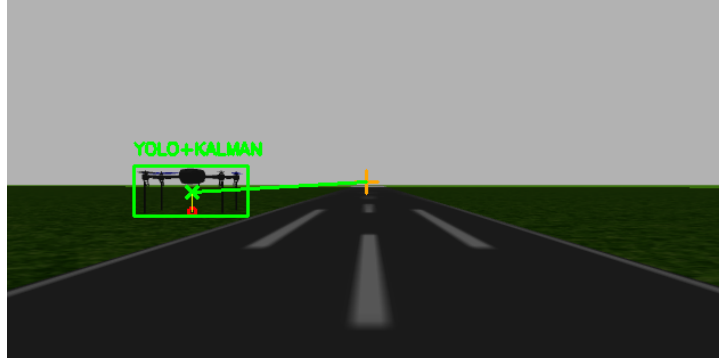


Figura 3.6. Representación del error visual entre el centro de la imagen y el UAV objetivo detectado. (Imagen de Elaboración Propia)

A partir de la caja delimitadora se obtiene el centro del objetivo. Si (x_{min}, y_{min}) y (x_{max}, y_{max}) representan las esquinas de la caja detectada, el centro se calcula como:

$$c_x = \frac{x_{min} + x_{max}}{2}, \quad c_y = \frac{y_{min} + y_{max}}{2} \quad (3.7)$$

Este punto se interpreta como una aproximación de la posición del UAV objetivo dentro de la imagen. Aunque no proporciona directamente una posición tridimensional real, sí permite obtener una señal visual suficiente para definir la dirección en la que debe corregirse el movimiento del UAV cazador.

3.5.5. Cálculo y Normalización del Error Visual

El error visual se define como la diferencia entre el centro del objetivo detectado y el centro de la imagen. Si W y H representan el ancho y alto del fotograma, el centro de la imagen viene dado por:

$$c_{img,x} = \frac{W}{2}, \quad c_{img,y} = \frac{H}{2} \quad (3.8)$$

Por tanto, el error horizontal y vertical en píxeles se expresa como:

$$e_x = c_x - c_{img,x}, \quad e_y = c_y - c_{img,y} \quad (3.9)$$

Esta normalización resulta especialmente importante porque no todas las cámaras empleadas trabajan con la misma resolución. En el entorno de simulación, la cámara configurada en *Gazebo* genera imágenes de 640×480 píxeles, mientras que la cámara considerada para el dron real trabaja con una resolución de 346×260 píxeles. Si el error se utilizara directamente en píxeles, una misma desviación relativa del objetivo produciría valores diferentes según la cámara utilizada.

Esta relación se representa gráficamente en la Figura 3.6, donde se observa cómo el desplazamiento del UAV objetivo respecto al centro de la imagen genera las componentes horizontal y vertical del error visual.

Para evitar esta dependencia directa de la resolución, el error se normaliza respecto a la mitad del ancho y de la altura de la imagen:

$$e_{x,norm} = \frac{e_x}{W/2}, \quad e_{y,norm} = \frac{e_y}{H/2} \quad (3.10)$$

Esta normalización permite expresar el error en una escala relativa. Un valor próximo a cero indica que el objetivo se encuentra centrado en la imagen, mientras que valores positivos o negativos indican desplazamientos hacia uno u otro lado del plano visual. De este modo, el controlador puede trabajar con una señal más homogénea y menos dependiente de la configuración concreta de la cámara.

3.6. Estimación de Distancia

Al plantear la estimación de distancia se valoró inicialmente emplear directamente el área de la caja delimitadora como señal de error. En ese enfoque, se fijaría un área deseada en la imagen y el controlador intentaría mantener la detección con ese tamaño: si el área medida fuese menor, el objetivo estaría más lejos; si fuese mayor, estaría demasiado cerca. Aunque esta solución es sencilla, el error queda expresado como una magnitud relativa de imagen y queda condicionado por el valor de área deseada elegido, la escala aparente del modelo y la estabilidad de la caja detectada.

Por ello, se consideró necesario transformar esa información visual en una distancia aproximada expresada en metros antes de entregarla al controlador. Esta decisión permite que el PID de distancia trabaje con una variable física, comparando la distancia estimada con una distancia deseada real de seguimiento. Además, desacopla el controlador del sensor concreto: si en el futuro se incorpora otro sistema capaz de medir distancia en metros, el PID de distancia no tendría que cambiar, ya que seguiría recibiendo una entrada con las mismas unidades.

A partir de esta necesidad, antes de adoptar el modelo geométrico final, se planteó el uso de un modelo de aprendizaje automático basado en regresión. Este modelo utilizaba principalmente el área normalizada de la caja delimitadora respecto al tamaño total de la imagen y predecía directamente la distancia al UAV objetivo en metros. Inicialmente parecía una opción mejor que usar el área como error directo, porque mantenía una entrada visual sencilla, evitaba fijar manualmente un área objetivo y proporcionaba una salida física compatible con el PID de distancia.

Sin embargo, al comparar esta alternativa con la estimación basada en el modelo geométrico de cámara, se observó que el método geométrico proporcionaba una estimación más precisa y coherente para el sistema. Al apoyarse en la distancia focal de la cámara y en las dimensiones reales aproximadas del UAV objetivo, la relación entre tamaño aparente y distancia quedaba definida de forma explícita, sin depender de que un modelo aprendiese dicha relación a partir de ejemplos concretos. Por este motivo, la alternativa basada en regresión fue descartada y se adoptó el planteamiento geométrico como solución final para alimentar el PID de distancia.

La idea implementada consiste en estimar una distancia aproximada entre la cámara del UAV cazador y el UAV objetivo utilizando la geometría proyectiva descrita anteriormente. Para ello, se aplica el modelo de cámara estenopeica no solo a la posición de un punto, sino también al tamaño aparente de un objeto cuyo tamaño real es conocido.

A partir de la relación geométrica básica, una dimensión real del objeto se proyecta en la imagen de forma proporcional a la distancia focal e inversamente proporcional a la profundidad:

$$x = f_x \frac{X_c}{Z_c}, \quad y = f_y \frac{Y_c}{Z_c} \quad (3.11)$$

En esta expresión, las variables se interpretan de la siguiente forma para el problema de seguimiento del UAV:

- Z_c representa la distancia real entre la cámara del UAV cazador y el UAV objetivo. Es la magnitud que se desea estimar.
- X_c e Y_c representan las dimensiones reales conocidas del UAV objetivo. En este trabajo se considera para la simulación un ancho aproximado de $X_c = 0.62$ m y una altura aproximada de $Y_c = 0.27$ m.
- x e y representan el tamaño del UAV objetivo en la imagen, medido en píxeles. En la práctica, estos valores corresponden al ancho y alto de la caja delimitadora entregada por YOLO, es decir, $x = w_{box}$ e $y = h_{box}$.
- f_x y f_y son las distancias focales de la cámara expresadas en píxeles. Estos parámetros se obtienen de la matriz intrínseca publicada por la cámara.

Como el objetivo es calcular la distancia Z_c , se despeja la expresión anterior. La estimación puede obtenerse usando el ancho de la caja delimitadora o usando su alto:

$$Z_{c,x} = \frac{f_x \cdot X_c}{x} \quad (3.12)$$

$$Z_{c,y} = \frac{f_y \cdot Y_c}{y} \quad (3.13)$$

La primera opción, basada en el ancho, resulta especialmente conveniente en este trabajo porque el UAV presenta una dimensión horizontal mayor que la vertical. Al proyectarse con más píxeles, el ancho de la caja suele ser una medida más estable frente a pequeñas variaciones de la detección. No obstante, la estimación basada en la altura también puede emplearse como comprobación adicional o como alternativa cuando el ancho de la caja no sea fiable.

Por ejemplo, si la distancia focal horizontal de la cámara simulada es $f_x = 500$ píxeles y YOLO detecta el UAV objetivo con un ancho de caja de $x = 100$ píxeles, tomando como ancho real $X_c = 0.55$ m se obtiene:

$$Z_c = \frac{500 \cdot 0.55}{100} = \frac{275}{100} = 2.75 \text{ m} \quad (3.14)$$

Su uso dentro del sistema consiste en transformar cada detección válida en una distancia aproximada, que posteriormente puede compararse con una distancia deseada para generar el error longitudinal del controlador. Debido a que la caja delimitadora puede fluctuar entre fotogramas, esta distancia debe interpretarse como una medida visual ruidosa y conviene suavizarla o validarla antes de utilizarla directamente en el control.

3.7. Filtrado

3.7.1. Filtro de Kalman

El filtro de Kalman es una técnica de estimación que permite suavizar medidas ruidosas y predecir brevemente cómo evoluciona una variable en el tiempo. En este proyecto no se utiliza para detectar el UAV, sino para hacer más estable la caja delimitadora entregada por YOLO antes de usarla en el cálculo del error visual y de la distancia [42, 43, 44].



Figura 3.7. Ejemplo de uso del filtro de Kalman sobre la detección visual.
(Imagen de Elaboración Propia)

De forma sencilla, el filtro mantiene un **estado** con la posición estimada del objetivo y su velocidad aparente. Para un caso bidimensional básico, este estado puede escribirse como:

$$\mathbf{x}_k = \begin{bmatrix} x_k & y_k & \dot{x}_k & \dot{y}_k \end{bmatrix}^T \quad (3.15)$$

En esta expresión, x_k e y_k representan la posición del objetivo en la imagen, mientras que $\dot{x}_k$ y $\dot{y}_k$ representan cómo cambia esa posición entre fotogramas. Para predecir el siguiente estado se utiliza la matriz de transición $\mathbf{F}_k$:

$$\mathbf{F}_k = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3.16)$$

La matriz $\mathbf{F}_k$ describe cómo evoluciona el estado: la nueva posición se calcula usando la posición anterior y la velocidad estimada. En cambio, la matriz de observación $\mathbf{H}_k$ indica qué parte del estado se mide realmente. Si la cámara solo mide la posición del objetivo, pero no su velocidad, puede expresarse como:

$$\mathbf{H}_k = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \quad (3.17)$$

También se emplean dos matrices para representar la incertidumbre del sistema:

$$\mathbf{Q}_k = \begin{bmatrix} \sigma_x^2 & 0 & 0 & 0 \\ 0 & \sigma_y^2 & 0 & 0 \\ 0 & 0 & \sigma_{\dot{x}}^2 & 0 \\ 0 & 0 & 0 & \sigma_{\dot{y}}^2 \end{bmatrix}, \quad \mathbf{R}_k = \begin{bmatrix} \sigma_{mx}^2 & 0 \\ 0 & \sigma_{my}^2 \end{bmatrix} \quad (3.18)$$

La matriz $\mathbf{Q}_k$ representa el ruido del modelo: cuanto mayor sea, menos se confía en la predicción. La matriz $\mathbf{R}_k$ representa el ruido de la medida: cuanto mayor sea, menos se confía en la detección de YOLO. Por tanto, estas matrices regulan el equilibrio entre seguir la predicción del movimiento o corregir con la nueva caja detectada.

La actualización se entiende de forma simple. Cuando llega una nueva detección, el filtro la compara con la predicción anterior. Si la caja de YOLO

es coherente, la estimación se corrige hacia esa medida. Si la caja cambia bruscamente, contiene ruido o desaparece durante unos instantes, el filtro confía más en la predicción. Así se obtiene una señal más suave y útil para el controlador.

3.7.2. Uso del Filtro de Kalman en el Sistema Propuesto

En este trabajo, el filtro de Kalman se utiliza directamente sobre la salida del detector YOLO. En lugar de filtrar únicamente el centro de la detección, se filtra la caja delimitadora completa. Por ello, el estado empleado en el seguimiento visual se define a partir de las cuatro coordenadas de la caja y de sus velocidades aparentes:

$$\mathbf{x}_k = \begin{bmatrix} x_{1,k} & y_{1,k} & x_{2,k} & y_{2,k} & \dot{x}_{1,k} & \dot{y}_{1,k} & \dot{x}_{2,k} & \dot{y}_{2,k} \end{bmatrix}^T \quad (3.19)$$

La medida proporcionada por YOLO está formada únicamente por las coordenadas observadas de la caja:

$$\mathbf{z}_k = \begin{bmatrix} x_{1,k} & y_{1,k} & x_{2,k} & y_{2,k} \end{bmatrix}^T \quad (3.20)$$

De este modo, el filtro estima no solo dónde se encuentra la caja en el fotograma actual, sino también cómo tiende a desplazarse entre imágenes consecutivas. Metodológicamente, se ha adoptado un modelo de velocidad constante con un intervalo de muestreo aproximado de $\Delta t = 0.05$. La matriz de transición propaga las cuatro coordenadas de la caja utilizando sus velocidades, mientras que la matriz de observación selecciona únicamente las coordenadas que realmente entrega el detector.

La aplicación práctica se organiza en tres estados. Cuando YOLO detecta el UAV objetivo, la caja observada corrige la predicción del filtro y se publica una detección válida. Cuando YOLO pierde temporalmente el objetivo, pero el filtro ya había sido inicializado, se utiliza la fase de predicción para mantener durante un número limitado de fotogramas una estimación aproximada de la caja. Finalmente, si la pérdida se prolonga demasiado, el seguimiento se reinicia y la medida se considera no válida. En la implementación desarrollada, este límite se fija en 15 fotogramas consecutivos sin detección.

Esta decisión se tomó porque las detecciones de YOLO pueden presentar saltos pequeños entre fotogramas o desaparecer durante instantes breves debido a cambios de escala, desenfoque, orientación del dron o pequeñas

oclusiones. Sin el filtro, estas variaciones se transmitirían directamente al cálculo del error visual y podrían generar consignas bruscas. Con Kalman, la caja delimitadora mantiene una evolución temporal más coherente y el sistema dispone de una predicción razonable cuando la detección falla de forma puntual.

3.7.3. Filtro EMA aplicado a la Caja de Detección

Además del filtro de Kalman, se utiliza un filtro EMA (*Exponential Moving Average*) sobre las cuatro coordenadas de la caja delimitadora recibida por el nodo de proyección. Por tanto, el suavizado no se aplica únicamente al centro, sino directamente a las esquinas de la detección, formadas por $(x_{1,k}, y_{1,k})$ y $(x_{2,k}, y_{2,k})$. Si la caja en el instante k se representa como:

$$\mathbf{b}_k = \begin{bmatrix} x_{1,k} & y_{1,k} & x_{2,k} & y_{2,k} \end{bmatrix}^T \quad (3.21)$$

el suavizado EMA de la caja puede expresarse de forma compacta como:

$$\bar{\mathbf{b}}_k = \alpha \mathbf{b}_k + (1 - \alpha) \bar{\mathbf{b}}_{k-1} \quad (3.22)$$

En el sistema desarrollado se emplea $\alpha = 0.55$. Este valor proporciona un compromiso entre suavizado y capacidad de respuesta: si α fuera demasiado bajo, la caja sería más estable pero reaccionaría con retraso ante maniobras del objetivo; si fuera demasiado alto, las coordenadas filtradas seguirían casi por completo las oscilaciones instantáneas del detector. Con el valor elegido, la caja conserva la tendencia de movimiento del UAV objetivo y reduce pequeñas variaciones entre fotogramas.

A partir de las coordenadas suavizadas se calcula el centro final de la detección, que es el punto utilizado para obtener el error visual horizontal y vertical del controlador:

$$\bar{c}_{x,k} = \frac{\bar{x}_{1,k} + \bar{x}_{2,k}}{2}, \quad \bar{c}_{y,k} = \frac{\bar{y}_{1,k} + \bar{y}_{2,k}}{2} \quad (3.23)$$

De este modo, el punto central no depende de una caja instantánea de YOLO, sino de una caja previamente suavizada. Esto reduce la fluctuación del centro entre fotogramas y evita que pequeñas vibraciones de las esquinas se transformen directamente en cambios rápidos del error visual. En caso de pérdida puntual de detección, el sistema puede mantener la última caja suavizada válida para evitar saltos artificiales hacia el origen de la imagen,

ya que un salto a $(0, 0)$ generaría un error visual falso y podría producir una orden de control incorrecta.

La misma caja suavizada también se utiliza para calcular la anchura y la altura empleadas en la estimación de distancia:

$$\bar{w}_k = \bar{x}_{2,k} - \bar{x}_{1,k}, \quad \bar{h}_k = \bar{y}_{2,k} - \bar{y}_{1,k} \quad (3.24)$$

Esto resulta importante porque la distancia estimada depende del tamaño aparente del UAV objetivo en la imagen. Si la anchura o la altura de la caja cambian bruscamente por una oscilación del detector, la distancia calculada también puede variar de forma brusca. Al aplicar EMA sobre las cuatro esquinas antes de obtener $\bar{w}_k$ y $\bar{h}_k$, se consigue que el cálculo de distancia reciba medidas más continuas y menos sensibles a vibraciones puntuales de YOLO.

La necesidad de esta segunda etapa se debe a que el filtro de Kalman, aunque mejora la coherencia temporal de la caja delimitadora y reduce parte de las vibraciones del detector, no elimina por completo las oscilaciones residuales entre fotogramas. Una variación pequeña en la posición o en el tamaño de la caja puede desplazar de forma apreciable su punto central y modificar la distancia estimada. Si estas señales se utilizaran directamente como entrada del controlador, el error visual y el error longitudinal podrían cambiar de manera continua, generando consignas alternantes y provocando vibraciones en el dron cazador.

Por ello, el filtro EMA se justifica como un suavizado final aplicado a las coordenadas de la caja que alimentan tanto el cálculo del centro como el cálculo del ancho y el alto. Kalman se encarga de estabilizar y predecir la detección completa, mientras que el EMA reduce las oscilaciones residuales de las variables que llegan al controlador y al estimador de distancia. Esta combinación permite reducir movimientos correctivos innecesarios del UAV perseguidor sin añadir una carga computacional elevada ni introducir una respuesta excesivamente lenta ante maniobras reales del objetivo.

3.8. Metodología de Control

La estrategia de control se plantea a partir de tres reguladores PID independientes, cada uno asociado a una variable visual o espacial concreta. Esta división permite separar el seguimiento angular, la corrección vertical y el mantenimiento de la distancia de seguridad respecto al UAV objetivo. De este modo, el dron cazador no recibe una única orden global, sino varias correcciones calculadas a partir de errores específicos obtenidos por el

sistema de percepción.

El primer regulador corresponde al control de *yaw*. Este PID utiliza como entrada el error horizontal de la imagen, es decir, la diferencia entre el centro de la detección y el centro de la cámara en el eje x . Dicho error se trabaja en píxeles normalizados para que la respuesta no dependa directamente de la resolución de la imagen. Si el objetivo aparece desplazado hacia un lado del fotograma, el controlador genera una corrección de guiñada que orienta progresivamente el dron cazador hacia el UAV objetivo.

El segundo regulador se emplea para el control de altura. En este caso, la entrada del PID es el error vertical de la imagen, calculado a partir del desplazamiento del centro de la detección respecto al centro de la cámara en el eje y . Inicialmente se consideró la posibilidad de corregir este error mediante el ángulo de *pitch*, ya que inclinar el dron podría modificar la posición vertical aparente del objetivo en la imagen. Sin embargo, esta opción no se adoptó finalmente para no comprometer la seguridad del UAV durante las pruebas. Una corrección basada en *pitch* podría introducir movimientos longitudinales no deseados, variaciones bruscas de actitud o acercamientos peligrosos al objetivo. Por ello, se optó por actuar sobre la altura, manteniendo una respuesta más segura y controlada.

El tercer regulador corresponde al control de distancia. Este PID compara la distancia estimada al UAV objetivo, expresada en metros, con una distancia deseada de seguimiento. El objetivo no es alcanzar físicamente al otro dron, sino mantenerse a una separación prefijada que permita realizar el seguimiento visual sin riesgo de colisión. Esta distancia de referencia actúa como margen de seguridad, evitando que el dron cazador se acerque demasiado al objetivo y reduciendo la posibilidad de choque durante las pruebas, lo que podría dañar o destruir alguno de los UAVs.

En conjunto, los tres PID permiten transformar las salidas del sistema de percepción en consignas de control interpretables: el error horizontal corrige la orientación mediante *yaw*, el error vertical corrige la altura y el error de distancia regula el acercamiento o alejamiento respecto al objetivo. Esta estructura modular facilita el ajuste independiente de cada comportamiento y permite limitar cada acción de control para priorizar la estabilidad y la seguridad del seguimiento.

Implementación

4.1. Solución de software

4.1.1. Arquitectura del sistema

La arquitectura del sistema se plantea como un lazo cerrado de seguimiento visual, en el que la información captada por la cámara se transforma progresivamente en órdenes de control para el UAV cazador. La Figura 4.1 resume este flujo completo, desde la adquisición de la imagen hasta el envío de consignas al autopiloto mediante *MAVROS*.

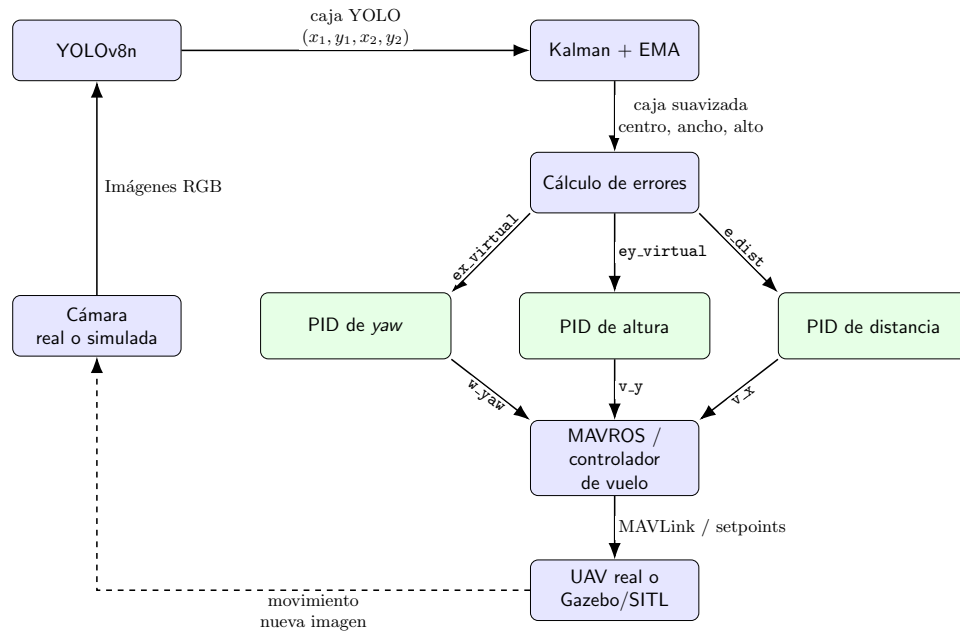


Figura 4.1: Arquitectura general del sistema de seguimiento visual, válida tanto para simulación como para despliegue real. (Imagen de elaboración propia)

Como se observa en la Figura 4.1, el proceso comienza en la cámara real o simulada, que proporciona imágenes RGB del entorno. Estas imágenes

son procesadas por YOLOv8n, que detecta el UAV objetivo y genera la caja delimitadora de la detección mediante las coordenadas (x_1, y_1, x_2, y_2) . Esta información constituye la entrada del bloque de filtrado, donde se aplica un filtrado basado en Kalman y EMA para suavizar las variaciones entre fotogramas y obtener una caja más estable.

A partir de la caja suavizada se calcula el centro aparente del objetivo, su anchura y su altura. Con estos datos se obtienen los tres errores que alimentan el control: `ex_virtual`, asociado al desplazamiento horizontal; `ey_virtual`, asociado al desplazamiento vertical; y `e_dist`, asociado a la diferencia entre la distancia estimada y la distancia deseada de seguimiento.

Cada uno de estos errores se envía a un PID independiente. El PID de *yaw* genera la velocidad angular `w_yaw`, el PID de altura genera la velocidad vertical `v_y` y el PID de distancia genera la velocidad frontal `v_x`. Las tres salidas llegan al bloque *MAVROS* / controlador de vuelo, que las convierte en consignas compatibles con el autopiloto. Finalmente, el movimiento del UAV modifica de nuevo la imagen recibida por la cámara, cerrando el lazo de realimentación visual del sistema.

4.1.2. Nodos y scripts desarrollados

En esta sección se describe la arquitectura de nodos y tópicos utilizados para integrar la percepción visual, la estimación del error y el control del UAV cazador. También se resumen los tópicos ROS más importantes observados mediante `rostopic list`, diferenciando entre los tópicos propios del sistema y los generados automáticamente por *Gazebo*, *MAVROS* y los sensores simulados.

Para visualizar la evolución de la arquitectura se emplea `rqt_graph`, que permite observar de forma gráfica qué nodos están activos y cómo se comunican mediante tópicos. En primer lugar, se muestra el sistema justo después de iniciar la simulación, sin activar todavía los nodos de visión ni el nodo de control. En esta etapa aparecen principalmente los nodos y conexiones asociados a *Gazebo*, *MAVROS*, la cámara simulada y los tópicos de soporte de ROS.

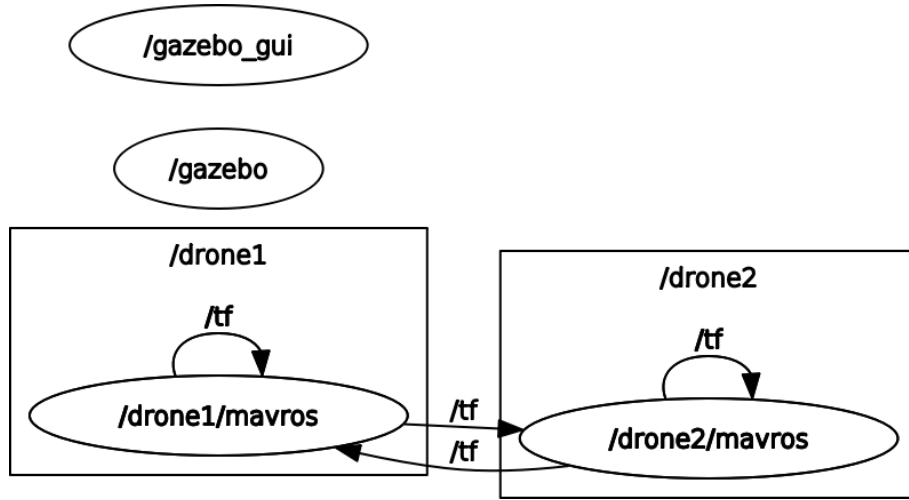


Figura 4.2. Grafo inicial del sistema sin nodos de visión ni control activos.
(Imagen de elaboración propia)

Posteriormente se activan los nodos de visión. En este segundo estado aparecen el nodo `yolo_sender_node`, encargado de procesar la imagen y publicar las coordenadas de la caja detectada, y el nodo `drone_projection_opencv_node`, encargado de recibir la detección, calcular el centro visual, aplicar la compensación de actitud y publicar el error visual. En el grafo se observa cómo la imagen de la cámara alimenta al nodo de YOLO y cómo sus salidas se conectan con el nodo de proyección.

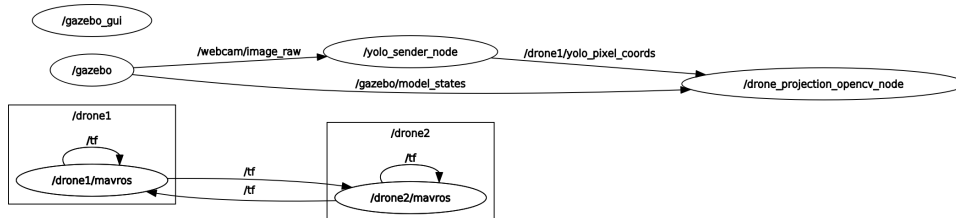


Figura 4.3. Grafo del sistema con los nodos de visión activados. (Imagen de elaboración propia)

Finalmente, se activa el nodo de control `drone1_follow_xy_pid`. Este nodo recibe el error publicado por `drone_projection_opencv_node` a través de `/drone1/vision_error`, comprueba el estado del UAV mediante `/drone1/mavros/state` y publica las consignas de velocidad en `/drone1/mavros/setpoint_velocity`. Con ello queda cerrado el flujo completo entre percepción, estimación del error, control y autopiloto.

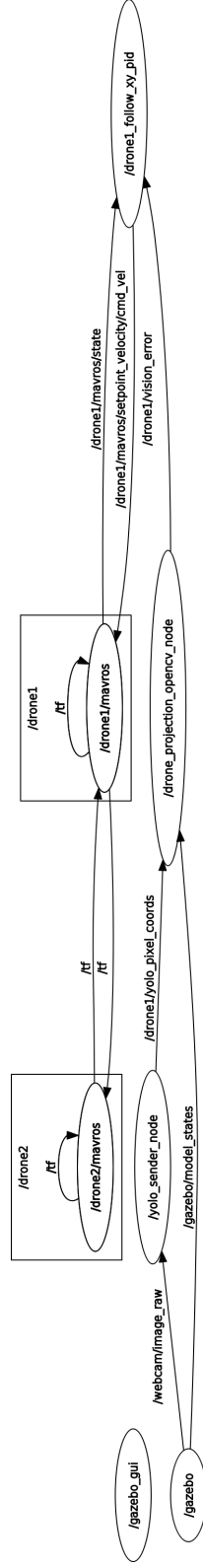


Figura 4.4. Grafo completo del sistema con visión y control activos. (Imagen de elaboración propia)

```

germanrv@germanrv-Pulse-GL66-12UEK:~$ rostopic list
/clock
/diagnostics
/drone1/estimated_distance
/drone1/mavlink/from
/drone1/mavlink/gcs_ip
/drone1/mavlink/to
/drone1/mavros/adsb/send
/drone1/mavros/adsb/vehicle
/drone1/mavros/battery
/drone1/mavros/cam_imu_sync/cam_imu_stamp
/drone1/mavros/camera/image_captured
/drone1/mavros/cellular_status/status
/drone1/mavros/companion_process/status
/drone1/mavros/distance_sensor/rangefinder_pub
/drone1/mavros/distance_sensor/rangefinder_sub
/drone1/mavros/esc_info
/drone1/mavros/esc_status
/drone1/mavros/esc_telemetry
/drone1/mavros/estimator_status
/drone1/mavros/extended_state
/drone1/mavros/fake_gps/mocap/pose
/drone1/mavros/geofence/waypoints
/drone1/mavros/global_position/compass_hdg
/drone1/mavros/global_position/global
/drone1/mavros/global_position/gp_lp_offset
/drone1/mavros/global_position/gp_origin
/drone1/mavros/global_position/local
/drone1/mavros/global_position/raw/fix
/drone1/mavros/global_position/raw/gps_vel
/drone1/mavros/global_position/raw/satellites
/drone1/mavros/global_position/rel_alt
/drone1/mavros/global_position/set_gp_origin
/drone1/mavros/gps_input/gps_input
/drone1/mavros/gps_rtk/rtk_baseline
/drone1/mavros/gps_rtk/send_rtcn
/drone1/mavros/gpsstatus/gps1/raw
/drone1/mavros/gpsstatus/gps1/rtk
/drone1/mavros/gpsstatus/gps2/raw
/drone1/mavros/gpsstatus/gps2/rtk
/drone1/mavros/home_position/home
/drone1/mavros/home_position/set
/drone1/mavros/imu/data
/drone1/mavros/imu/data_raw
/drone1/mavros/imu/diff_pressure
/drone1/mavros/imu/mag
/drone1/mavros/imu/static_pressure
/drone1/mavros/imu/temperature_baro
/drone1/mavros/imu/temperature_imu
/drone1/mavros/landing_target/lt_marker
/drone1/mavros/landing_target/pose
/drone1/mavros/landing_target/pose_in
/drone1/mavros/local_position/accel

```

Figura 4.5. Listado de algunos tópicos ROS obtenido mediante `rostopic list`.
(Imagen de elaboración propia)

La salida de `rostopic list` muestra una gran cantidad de tópicos porque se lanzan dos instancias de UAV, cada una con su propio espacio de nombres de *MAVROS*. Sin embargo, para el funcionamiento del sistema propuesto, los tópicos más relevantes son los siguientes:

- `/webcam/image_raw`: imagen RGB de entrada utilizada por YOLO y

por el nodo de proyección para visualizar la detección y calcular el error sobre el plano de imagen.

- `/webcam/camera_info`: información de calibración de la cámara. Aunque en la implementación también se cargan parámetros desde archivo, este tópico representa la fuente estándar de parámetros intrínsecos en ROS.
- `/drone1/yolo_pixel_coords`: tópico publicado por el nodo de YOLO con las cuatro coordenadas de la caja delimitadora del objetivo, codificadas como **Quaternion**: x_1 , y_1 , x_2 e y_2 .
- `/drone1/yolo_state`: tópico de estado de la detección. El valor 0 indica objetivo perdido, el valor 1 indica detección directa de YOLO y el valor 2 indica predicción temporal mediante Kalman.
- `/drone1/vision_error`: tópico publicado por el nodo de proyección. Contiene el error horizontal, el error vertical y el error de distancia en un mensaje **PointStamped**, que posteriormente consume el controlador PID.
- `/drone1/mavros/state`: estado de conexión, armado y modo de vuelo del UAV cazador. El controlador lo consulta antes de publicar órdenes para evitar actuar cuando el dron no está listo.
- `/drone1/mavros/setpoint_velocity/cmd_vel`: tópico de salida del controlador. En él se publican las velocidades lineales y angulares que *MAVROS* envía al autopiloto para mover el dron cazador.
- `/gazebo/model_states`: poses de los modelos simulados en *Gazebo*. Se utiliza para obtener la pose de `drone1` y `drone2`, comprobar la geometría de la simulación y apoyar la compensación/proyección.

De forma resumida, el flujo de información comienza en `/webcam/image_raw`, pasa por el nodo YOLO, se transforma en coordenadas de caja mediante `/drone1/yolo_pixel_coords`, se convierte en error visual y longitudinal en `/drone1/vision_error` y finalmente se traduce en comandos de velocidad publicados en `/drone1/mavros/setpoint_velocity/cmd_vel`. El resto de tópicos de *MAVROS* que aparecen bajo `/drone1/mavros/` y `/drone2/mavros/` corresponden a telemetría, sensores, misiones, parámetros y servicios auxiliares generados automáticamente por la interfaz con el autopiloto.

4.2. Componente de percepción

4.2.1. Entrenamiento de modelos YOLO

El modelo de detección finalmente integrado en el sistema fue YOLOv8n. Esta arquitectura se seleccionó mediante un torneo comparativo entre distintas variantes de YOLO, atendiendo al equilibrio entre precisión, tiempo de inferencia y tamaño del modelo. Dicho proceso de selección se detalla en la sección de experimentación, mientras que en este apartado se describe la preparación del conjunto de datos y el entrenamiento del modelo elegido.

Antes del entrenamiento se aplicó un preprocesamiento común a todas las imágenes del conjunto de datos. En primer lugar, todas las muestras se redimensionaron a 640×640 píxeles mediante un ajuste de tipo *Fit*, manteniendo la relación de aspecto original e incorporando bordes negros cuando fue necesario. Este formato coincide con el tamaño de entrada empleado durante el entrenamiento de YOLOv8n y permite trabajar con imágenes homogéneas sin deformar el objeto de interés.

Sobre el conjunto de datos preprocesado se aplicó un proceso de aumento de datos con el objetivo de mejorar la capacidad de generalización del detector. Las transformaciones utilizadas fueron volteo horizontal, recorte con zoom mínimo del 0 % y zoom máximo del 13 %, variación de saturación entre -4% y $+4\%$, y variación de exposición entre -8% y $+8\%$. Estas modificaciones permiten que el modelo observe el UAV objetivo en condiciones visuales ligeramente distintas, simulando cambios de posición, escala, iluminación y color. De esta forma se reduce la dependencia respecto a las imágenes originales y se aumenta la robustez frente a situaciones que pueden aparecer durante la simulación o en un despliegue real.

Tras aplicar el preprocesamiento y el aumento de datos, el conjunto final quedó formado por 9651 imágenes. La división empleada para el entrenamiento fue de 6755 imágenes para *train*, 1447 imágenes para *valid* y 1449 imágenes para *test*. Esta separación permitió entrenar el modelo con la mayor parte de los datos, supervisar su evolución sobre un conjunto de validación independiente y reservar un subconjunto final para comprobar posteriormente el comportamiento del detector.

El entrenamiento se realizó en Google Colab, donde se cargó el conjunto de datos ya organizado y se configuró el archivo YAML correspondiente con las rutas de entrenamiento, validación y prueba. Como YOLOv8n ya dispone de pesos preentrenados, el proceso se planteó como un ajuste fino sobre el modelo base `yolov8n.pt`, en lugar de entrenar la red desde cero. La ejecución se configuró para entrenar durante 30 épocas, con imágenes de entrada de 640×640 píxeles y un tamaño de lote de 16. Además, se activó la generación de gráficas del entrenamiento para analizar posteriormente

la evolución de las pérdidas, las métricas de validación y los ejemplos de detección obtenidos.

Se mantuvieron los parámetros por defecto proporcionados por Ultralytics para el entrenamiento, dejando que la opción `optimizer=auto` seleccionara automáticamente el optimizador y los hiperparámetros principales. En este caso, el entrenamiento utilizó AdamW con `lr=0.002` y `momentum=0.9`, con pesos preentrenados activados mediante `pretrained=True`. Esto permitió aprovechar las características visuales ya aprendidas por YOLOv8n y adaptar únicamente el detector al UAV objetivo del conjunto de datos generado.

La evolución del entrenamiento se resume en la Figura 4.6. En ella se observa que las pérdidas asociadas a la localización de la caja, la clasificación y la función DFL disminuyen de forma progresiva tanto en entrenamiento como en validación. Al mismo tiempo, las métricas de precisión, *recall*, *mAP50* y *mAP50-95* aumentan y se estabilizan en valores elevados, lo que indica que el modelo aprende a detectar el UAV objetivo sin mostrar una divergencia clara entre entrenamiento y validación.

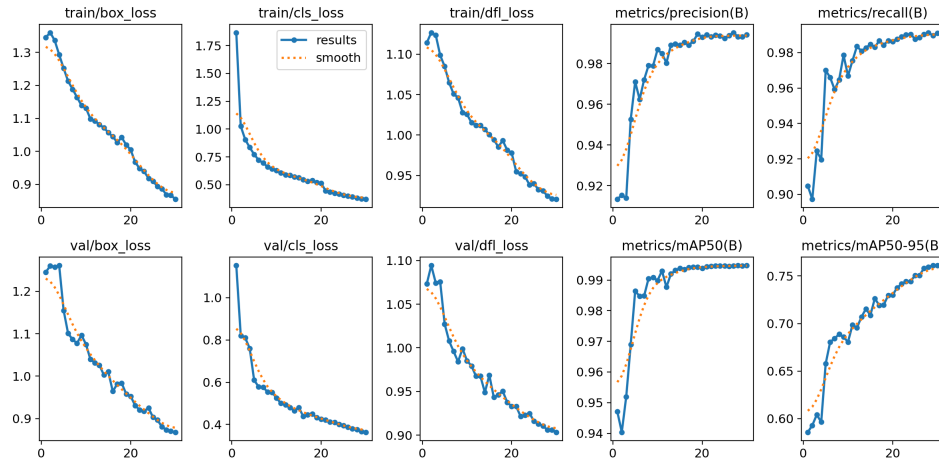


Figura 4.6: Evolución de pérdidas y métricas durante el entrenamiento de YOLOv8n. (Imagen de elaboración propia)

La matriz de confusión de la Figura 4.7 refuerza esta conclusión, ya que concentra la mayor parte de las predicciones en la clase correcta y muestra una separación clara entre el UAV objetivo y el fondo. Este resultado es especialmente importante para el sistema de seguimiento, porque una confusión frecuente con el fondo provocaría cajas falsas, errores visuales incorrectos y órdenes de control innecesarios.

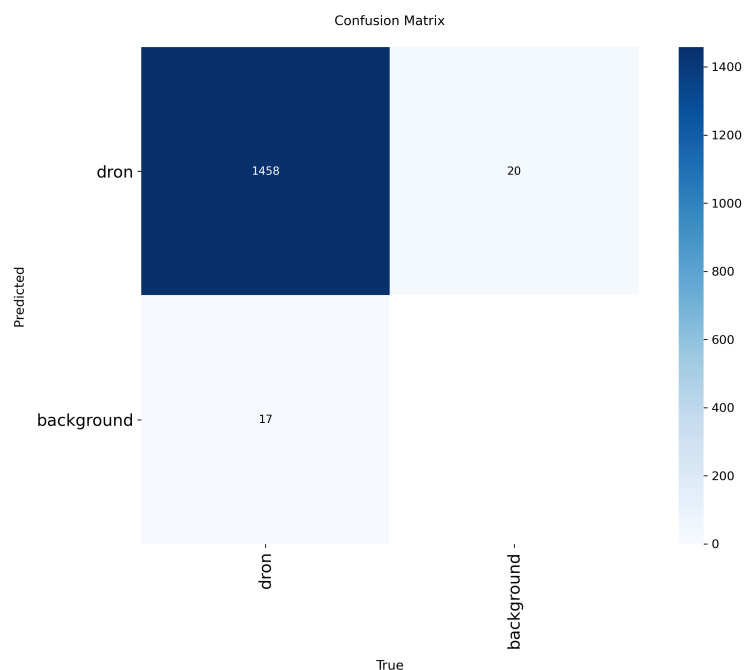


Figura 4.7: Matriz de confusión obtenida tras el entrenamiento de YOLOv8n. (Imagen de elaboración propia)

Por último, la Figura 4.8 muestra ejemplos de predicción sobre imágenes de validación. En estos casos, el detector localiza correctamente el UAV y ajusta la caja delimitadora alrededor del objetivo, incluso cuando cambia su escala o su posición dentro de la imagen. Esta comprobación visual resulta útil porque permite verificar que las métricas numéricas se corresponden con detecciones coherentes desde el punto de vista del sistema de control.

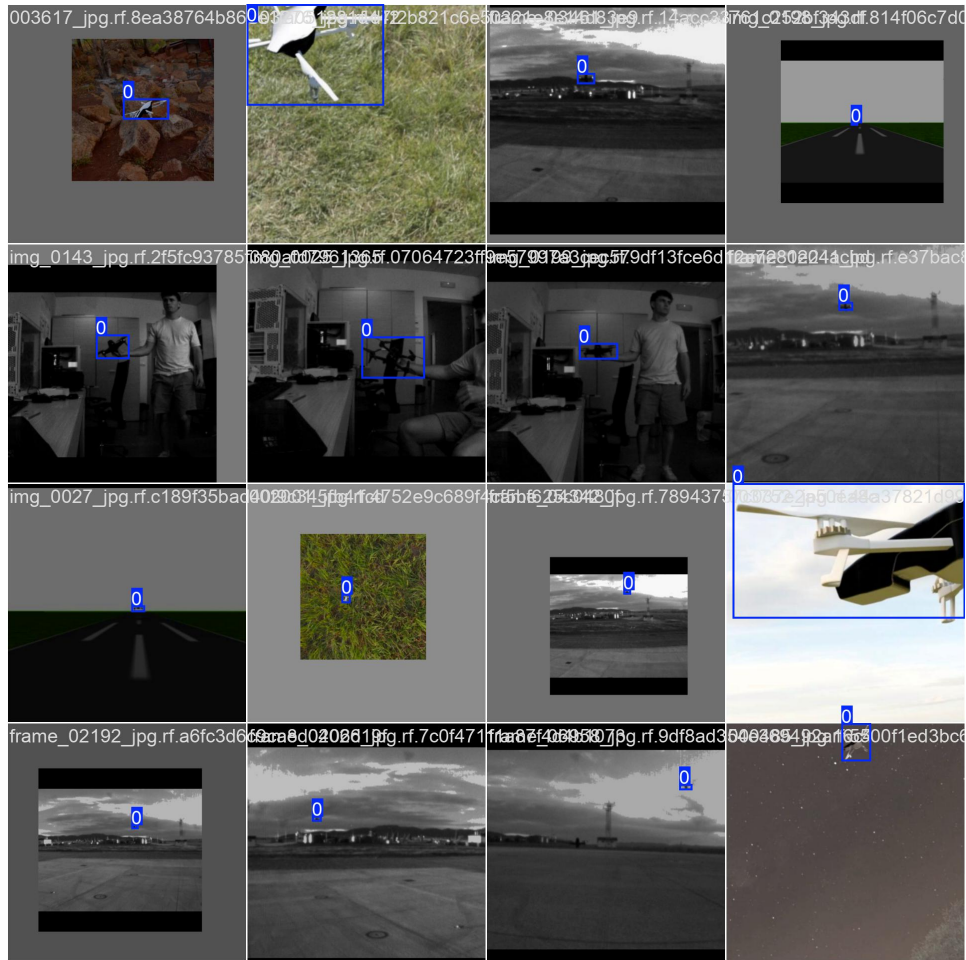


Figura 4.8: Ejemplos de detección sobre el conjunto de validación tras el entrenamiento. (Imagen de elaboración propia)

En conjunto, los resultados obtenidos indican que el entrenamiento fue satisfactorio. Las métricas de precisión y *recall* se mantuvieron muy elevadas, el *mAP50* permaneció prácticamente saturado en 0.995 y el *mAP50-95* siguió mejorando de forma progresiva. Además, la reducción simultánea de las pérdidas y la calidad de las predicciones visuales sugieren que el ajuste fino refinó correctamente la localización de las cajas y la clasificación del objetivo sin mostrar signos claros de degradación en validación. Una vez finalizado el entrenamiento, se guardó el mejor modelo obtenido, **best.pt**, para emplearlo posteriormente en el nodo de percepción visual del sistema.

4.2.2. Estimación de la distancia mediante el ancho de la caja

La estimación de la distancia al UAV objetivo se realiza a partir de la caja delimitadora proporcionada por el sistema de percepción. En este apartado no se vuelve a desarrollar el suavizado temporal, ya que se describe en el apartado de estimación y filtrado. Únicamente se considera que la caja utilizada para el cálculo ya ha sido estabilizada previamente, de forma que sus coordenadas, su anchura y su altura presentan menos variaciones entre fotogramas.

El proceso implementado parte de las coordenadas de la caja una vez suavizadas. Después de estabilizar las cuatro coordenadas de la detección, se calcula la anchura y la altura aparentes del UAV objetivo en la imagen:

$$\bar{w}_k = \bar{x}_{2,k} - \bar{x}_{1,k}, \quad \bar{h}_k = \bar{y}_{2,k} - \bar{y}_{1,k} \quad (4.1)$$

Con estos dos valores se calculan dos estimaciones independientes de la distancia. La primera utiliza la anchura real aproximada del UAV y la focal horizontal de la cámara, mientras que la segunda emplea la altura real aproximada y la focal vertical:

$$D_W = \frac{f_x \cdot W_{real}}{\bar{w}_k}, \quad D_H = \frac{f_y \cdot H_{real}}{\bar{h}_k} \quad (4.2)$$

En estas expresiones, f_x y f_y representan las focales de la cámara en píxeles, y W_{real} y H_{real} corresponden a las dimensiones reales aproximadas del UAV objetivo. En una primera aproximación se planteó calcular la distancia final como la media aritmética de ambas estimaciones:

$$D_{final} = \frac{D_W + D_H}{2} \quad (4.3)$$

Esta formulación inicial permitía aprovechar simultáneamente la información horizontal y vertical de la detección. Si YOLO generaba una caja ligeramente más ancha que la real, pero mantenía una altura coherente, la estimación basada en la altura podía compensar parcialmente el error. Del mismo modo, si la altura presentaba una pequeña variación, la anchura ayudaba a estabilizar el resultado. Por tanto, desde el punto de vista teórico, el promedio entre D_W y D_H ofrecía una medida de profundidad más robusta que una estimación dependiente únicamente de una sola dimensión de la caja.

Sin embargo, durante las pruebas se observó que esta relación no era suficientemente estable en todas las maniobras. En particular, cuando el UAV se

inclinaba para avanzar, la altura aparente de la caja delimitadora podía aumentar de forma notable aunque la distancia real al objetivo no cambiara en la misma proporción. Este efecto rompía parcialmente la relación geométrica utilizada para estimar la distancia a partir de $\hat{h}_k$, ya que el incremento de la altura de la caja se interpretaba como un acercamiento artificial. Como consecuencia, la estimación D_H introducía variaciones bruscas en el error longitudinal y podía generar órdenes de control poco estables.

A partir de estas pruebas se comprobó que la anchura de la caja era una medida más estable para el escenario considerado. Por este motivo, después de evaluar la estimación mediante la media entre D_W y D_H , se prefirió utilizar únicamente la distancia obtenida a partir del ancho de la caja. Esta decisión se debe a que la altura aparente del UAV podía variar mucho durante el vuelo, especialmente cuando el objetivo cambiaba de inclinación o cuando la detección ajustaba la caja con pequeñas diferencias entre fotografías.

De este modo, la implementación final toma como referencia principal D_W . La estimación basada en la altura, D_H , se mantiene como parte del análisis geométrico inicial, pero no se utiliza para alimentar el controlador longitudinal. Así se evita que variaciones bruscas de la altura de la caja se traduzcan en cambios artificiales de distancia y en órdenes de control menos estables.

Finalmente, la distancia estimada a partir del ancho de la caja se utiliza como entrada del controlador longitudinal, comparándola con una distancia deseada de seguimiento. De esta forma, el dron cazador puede acercarse o alejarse del objetivo manteniendo un margen de seguridad, sin depender únicamente del desplazamiento del centro de la imagen ni de una altura de caja demasiado sensible a la inclinación del UAV.

4.3. Compensación de actitud y proyección

Durante las primeras pruebas se observó que el error visual no dependía únicamente de la posición real del UAV objetivo dentro de la imagen, sino también de la actitud instantánea del dron cazador. Si se tomaba siempre el centro geométrico de la imagen como referencia, cualquier inclinación de la cámara producía un desplazamiento aparente del objetivo. Esto generaba errores falsos que el controlador interpretaba como movimientos reales del objetivo.

El problema aparecía especialmente en dos situaciones. En primer lugar, cuando el dron cazador avanzaba, debía inclinarse para generar movimiento longitudinal. Esta inclinación modificaba la orientación de la cámara y hacía que el objetivo apareciera desplazado verticalmente en la imagen, aunque su

posición relativa no hubiera cambiado de forma equivalente. Como consecuencia, el error vertical aumentaba y el controlador de altura podía intentar corregir una desviación que realmente estaba producida por el propio *pitch* del UAV.

En segundo lugar, al realizar giros o movimientos laterales, el dron podía ladearse. Este *roll* rotaba el plano de la imagen y acoplaba parte del error horizontal con el error vertical. Es decir, un desplazamiento producido por la actitud del dron podía aparecer como una variación en el eje vertical de la imagen, generando órdenes de altura innecesarias. Para evitar este comportamiento, se decidió compensar la referencia visual usando la orientación real de la cámara antes de calcular el error.

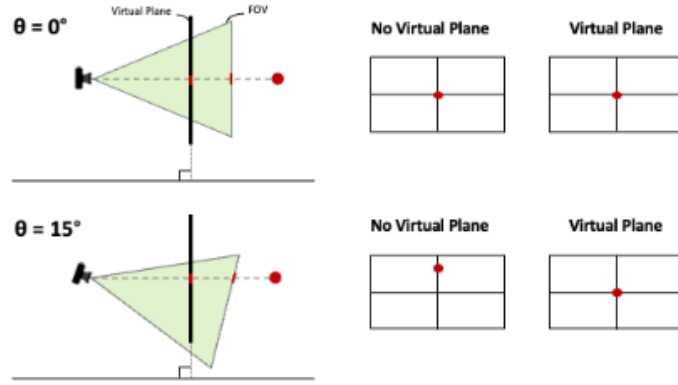


Figura 4.9. Esquema de compensación de actitud aplicado al plano de imagen.
Fuente: ACTA Press [49].

La Figura 4.9 ilustra la idea utilizada para desplazar la referencia de la imagen en función de la actitud de la cámara. En lugar de emplear un centro fijo, se proyecta un rayo compensado que representa el nuevo punto de referencia sobre el plano visual.

La compensación se implementa a partir de la pose del dron cazador en *Gazebo* o del dron real y de la transformación fija entre el cuerpo del dron y la cámara. En primer lugar, se construye la transformación homogénea del dron en el mundo y se compone con la transformación de la cámara:

$$\mathbf{T}_w^c = \mathbf{T}_w^{d1} \mathbf{T}_{d1}^c \quad (4.4)$$

De esta transformación se extrae la matriz de rotación de la cámara y se convierte en ángulos de *roll*, *pitch* y *yaw*:

$$\mathbf{R}_w^c = \mathbf{T}_w^c(1:3, 1:3), \quad (\phi, \theta, \psi) = \text{RPY}(\mathbf{R}_w^c) \quad (4.5)$$

A continuación, se genera una rotación de compensación utilizando únicamente las componentes de *pitch* y *roll*. Siguiendo la convención de ejes empleada en el nodo, esta rotación se construye como:

$$\mathbf{q}_{comp} = \mathbf{q}_{pitch}(\theta)\mathbf{q}_{roll}(\phi) \quad (4.6)$$

Esta rotación se aplica sobre el rayo ideal de la cámara, definido como un vector que apunta hacia delante en el eje óptico:

$$\mathbf{r}_0 = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix}^T, \quad \mathbf{r}_{comp} = \mathbf{R}(\mathbf{q}_{comp})\mathbf{r}_0 \quad (4.7)$$

El vector resultante representa hacia dónde debería desplazarse el centro de referencia de la imagen cuando la cámara está inclinada. Para obtener ese punto en coordenadas de imagen, el rayo compensado se proyecta mediante el modelo de cámara y los parámetros intrínsecos:

$$\begin{bmatrix} u_c & v_c & 1 \end{bmatrix}^T \sim \mathbf{K}\mathbf{r}_{comp} \quad (4.8)$$

En la implementación, esta proyección se realiza con `cv::projectPoints` y posteriormente se corrige con `cv::undistortPoints`, empleando la matriz intrínseca `camera_matrix` y los coeficientes de distorsión `dist_coeffs`. El punto obtenido, (u_c, v_c) , sustituye al centro geométrico de la imagen como referencia del controlador. Por ello, el error visual final se calcula como:

$$e_x = \frac{u_c - u_{YOLO}}{W/2}, \quad e_y = \frac{v_c - v_{YOLO}}{H/2} \quad (4.9)$$

De este modo, el controlador no intenta centrar el objetivo respecto al centro fijo de la imagen, sino respecto al centro compensado según la inclinación real de la cámara. Esto reduce los errores verticales artificiales producidos por el avance del dron y por los ladeos durante maniobras laterales.

No obstante, la compensación no se aplicó a todas las componentes de la orientación. En particular, no se incluyó el *yaw* dentro de la rotación compensada, ya que esto podía producir un efecto no deseado similar a una brújula: el punto de referencia de la imagen quedaría condicionado por el rumbo absoluto del dron y no solo por la inclinación de la cámara. En ese caso, durante los giros, el sistema podría desplazar artificialmente la referencia visual y generar correcciones que no corresponden al centrado real del objetivo. Por este motivo, la compensación se limitó a las inclinaciones que afectan directamente al plano de imagen, dejando que el control de *yaw* se encargue de corregir el error horizontal de seguimiento.

4.4. Componente de control

Los valores de los controladores PID se obtuvieron mediante un proceso de ajuste empírico iterativo en simulación. En primer lugar, se fijaron ganancias proporcionales bajas para comprobar el sentido de actuación de cada eje y evitar respuestas bruscas. Después se aumentó progresivamente la ganancia proporcional hasta obtener una corrección rápida del error sin provocar oscilaciones sostenidas. A continuación, se introdujo la componente derivativa para amortiguar los cambios rápidos de la señal de error y reducir el sobreimpulso. Finalmente, se añadió una componente integral pequeña para compensar errores residuales en régimen permanente, evitando que la acumulación integral generara órdenes excesivas.

Durante este ajuste también se limitaron las salidas de cada controlador. De esta forma, aunque el error visual o el error de distancia aumenten de forma puntual, las consignas enviadas al autopiloto permanecen dentro de valores admisibles para la simulación. La ley general utilizada para cada controlador se expresa como:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (4.10)$$

Posteriormente, la señal calculada se satura para limitar la velocidad angular o lineal enviada a *MAVROS*:

$$u_{sat}(t) = \text{sat}_{[-u_{max}, u_{max}]}(u(t)) \quad (4.11)$$

En el caso del control horizontal, el error de entrada es el desplazamiento visual compensado en el eje horizontal, denominado **ex_virtual**. Este error alimenta el PID de *yaw*, cuya salida es la velocidad angular **w_yaw**:

$$\omega_{yaw} = \text{sat}_{[-10,10]} \left(2.365625 e_{x,virtual}(t) + 0.06 \int_0^t e_{x,virtual}(\tau) d\tau + 0.320 \frac{de_{x,virtual}(t)}{dt} \right) \quad (4.12)$$

Para el control vertical se emplea el error **ey_virtual**. Este controlador genera la velocidad vertical **v_y**, responsable de corregir la posición aparente del objetivo en altura:

$$v_y = \text{sat}_{[-5,5]} \left(2.15 e_{y,virtual}(t) + 0.0 \int_0^t e_{y,virtual}(\tau) d\tau + 0.205 \frac{de_{y,virtual}(t)}{dt} \right) \quad (4.13)$$

Finalmente, el control longitudinal utiliza el error de distancia estimada, **e_dist**, calculado a partir de la diferencia entre la distancia estimada al

UAV objetivo y la distancia de referencia. Su salida es la velocidad frontal v_x :

$$v_x = \text{sat}_{[-10,10]} \left(0.9 e_{dist}(t) + 0.0 \int_0^t e_{dist}(\tau) d\tau + 0.35 \frac{de_{dist}(t)}{dt} \right) \quad (4.14)$$

La Tabla 4.1 resume las ganancias y los límites finalmente utilizados en los tres controladores implementados.

Controlador	Entrada	Salida	K_p	K_i	K_d	Límite de salida
PID de <i>yaw</i>	ex_virtual	w_yaw	2.365625	0.06	0.320	± 10.0 rad/s
PID de altura	ey_virtual	v_y	2.15	0.0	0.205	± 5.0 m/s
PID de distancia	e_dist	v_x	0.9	0.0	0.35	± 10.0 m/s

Cuadro 4.1: Parámetros finales de los controladores PID utilizados para el seguimiento visual.

4.5. Configuración para pruebas reales

4.5.1. Dron real

La validación experimental del sistema requiere adaptar la arquitectura desarrollada en simulación a una plataforma física. Para ello se emplea un UAV real equipado con una cámara frontal, que actúa como sensor principal de percepción visual. Esta configuración permite capturar imágenes del entorno, ejecutar el detector de objetos y generar las señales de error necesarias para el seguimiento del UAV objetivo.

En las Figuras 4.10 y 4.11 se muestran dos vistas del dron real utilizado. La vista frontal permite observar la disposición de la cámara y su orientación respecto al eje de avance, mientras que la vista posterior ayuda a identificar la distribución general de la plataforma y el montaje de sus elementos principales.

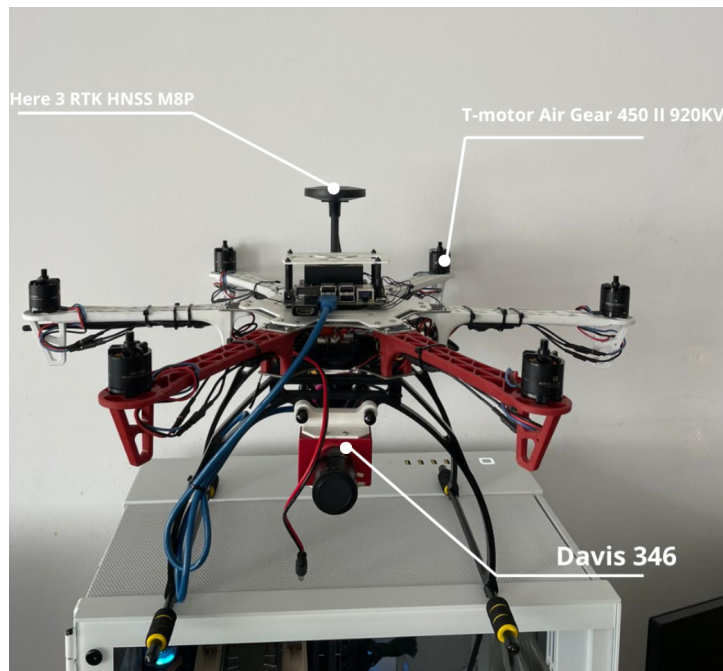


Figura 4.10. Vista frontal del dron real utilizado en las pruebas. (Imagen de elaboración propia)

En la vista frontal se identifican los elementos directamente relacionados con la percepción y la propulsión del UAV. En la parte superior se encuentra el módulo *Here 3 RTK GNSS M8P*, utilizado como receptor de posicionamiento para disponer de una referencia global precisa durante la operación del dron. En los extremos de los brazos se observan los motores *T-Motor Air Gear 450 II 920KV*, responsables de generar el empuje necesario para el vuelo. En la zona inferior frontal se sitúa la cámara *DAVIS 346*, con resolución de 346×260 píxeles, captura en escala de grises y salida simultánea de eventos DVS, orientada hacia el eje de avance del dron y empleada como sensor principal para capturar la imagen sobre la que se ejecuta el sistema de detección y seguimiento visual.

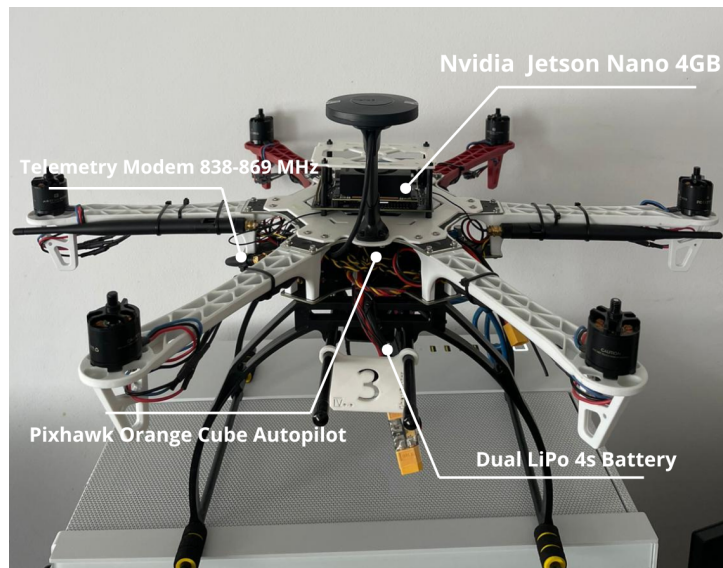


Figura 4.11. Vista posterior del dron real utilizado en las pruebas. (Imagen de elaboración propia)

La vista posterior permite distinguir los elementos principales de cómputo, comunicación, alimentación y control de vuelo. La placa *NVIDIA Jetson Nano 4 GB*, situada en la parte superior de la estructura, actúa como unidad de procesamiento embarcada y permite ejecutar los nodos de visión y control. El módem de telemetría de 838–869 MHz proporciona el enlace de comunicación con la estación de tierra. En la zona central se ubica el autopiloto *Pixhawk Orange Cube*, encargado de estabilizar el UAV y ejecutar las consignas recibidas a través de *MAVROS*. Por último, la doble batería LiPo 4S suministra la energía necesaria a los sistemas de propulsión y a la electrónica embarcada.

4.5.2. Preparación del dron real

Antes de trasladar el sistema desde el entorno simulado al UAV real fue necesario preparar los elementos de percepción que dependen directamente del hardware. En particular, la cámara utilizada en las pruebas reales debía proporcionar medidas coherentes para que el cálculo del centro de la detección, la compensación de actitud y la estimación de distancia no estuvieran afectados por errores de distorsión o por parámetros intrínsecos incorrectos. Por ello, la primera tarea de preparación consistió en calibrar la cámara real y guardar sus parámetros para que pudieran ser reutilizados por los nodos de visión.

4.5.3. Calibración de la cámara

La calibración se realizó siguiendo el procedimiento de calibración monocular propuesto en la documentación de ROS para el paquete `camera_calibration` [53]. Para ello se empleó un tablero de calibración de 6×9 cuadrados, con cuadrados de 30 mm de lado. Aunque físicamente el tablero contaba con 6×9 cuadrados, en la herramienta de ROS se indicó el tamaño 5×8 , ya que el parámetro `--size` hace referencia al número de esquinas interiores detectables y no al número total de cuadrados del patrón.

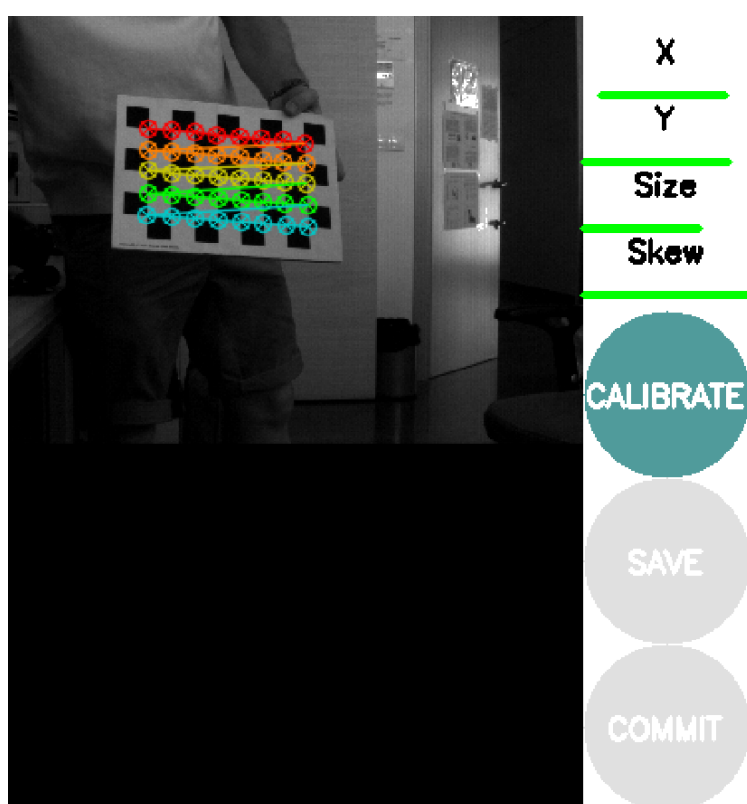


Figura 4.12. Proceso de calibración de la cámara real mediante tablero de ajedrez. (Imagen de elaboración propia)

En primer lugar, se instalaron las dependencias necesarias del paquete de calibración mediante `rosdep`:

```
rosdep install camera_calibration
```

Posteriormente se lanzó la herramienta de calibración monocular, reasignando los tópicos de imagen y de cámara al nodo de captura utilizado durante las pruebas reales:


```

roslaunch camera_calibration cameracalibrator.py --size 5x8 --
square 0.03 \
image:=/capture_node/camera/image \
camera:=/capture_node/camera \
/capture_node/camera/set_camera_info:=/capture_node/
set_camera_info

```

El parámetro `--square 0.03` indica que el lado de cada cuadrado del tablero es de 0.03 m, equivalente a los 30 mm empleados físicamente. Durante el proceso se movió el tablero frente a la cámara cubriendo distintas posiciones, orientaciones y distancias, de forma que la herramienta pudiera observar las esquinas interiores en varias zonas de la imagen. Con estas muestras se estimaron los parámetros intrínsecos de la cámara y los coeficientes de distorsión necesarios para corregir la imagen.

Una vez finalizada la calibración, se obtuvieron los parámetros de la cámara, incluyendo la matriz intrínseca y los coeficientes de distorsión. Estos valores se copiaron al archivo de configuración `camara_params.yaml`, que posteriormente lee el nodo de visión para trabajar siempre con los mismos parámetros calibrados. De este modo, las pruebas reales no dependen de una calibración manual en cada ejecución y el sistema mantiene una referencia de cámara coherente con la empleada en los cálculos de proyección, compensación y estimación de distancia.

4.5.4. Medición del UAV objetivo y posición de la cámara

Tras la calibración de la cámara, fue necesario medir físicamente el UAV objetivo para poder aplicar correctamente la estimación de distancia. Como se explicó anteriormente, esta estimación depende de las dimensiones reales del objeto detectado, por lo que los valores W_{real} y H_{real} no pueden elegirse de forma arbitraria. Tras medir el dron objetivo, se obtuvo una anchura aproximada de 34 cm y una altura aproximada de 14 cm:

$$W_{real} = 0.34 \text{ m}, \quad H_{real} = 0.14 \text{ m} \quad (4.15)$$

Estos valores se emplean como referencia física en el cálculo de D_W y D_H . De este modo, la anchura y la altura de la caja detectada en píxeles pueden transformarse en una estimación de distancia en metros. Esta medición resulta especialmente importante en las pruebas reales, ya que un error en las dimensiones del UAV objetivo se trasladaría directamente al cálculo de profundidad y, por tanto, al error longitudinal utilizado por el controlador de distancia.

También fue necesario determinar la posición de la cámara respecto al centro geométrico del dron cazador. Esta información es necesaria para que

la compensación de actitud y la proyección geométrica representen correctamente el punto desde el que se observa la escena. Si se asumiera que la cámara se encuentra exactamente en el centro del UAV, se introduciría un desplazamiento artificial entre el sistema de referencia utilizado en los cálculos y la posición real del sensor.

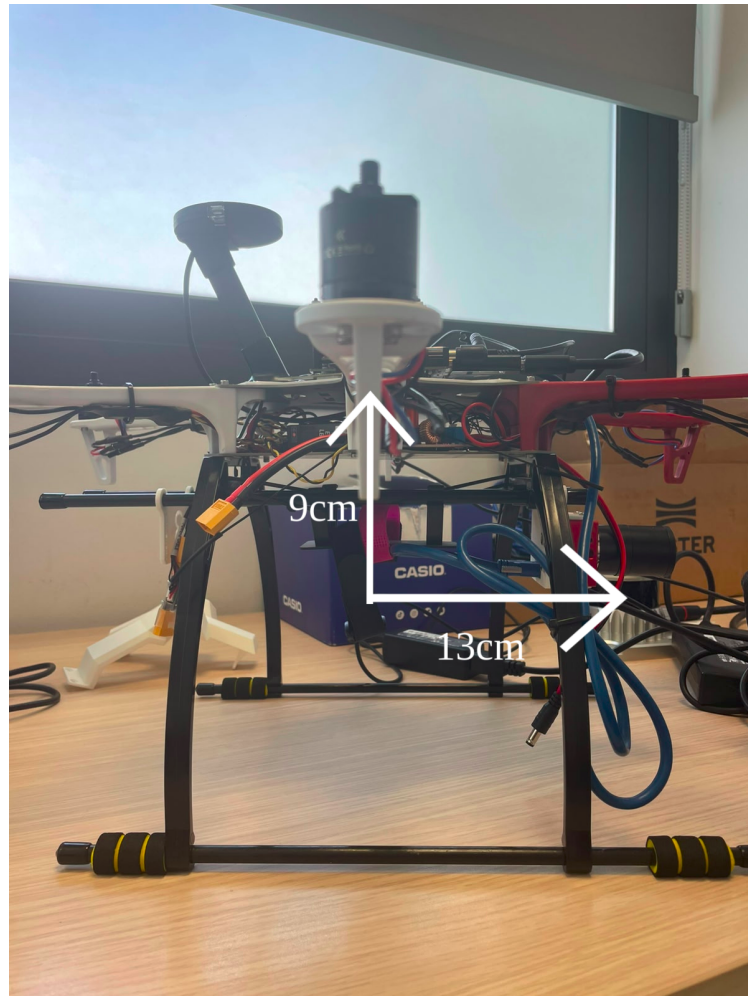


Figura 4.13. Montaje del dron cazador con la cámara utilizada en las pruebas reales. (Imagen de elaboración propia)

La Figura 4.13 muestra la posición relativa de la cámara respecto a la estructura del dron cazador. La flecha vertical indica que el sensor se encuentra aproximadamente 9 cm por debajo del centro geométrico del UAV, mientras que la flecha horizontal representa un desplazamiento de unos 13 cm respecto a dicho centro en la dirección de montaje. Por tanto, esta traslación se tuvo en cuenta como parte de la relación geométrica entre el cuerpo del

UAV y la cámara. Con ello, las proyecciones sobre el plano de imagen y la compensación de actitud se calculan a partir de una posición de cámara más próxima al montaje real, reduciendo errores sistemáticos durante las pruebas fuera de simulación.

Pruebas y Validaciones

5.1. Planificación de las Pruebas

La validación del sistema se organizó de forma progresiva, comenzando por los componentes más básicos y avanzando hasta pruebas completas de seguimiento. La idea principal fue no evaluar directamente el sistema completo, sino comprobar primero cada bloque por separado y, después, analizar si todos funcionan de forma coordinada dentro del lazo de seguimiento visual.

De forma esquemática, la planificación de las pruebas se estructuró del siguiente modo:

- **Definición de métricas de evaluación.** Antes de ejecutar los experimentos se definen las magnitudes que permiten interpretar los resultados: precisión del detector, tiempo de inferencia, tamaño del modelo, error visual de centrado, error de distancia, tiempo de establecimiento y continuidad de la detección. Con ello se intenta demostrar que la comparación entre pruebas se apoya en criterios objetivos y no únicamente en una valoración visual.
- **Selección del detector YOLO.** Se comparan distintas versiones de YOLO bajo condiciones equivalentes, atendiendo al equilibrio entre precisión, velocidad y tamaño. Con esta prueba se intenta demostrar que el modelo elegido no solo detecta correctamente el UAV objetivo, sino que también puede integrarse en un sistema de control en tiempo real sin introducir una carga computacional excesiva.
- **Optimización del detector con TensorRT.** Tras seleccionar el modelo, se evalúa su conversión a TensorRT para el despliegue en una plataforma Jetson, comparando los tiempos de preprocesamiento, inferencia y postprocesamiento del modelo original y del modelo optimizado. Con esta prueba se intenta demostrar que la optimización reduce la latencia del detector y deja margen temporal para ejecutar el resto de nodos de ROS, MAVROS, filtrado y control.
- **Validación individual de los controladores PID.** Se evalúan por separado los controladores de *yaw*, altura y distancia, de manera que

la respuesta de cada eje pueda analizarse sin mezclar efectos de otros controladores. Con esta prueba se intenta demostrar que cada PID reduce su error asociado de forma estable y con un tiempo de respuesta aceptable.

- **Prueba conjunta de control visual.** Una vez validados los controladores por separado, se activan conjuntamente los tres PID para comprobar la respuesta global del UAV cazador. Con esta prueba se intenta demostrar que las salidas de los controladores son compatibles entre sí y que los errores visuales y de distancia pueden transformarse en consignas de movimiento coherentes.
- **Seguimiento en trayectorias simuladas.** En simulación se prueban trayectorias más completas, como movimientos en zigzag, trayectorias circulares con cambios de altura y una ruta final por *waypoints*. Con estas pruebas se intenta demostrar que el sistema mantiene el seguimiento cuando el UAV objetivo se mueve de forma dinámica y aparecen variaciones de posición, escala y orientación.
- **Prueba con cámara real.** Finalmente, se realiza una primera validación con el UAV objetivo físico, centrada en la estimación de distancia a partir del tamaño aparente de la caja detectada. Con esta prueba se intenta demostrar que la percepción desarrollada en simulación puede trasladarse a imágenes reales y producir una estimación de distancia coherente en rangos cercanos y medios.

Esta planificación permite avanzar desde una validación aislada de cada módulo hasta una evaluación global del sistema. Así, los resultados de cada prueba no solo indican si un bloque funciona, sino también qué limitaciones puede introducir cuando se integra con el resto de la arquitectura.

5.2. Desarrollo de los Experimentos

5.2.1. Métricas de Evaluación

Para comparar las distintas pruebas de esta sección se emplearon métricas asociadas a cuatro aspectos del sistema: la calidad del detector visual, el rendimiento de inferencia, la respuesta de los controladores y la continuidad del seguimiento del objetivo. De este modo, no solo se analiza si el UAV objetivo se detecta correctamente, sino también si las señales generadas por la percepción son suficientemente estables para alimentar el lazo de control.

- **Selección del detector YOLO.** En este experimento se comparan modelos de detección. Se utilizan el número de parámetros, el tiempo

de inferencia, mAP_{50} y mAP_{50-95} . El tiempo medio de inferencia se calcula como:

$$t_{\text{inf}} = \frac{1}{N} \sum_{i=1}^N t_i,$$

donde t_i es el tiempo necesario para procesar la imagen i y N es el número total de imágenes evaluadas. Esta métrica sirve para saber si el detector puede ejecutarse suficientemente rápido dentro del lazo de control. La precisión se mide mediante:

$$\text{mAP}_{50} = \frac{1}{C} \sum_{c=1}^C AP_c(0.50),$$

$$\text{mAP}_{50-95} = \frac{1}{10C} \sum_{\tau \in \{0.50, 0.55, \dots, 0.95\}} \sum_{c=1}^C AP_c(\tau).$$

En estas expresiones, AP_c es el área bajo la curva precisión–recobrado de la clase c , C es el número de clases y τ representa el umbral de IoU . La métrica mAP_{50} indica la calidad general de detección, mientras que mAP_{50-95} es más exigente porque penaliza cajas delimitadoras peor ajustadas. El número de parámetros complementa estas métricas indicando el tamaño y coste computacional del modelo.

- **Optimización del detector con TensorRT.** En este experimento se evalúa si el detector seleccionado puede ejecutarse con menor latencia en la plataforma Jetson. Para ello se separa el tiempo total de procesamiento en tres partes:

$$t_{\text{total}} = t_{\text{pre}} + t_{\text{inf}} + t_{\text{post}},$$

donde t_{pre} es el tiempo de preprocesamiento, t_{inf} es el tiempo de inferencia y t_{post} es el tiempo de postprocesamiento. A partir del tiempo total se calcula la frecuencia máxima teórica:

$$\text{FPS}_{\text{teor}} = \frac{1000}{t_{\text{total}}},$$

expresando t_{total} en milisegundos. Estas métricas sirven para comprobar si la conversión a TensorRT reduce la latencia y si queda margen temporal para ejecutar, además del detector, los nodos de ROS, MAVROS, filtrado y control.

- **Validación individual de los controladores PID.** En estas pruebas se evalúa cada eje de control por separado. Para el centrado visual se utilizan los errores normalizados:

$$E_{x,i} = \frac{u_i - W/2}{W/2}, \quad E_{y,i} = \frac{v_i - H/2}{H/2},$$

donde (u_i, v_i) es el centro de la caja detectada en el frame i , y W y H son la anchura y la altura de la imagen. El error E_x sirve para evaluar el PID de *yaw*, ya que mide el desplazamiento horizontal del objetivo. El error E_y sirve para evaluar el PID de altura, ya que mide el desplazamiento vertical. Para el eje longitudinal se utiliza:

$$E_{z,i} = d_i - d_{\text{ref}},$$

donde d_i es la distancia estimada al objetivo y d_{ref} es la distancia deseada de seguimiento. Además, se calcula el tiempo de establecimiento:

$$T_s = \min\{t_s : |e(t)| \leq \varepsilon, \forall t \geq t_s\}.$$

Esta métrica indica cuánto tarda cada controlador en entrar y permanecer dentro de una banda de tolerancia ε . En conjunto, estas medidas permiten comprobar si cada PID reduce su error de forma estable antes de combinar los tres ejes.

- **Prueba conjunta de control visual.** En este experimento se activan simultáneamente los PID de *yaw*, altura y distancia. Para resumir el error de centrado en imagen se utiliza el EPE 2D:

$$\text{EPE}_i = \sqrt{E_{x,i}^2 + E_{y,i}^2}.$$

Esta métrica combina el error horizontal y vertical en una única magnitud, por lo que sirve para saber si el objetivo se mantiene cerca del centro de la cámara. Para el eje longitudinal se utiliza el RMSE de distancia:

$$\text{RMSE}_z = \sqrt{\frac{1}{|\mathcal{I}_V|} \sum_{i \in \mathcal{I}_V} E_{z,i}^2}.$$

El RMSE penaliza más los errores grandes que los pequeños y permite detectar desviaciones importantes respecto a la separación deseada. Junto con la evolución temporal de E_z , esta métrica sirve para comprobar que el controlador actúa de forma compatible y sin oscilaciones crecientes.

- **Seguimiento en trayectorias simuladas.**

En estas pruebas se evalúa el lazo cerrado completo durante trayectorias dinámicas. La continuidad de la percepción se mide mediante:

$$P_{\text{YOLO}} = \frac{N_{\text{YOLO}}}{N} \cdot 100, \quad P_{\text{Kalman}} = \frac{N_{\text{Kalman}}}{N} \cdot 100,$$

$$\text{TLR} = \frac{N_{\text{perdidos}}}{N} \cdot 100.$$

En estas expresiones, N es el número total de frames, N_{YOLO} es el número de frames con detección directa, N_{Kalman} es el número de frames mantenidos por predicción y N_{perdidos} es el número de frames sin referencia válida. P_{YOLO} sirve para medir la disponibilidad del detector, P_{Kalman} indica cuánto apoyo requiere el filtro predictivo y TLR mide la tasa de pérdida completa del objetivo. Además, el error de centrado se resume por estado de percepción:

$$\begin{aligned} \text{EPE}_{\text{YOLO}} &= \frac{1}{|\mathcal{I}_{\text{YOLO}}|} \sum_{i \in \mathcal{I}_{\text{YOLO}}} \text{EPE}_i, \\ \text{EPE}_{\text{Kalman}} &= \frac{1}{|\mathcal{I}_{\text{Kalman}}|} \sum_{i \in \mathcal{I}_{\text{Kalman}}} \text{EPE}_i, \\ \text{EPE}_{\text{total}} &= \frac{1}{|\mathcal{I}_V|} \sum_{i \in \mathcal{I}_V} \text{EPE}_i. \end{aligned}$$

Esta separación permite comprobar si el seguimiento se mantiene principalmente con detecciones directas o si depende de la predicción. Los frames sin referencia válida no se promedian en EPE ni en RMSE; si el código marca una medida con un valor como -999.0 , dicho valor se interpreta como pérdida de percepción y se contabiliza mediante el TLR.

- **Prueba con cámara real.** En este experimento se evalúa la estimación de distancia con imágenes reales. Para cada medida se calcula el error de distancia:

$$e_{d,i} = \hat{d}_i - d_i^{\text{real}},$$

donde $\hat{d}_i$ es la distancia estimada a partir de la caja detectada y d_i^{real} es la distancia medida físicamente. El error absoluto se define como:

$$|e_{d,i}| = |\hat{d}_i - d_i^{\text{real}}|,$$

y resume cuánto se desvía la estimación en cada muestra sin tener en cuenta el signo. Para resumir el comportamiento global se utiliza el error medio absoluto:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |e_{d,i}|,$$

y la desviación típica:

$$\sigma_d = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (e_{d,i} - \bar{e}_d)^2}.$$

El MAE sirve para cuantificar el error medio de estimación, mientras que σ_d indica la dispersión entre repeticiones. Analizar estas métricas en función de la distancia real permite identificar en qué rango la estimación es más fiable y a partir de qué separación comienza a degradarse.

5.2.2. Selección del Modelo

El primer experimento tuvo como objetivo seleccionar el detector más adecuado para el sistema de seguimiento visual. Para ello se realizó un torneo comparativo entre distintas arquitecturas de la familia YOLO, utilizando el dataset descrito previamente y manteniendo las mismas condiciones de entrenamiento para todos los modelos. El experimento se ejecutó en Google Colab, utilizando una GPU Nvidia T4 para entrenar de forma secuencial las variantes YOLOv8, YOLOv10, YOLO11 y YOLO26 en sus versiones *nano* (n), *small* (s) y *medium* (m).

La comparación se planteó como un proceso automatizado en el que se recorría una lista de pesos preentrenados. Para cada arquitectura se cargaba el modelo correspondiente, se inicializaba a partir de sus pesos base y se realizaba un ajuste fino sobre el dataset del proyecto. De esta forma, todos los modelos partían de conocimiento visual previo y se especializaban posteriormente en la detección del UAV objetivo, siguiendo el mismo enfoque de *fine-tuning* empleado para YOLOv8n en el capítulo de implementación.

Todos los entrenamientos se realizaron con una configuración común: 25 épocas, tamaño de imagen de 640×640 píxeles y *batch size* de 16. Además, se mantuvo la selección automática del optimizador mediante `optimizer=auto`, de forma que Ultralytics determinara los hiperparámetros más adecuados para cada caso. Cada ejecución se guardó en un directorio independiente, identificado con el nombre del modelo entrenado, lo que permitió comparar posteriormente las métricas obtenidas de forma ordenada y homogénea.

Los resultados del torneo se resumen en la Tabla 5.1. Para la comparación se utilizaron cuatro criterios principales: número de parámetros, *mAP50*, *mAP50-95* y tiempo de inferencia. Estos indicadores permiten evaluar no solo la precisión del detector, sino también su coste computacional y su idoneidad para integrarse en un lazo de control en tiempo real.

A partir de estos resultados se seleccionó YOLOv8n como modelo principal para el sistema de percepción. La elección no se basó únicamente en obtener el mayor valor de precisión absoluta, sino en encontrar el mejor compromiso entre precisión, velocidad de inferencia y tamaño del modelo. Este criterio es especialmente relevante en un sistema de seguimiento visual para

Cuadro 5.1: Comparativa de modelos YOLO evaluados para la detección visual.

Modelo	Parámetros (M)	mAP50	mAP50-95	Inferencia (ms)
YOLOv8n	3.2	0.992507	0.733692	2.660333
YOLOv8s	11.2	0.993716	0.734467	6.248403
YOLOv8m	25.9	0.993991	0.750974	14.657198
YOLOv10n	2.3	0.988186	0.711153	3.258539
YOLOv10s	7.2	0.989443	0.726444	6.054465
YOLOv10m	16.4	0.983691	0.718956	14.397108
YOLO11n	2.6	0.993189	0.728236	2.814279
YOLO11s	9.4	0.994793	0.739400	6.217230
YOLO11m	20.1	0.990841	0.716693	17.582637
YOLO26n	2.4	0.986090	0.717900	2.899324
YOLO26s	9.5	0.991955	0.732011	6.551616
YOLO26m	20.04	0.988004	0.729559	17.643815

UAV, donde el detector no trabaja de forma aislada, sino como parte de un lazo cerrado que debe proporcionar medidas frecuentes al controlador.

Desde el punto de vista de la precisión, YOLOv8n obtuvo un $mAP50$ de 0.992507 y un $mAP50-95$ de 0.733692, valores suficientemente altos para detectar el UAV objetivo de forma fiable. Aunque YOLOv8m alcanzó el mayor $mAP50-95$ de la tabla, con 0.750974, esta mejora fue reducida frente al incremento de coste computacional: YOLOv8m utiliza 25.9 millones de parámetros y requiere 14.657198 ms de inferencia, frente a los 3.2 millones de parámetros y 2.660333 ms de YOLOv8n. Por tanto, la ganancia de precisión no compensaba el aumento de tamaño ni el retardo introducido.

La comparación con YOLOv8s refuerza esta decisión, obteniendo un $mAP50-95$ muy similar, de 0.734467, pero aumentó el número de parámetros hasta 11.2 millones y el tiempo de inferencia hasta 6.248403 ms. Es decir, la mejora en precisión fue prácticamente despreciable para el objetivo del proyecto, mientras que el coste computacional creció de forma considerable. En un sistema de control visual, este incremento puede traducirse en menor frecuencia de actualización, mayor retardo en las correcciones y peor capacidad de reacción ante movimientos rápidos del UAV objetivo.

El resto de familias tampoco ofreció un compromiso superior. YOLOv10n y YOLO26n presentan menos parámetros que YOLOv8n, pero obtienen peores métricas de precisión y tiempos de inferencia superiores. YOLO11n mantiene un tamaño reducido y un $mAP50$ elevado, pero su $mAP50-95$ queda por debajo del de YOLOv8n y su inferencia también es ligeramente más lenta. Las variantes *small* y *medium* de YOLOv10, YOLO11 y YOLO26 incrementan el número de parámetros y el tiempo de inferencia sin aportar una mejora global suficiente para justificar su uso.

Por todo ello, YOLOv8n fue considerado el modelo más adecuado para el sistema implementado. Su principal ventaja es que mantiene una precisión elevada con el menor tiempo de inferencia de todos los modelos evaluados. Esta combinación permite alimentar al controlador con detecciones rápidas y suficientemente estables, reduciendo el riesgo de respuestas tardías, oscilaciones o pérdida temporal del objetivo durante las maniobras de seguimiento.

5.2.3. Optimización del Modelo en Dron Real con TensorRT

La selección de un detector ligero no es suficiente por sí sola cuando el sistema debe ejecutarse en un dron real. En este proyecto, el modelo de percepción está pensado para funcionar dentro de un sistema embarcado basado en una Jetson Origin Nano, donde los recursos de cómputo, memoria y consumo energético son más limitados que en un ordenador de sobremesa. Además, la detección no se utiliza de forma aislada: cada imagen procesada debe alimentar al filtro de Kalman y a los controladores PID del lazo IBVS, por lo que la inferencia debe realizarse con una latencia baja y de forma sostenida en el tiempo.

En este contexto, optimizar el modelo resulta necesario para que el detector pueda trabajar en tiempo real junto con el resto de nodos del sistema. Si la red neuronal introduce demasiado retardo, las medidas visuales llegan tarde al controlador y la respuesta del UAV cazador puede volverse menos precisa, especialmente cuando el objetivo realiza maniobras rápidas. Por este motivo, tras seleccionar YOLOv8n como detector principal, se evaluó su conversión a TensorRT, una herramienta orientada a acelerar la inferencia de redes neuronales en plataformas NVIDIA.

El experimento consistió en comparar el rendimiento del modelo original entrenado, almacenado en formato `.pt`, frente al mismo modelo exportado como motor TensorRT en formato `.engine`. La conversión no modifica el entrenamiento ni cambia la arquitectura utilizada para detectar el UAV, sino que transforma la red a una representación más eficiente para la GPU de la Jetson. Para ello se utilizó precisión reducida FP16, manteniendo el tamaño de entrada de 640 píxeles:

```
yolo export \
  model=~/.catkin_ws/src/drone_lap/models/best.pt \
  format=engine \
  half=True \
  device=0 \
  workspace=2 \
  imgsz=640
```

La comparativa de rendimiento se realizó sobre 100 fotogramas con re-

solución de entrada 640×480 píxeles. En cada ejecución se midieron por separado los tiempos de preprocesamiento, inferencia en GPU y postprocesamiento, además del tiempo total medio por frame y los FPS máximos teóricos. Esta separación permite identificar si la mejora procede realmente de la ejecución de la red neuronal o de otras etapas del flujo de percepción. Los resultados se resumen en la Tabla 5.2.

Cuadro 5.2: Comparativa de rendimiento antes y después de la optimización con TensorRT.

Ejecución	Preproc. (ms)	Infer. GPU (ms)	Postproc. (ms)	Total (ms/frame)	FPS teóricos
Modelo original .pt	2.04	17.11	3.37	22.53	44.4
TensorRT .engine FP16	2.26	5.75	3.73	11.74	85.2

La mejora principal se observa en la fase de inferencia. El tiempo de GPU pasó de 17.11 ms a 5.75 ms, lo que supone una reducción aproximada del 66.4 %. Esta diferencia se debe a que TensorRT genera un motor adaptado a la GPU concreta, selecciona operaciones más eficientes y aprovecha la precisión FP16. En la práctica, esto permite ejecutar la misma red neuronal con menor tiempo de cómputo, liberando recursos para el resto del sistema embarcado.

El tiempo total por fotograma se redujo de 22.53 ms a 11.74 ms, una disminución cercana al 47.9 %. En un sistema IBVS, este dato es especialmente importante porque el retardo entre la captura de la imagen y la reacción del UAV afecta directamente a la estabilidad del lazo de control. Al reducir la latencia, las medidas visuales que llegan al filtro de Kalman y a los controladores PID representan mejor el estado actual del objetivo, disminuyendo el riesgo de correcciones tardías cuando el UAV objetivo realiza maniobras rápidas.

También se aprecia un cambio en el cuello de botella del sistema. En la ejecución original, la inferencia en GPU representaba aproximadamente el 75.9 % del tiempo total, por lo que la red neuronal era el componente dominante. Tras la conversión a TensorRT, la inferencia baja hasta el 49.0 % del tiempo total y el conjunto de preprocesamiento y postprocesamiento pasa a tener un peso similar. Esto indica que la optimización ha desplazado la limitación principal desde la red neuronal hacia etapas auxiliares, ejecutadas principalmente mediante operaciones clásicas de tratamiento de imagen y preparación de datos.

La cámara utilizada trabaja a 30 FPS, por lo que cada imagen llega aproximadamente cada 33.3 ms. Con el modelo original, el procesamiento consumía alrededor del 67.7 % de esa ventana temporal. Con TensorRT, el consumo baja hasta aproximadamente el 35.3 %. Aunque la cámara no per-

mite aprovechar directamente los 85.2 FPS teóricos, este margen adicional resulta muy valioso: reduce la probabilidad de acumulación de retardos, facilita la ejecución concurrente de ROS, MAVROS y los controladores, y deja capacidad disponible para registrar datos, aplicar filtros o incorporar tareas adicionales sin saturar la Jetson.

En consecuencia, la conversión a TensorRT no solo aumenta la velocidad máxima teórica, que pasa de 44.4 a 85.2 FPS, sino que mejora la robustez temporal del sistema completo. El detector queda preparado para sensores de mayor frecuencia en trabajos futuros, pero incluso con una cámara de 30 FPS aporta beneficios directos: menor latencia, mayor margen de seguridad computacional, menor carga de GPU y una integración más adecuada para ejecución embarcada durante pruebas reales.

5.2.4. Pruebas de Control

Pruebas Individuales en Simulación

Estas pruebas sirven para comprobar por separado los tres controladores principales del sistema: *yaw*, altura y distancia. Antes de activar el seguimiento completo, es necesario verificar que cada eje reduce su propio error de forma estable y que ninguna respuesta individual introduce oscilaciones o maniobras bruscas.

Con este bloque se responde a la pregunta de si cada PID es válido como componente aislado del lazo de control visual. Para ello, cada experimento excita principalmente un único eje mediante una trayectoria específica, se registra la evolución del error asociado y, después, se interpretan los resultados antes de combinar los tres controladores en una prueba conjunta.

Prueba del PID de *yaw* con trayectoria circular

Esta prueba sirve para validar el PID de *yaw*, responsable de corregir el desplazamiento horizontal del objetivo en la imagen. La pregunta que se busca responder es si el UAV cazador puede girar de forma continua y estable para mantener al UAV objetivo cerca del centro horizontal de la cámara cuando este se desplaza lateralmente alrededor de él.

El experimento consiste en mantener el UAV cazador en una posición central mientras el UAV objetivo describe una trayectoria circular a su alrededor con un radio de 4 metros. Esta configuración genera un error horizontal variable y obliga al controlador a transformar dicho error visual en una consigna de giro de *yaw*.

La trayectoria circular se generó actualizando de forma periódica la posición del UAV objetivo en el plano horizontal. Para ello, se calculó una velocidad angular a partir de la relación entre la velocidad lineal deseada y

el radio de la circunferencia. Con un periodo de actualización de 20 Hz, en cada iteración se incrementó el tiempo de simulación, se obtuvo el ángulo instantáneo de la trayectoria y se calcularon las coordenadas del objetivo mediante las funciones trigonométricas seno y coseno. La altura se mantuvo constante durante toda la prueba, de modo que la variación principal observada por la cámara correspondiera al desplazamiento horizontal del objetivo.

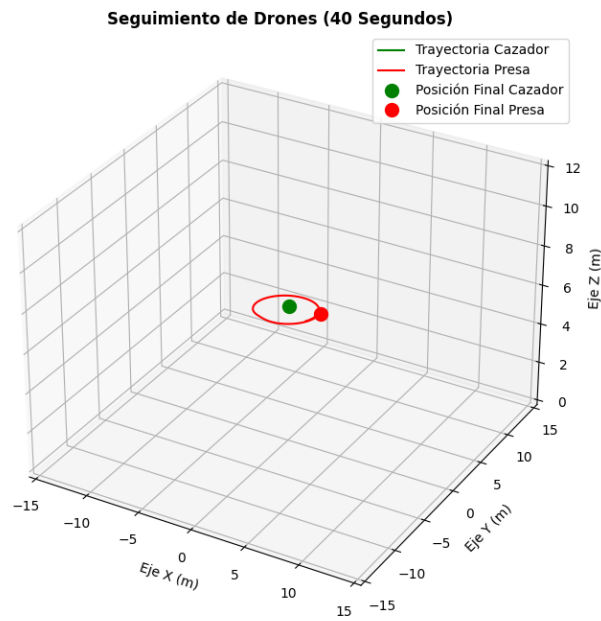


Figura 5.1: Trayectoria circular utilizada para la prueba del PID de *yaw*. (Imagen de Elaboración Propia)

Como se observa en la Figura 5.1, el UAV objetivo realiza una circunferencia alrededor del UAV cazador, manteniendo una altura constante. Esta trayectoria resulta adecuada para analizar el control de *yaw*, ya que obliga al sistema a corregir continuamente el error horizontal sin introducir cambios relevantes en el eje vertical.

La respuesta obtenida por el controlador se muestra en la Figura 5.2. La señal representada corresponde al error angular horizontal, denominado E_x , mientras que la línea discontinua roja indica la referencia deseada, es decir, error nulo. Al inicio de la prueba aparece un error negativo debido a

que el objetivo no se encuentra perfectamente centrado en la imagen. Sin embargo, el PID corrige rápidamente esta desviación y lleva la señal hacia valores próximos a cero.

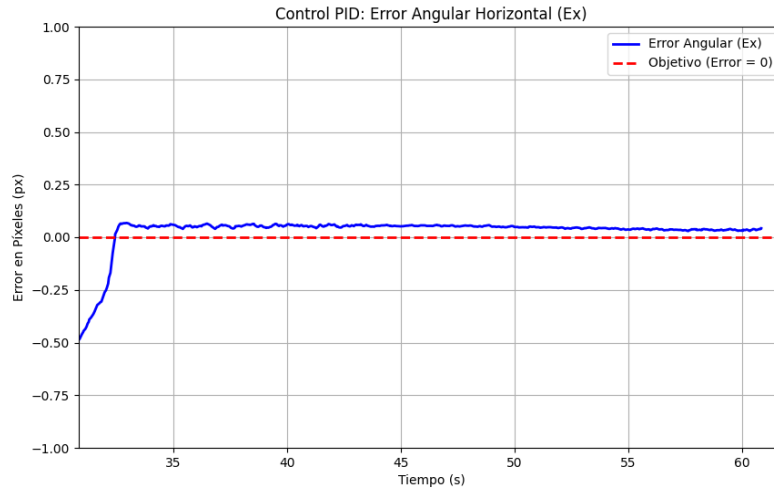


Figura 5.2: Respuesta del PID de *yaw* durante la trayectoria circular. (Imagen de Elaboración Propia)

La evolución de la señal muestra una respuesta estable tras el transitorio inicial. Aunque se aprecia un pequeño error residual positivo, este permanece acotado y con oscilaciones reducidas, lo que indica que el controlador mantiene el objetivo cerca del centro horizontal de la imagen durante la trayectoria circular. Por tanto, esta prueba confirma que el PID de *yaw* es capaz de compensar el movimiento lateral relativo del UAV objetivo y generar una respuesta suficientemente suave para el seguimiento visual.

Prueba del PID de altura con trayectoria vertical

Esta prueba sirve para validar el PID de altura, encargado de corregir el desplazamiento vertical del objetivo dentro de la imagen. Con ella se responde a la pregunta de si el UAV cazador puede modificar su altura de forma suave para mantener centrado al objetivo cuando este sube y baja durante el vuelo.

El experimento consiste en situar el UAV objetivo alrededor de una altura de referencia de 7 metros y hacerlo oscilar verticalmente alrededor de dicha posición. De esta manera se genera un error vertical controlado, sin introducir cambios dominantes en el eje horizontal, y se puede analizar la respuesta específica del controlador de altura.

La trayectoria se generó mediante una oscilación sinusoidal aplicada únicamente sobre la coordenada vertical del UAV objetivo. Para ello, se definió un valor central de altura, una amplitud de oscilación y un periodo de movimiento. En cada iteración, con una frecuencia de actualización de 20 Hz, se incrementó el tiempo de simulación y se calculó la nueva altura como la suma de la altura central y una componente sinusoidal. Además, se mantuvo fija la posición horizontal del UAV objetivo, de forma que la prueba excitara principalmente el eje vertical y no mezclara la respuesta con correcciones de *yaw* o desplazamiento lateral. También se incluyó una limitación inferior de seguridad para evitar que la altura objetivo pudiera tomar valores demasiado bajos durante la simulación.

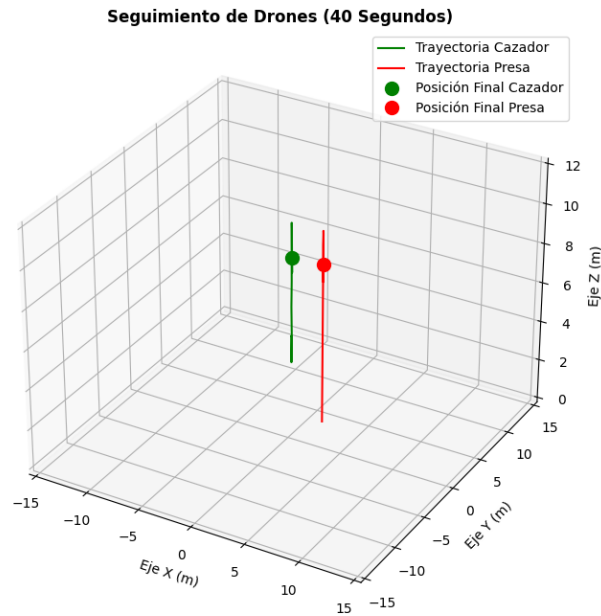


Figura 5.3: Trayectoria vertical utilizada para la prueba del PID de altura. (Imagen de Elaboración Propia)

En la Figura 5.3 se observa que el movimiento del UAV objetivo se concentra en el eje vertical, mientras el UAV cazador permanece en una posición de referencia. Esta configuración permite analizar de forma directa la capacidad del controlador de altura para seguir movimientos ascendentes y descendentes del objetivo sin introducir cambios en el plano horizontal.

La respuesta del controlador se representa en la Figura 5.4. La señal verde corresponde al error vertical E_y , mientras que la línea roja discontinua marca la referencia de error nulo. A lo largo de la prueba, el error se mantiene en valores reducidos alrededor de cero, con pequeñas oscilaciones asociadas al movimiento sinusoidal del objetivo.

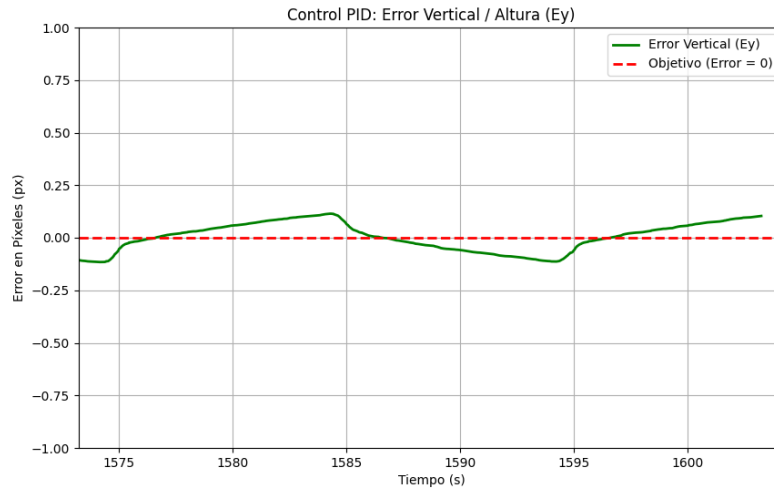


Figura 5.4: Respuesta del PID de altura durante la trayectoria vertical. (Imagen de Elaboración Propia)

La evolución de la señal muestra que el controlador responde de forma suave ante las variaciones de altura. No aparecen oscilaciones crecientes ni sobreimpulsos elevados, y el error vertical permanece acotado durante toda la trayectoria. Aunque se aprecia una ligera desviación respecto a la referencia en algunos instantes, esta se corrige progresivamente y se mantiene dentro de un margen pequeño. Por tanto, la prueba confirma que el PID de altura es capaz de compensar los movimientos verticales del UAV objetivo y mantener una respuesta estable para el seguimiento visual.

Prueba del PID de distancia con trayectoria recta

Esta prueba sirve para validar el PID de distancia, encargado de regular el avance longitudinal del UAV cazador respecto al UAV objetivo. La pregunta que se busca responder es si el dron cazador puede aproximarse y seguir a un objetivo en movimiento sin invadir la distancia de seguridad definida para el seguimiento.

El experimento consiste en hacer que el UAV objetivo siga una trayectoria recta a una velocidad constante de 3 m/s, avanzando un total aproximado

de 25 metros. Durante esta maniobra, el controlador de distancia debe adaptar la velocidad frontal del UAV cazador para reducir la separación inicial y mantener una distancia longitudinal controlada.

Aunque el planteamiento general del TFG está orientado a tareas de interceptación, en las pruebas de validación se decidió no forzar una colisión ni un contacto directo entre ambos UAV. Esta decisión permite que los experimentos sean repetibles, comparables y más fáciles de trasladar posteriormente a pruebas reales. Por este motivo, el sistema se configuró para mantener una distancia de seguridad de 1 metro y medio respecto al UAV objetivo. De este modo, se evalúa la capacidad de aproximación y seguimiento sin comprometer la estabilidad de la simulación ni la seguridad de una posible plataforma física.

La trayectoria recta se generó desplazando progresivamente el UAV objetivo en una única dirección, manteniendo constante su altura y su orientación. Al imponer una velocidad de 3 m/s durante el recorrido, el controlador de distancia debe aumentar o reducir la velocidad frontal del UAV cazador para acercarse al objetivo y conservar la referencia deseada. Esta prueba resulta especialmente relevante porque combina una condición sencilla de analizar con una exigencia directa sobre el control longitudinal.

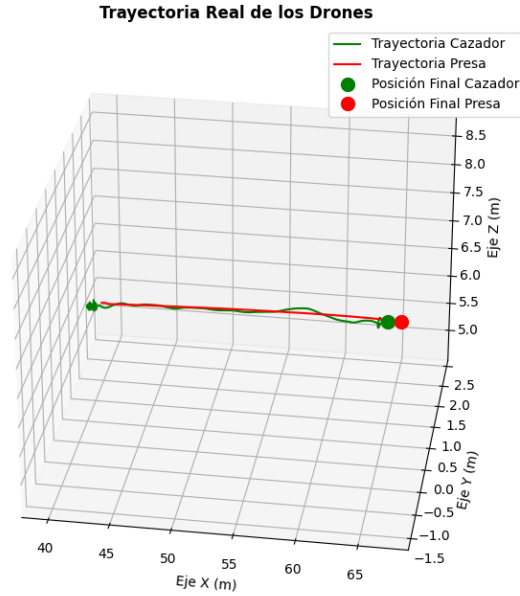


Figura 5.5: Trayectoria recta utilizada para la prueba del PID de distancia. (Imagen de Elaboración Propia)

En la Figura 5.5 se representa el desplazamiento rectilíneo del UAV objetivo y la respuesta del UAV cazador durante el seguimiento. La prueba permite observar si el controlador longitudinal es capaz de reducir la separación inicial y acompañar al objetivo durante el avance, manteniendo la distancia de seguridad establecida.

La respuesta del PID de distancia se muestra en la Figura 5.6. En esta gráfica se analiza la evolución del error de distancia asociado a la diferencia entre la distancia estimada al objetivo y la distancia de referencia. Una respuesta adecuada debe tender hacia valores próximos a cero, lo que indica que el sistema ha alcanzado la separación deseada y que el UAV cazador acompaña al objetivo sin alejarse ni acercarse en exceso.

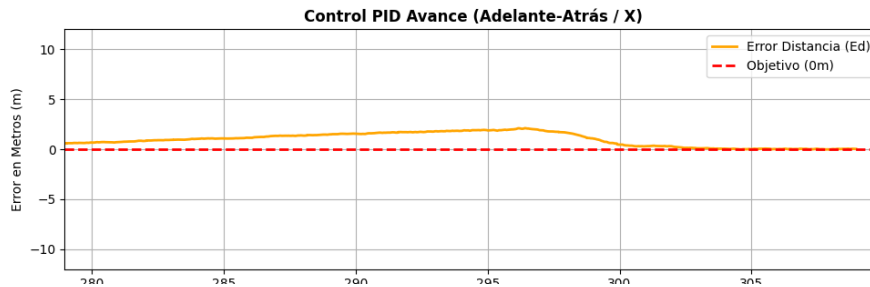


Figura 5.6: Respuesta del PID de distancia durante la trayectoria recta. (Imagen de Elaboración Propia)

Los resultados muestran un comportamiento satisfactorio del controlador longitudinal. Tras el transitorio inicial, el error de distancia se reduce y permanece acotado, lo que indica que el UAV cazador es capaz de adaptarse al movimiento del objetivo a 3 m/s. Además, el seguimiento se realiza manteniendo la referencia de seguridad de 1 metro y medio, por lo que el sistema no solo se aproxima al objetivo, sino que también evita una interceptación directa. Esta respuesta confirma que el PID de distancia proporciona una base adecuada para escenarios de persecución controlada y para futuras pruebas reales donde la repetibilidad y la seguridad son requisitos fundamentales.

Tiempos de establecimiento de los controladores PID

Además de comprobar la estabilidad de cada controlador durante trayectorias continuas, se realizaron tres pruebas específicas para estimar su tiempo de establecimiento. Estas pruebas sirven para cuantificar la rapidez de respuesta de cada PID, no solo para observar si finalmente alcanza la referencia.

Con ellas se responde a la pregunta de cuánto tarda cada eje en reducir su error cuando parte de una desviación inicial conocida. Esta medida es importante porque, en un sistema de seguimiento visual, no basta con que el controlador sea estable: también debe reaccionar con suficiente rapidez para que el objetivo no salga del campo de visión.

El experimento consiste en colocar inicialmente el UAV cazador con un error controlado respecto al UAV objetivo y registrar el tiempo necesario para que la señal entre y permanezca dentro de una banda de tolerancia alrededor de la referencia. Para que los resultados fueran representativos del caso de uso del proyecto, las pruebas se plantearon con distancias cortas entre drones, ya que el sistema está orientado a seguimiento cercano del UAV objetivo y no a navegación a larga distancia.

En la prueba del PID de *yaw*, el UAV cazador se situó inicialmente 1

metro desplazado hacia la izquierda respecto al objetivo, manteniendo una separación aproximada de 3 metros entre ambos drones. Esta configuración genera un error horizontal claro en la imagen sin introducir de forma dominante errores verticales o longitudinales. La respuesta obtenida se muestra en la Figura 5.7.

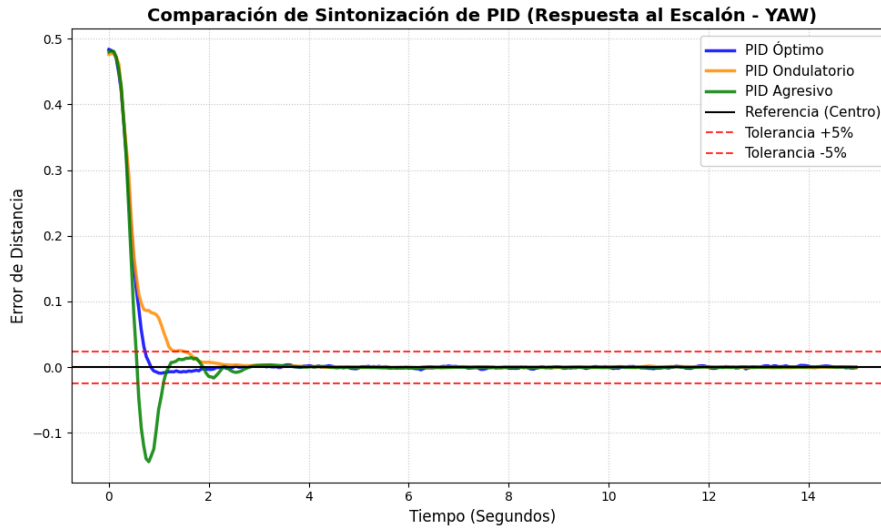


Figura 5.7: Tiempo de establecimiento del PID de *yaw*. (Imagen de Elaboración Propia)

El tiempo de establecimiento obtenido para el PID de *yaw* fue de aproximadamente 0.65 s. Este valor es el menor de las tres pruebas porque la corrección de orientación se realiza mediante una velocidad angular, sin requerir un desplazamiento lineal completo del UAV cazador. Por ello, el sistema puede reducir rápidamente el error horizontal y centrar el objetivo en la imagen con una respuesta directa y poco oscilatoria.

En la prueba del PID de altura, el UAV cazador se colocó con un desfase vertical de 1 metro hacia arriba, manteniendo también una distancia aproximada de 3 metros respecto al UAV objetivo. Con esta configuración se fuerza un error vertical inicial que debe ser compensado por el controlador de altura, mientras que la separación cercana entre drones mantiene la prueba dentro del rango habitual de seguimiento visual. La respuesta se representa en la Figura 5.8.

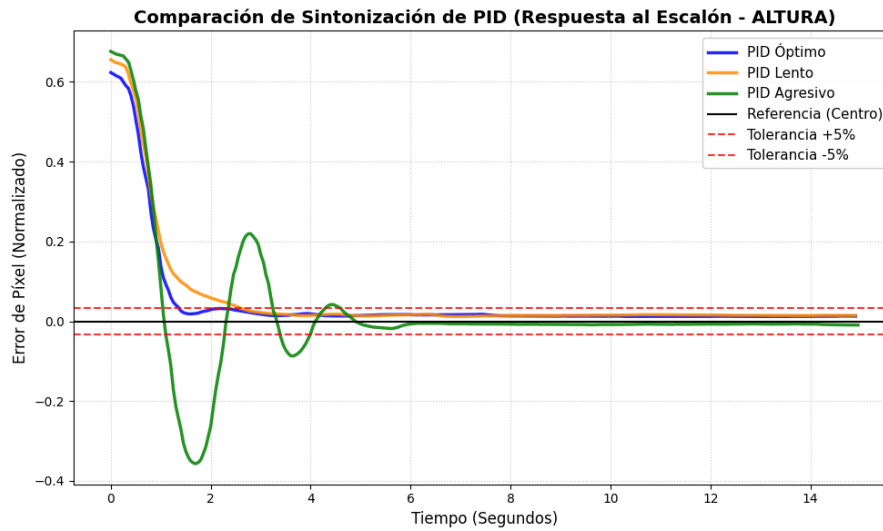


Figura 5.8: Tiempo de establecimiento del PID de altura. (Imagen de Elaboración Propia)

El tiempo de establecimiento del PID de altura fue de aproximadamente 1.25 s. Esta respuesta es más lenta que la del control de *yaw* porque la corrección vertical depende del movimiento físico del UAV y de las limitaciones impuestas por el controlador de vuelo. Dichas limitaciones reducen la velocidad y la aceleración vertical máximas para evitar maniobras bruscas, por lo que el sistema tarda algo más en alcanzar la referencia, aunque mantiene una evolución estable y sin oscilaciones crecientes.

Por último, se evaluó el PID de distancia situando inicialmente los drones con una separación longitudinal de 3 metros. Esta prueba permite analizar la capacidad del controlador para regular el acercamiento al objetivo manteniendo una distancia de seguimiento segura. La elección de una distancia moderada responde a la necesidad de evitar choques entre drones y de hacer que la prueba sea repetible tanto en simulación como en una futura validación física. La respuesta se muestra en la Figura 5.9.

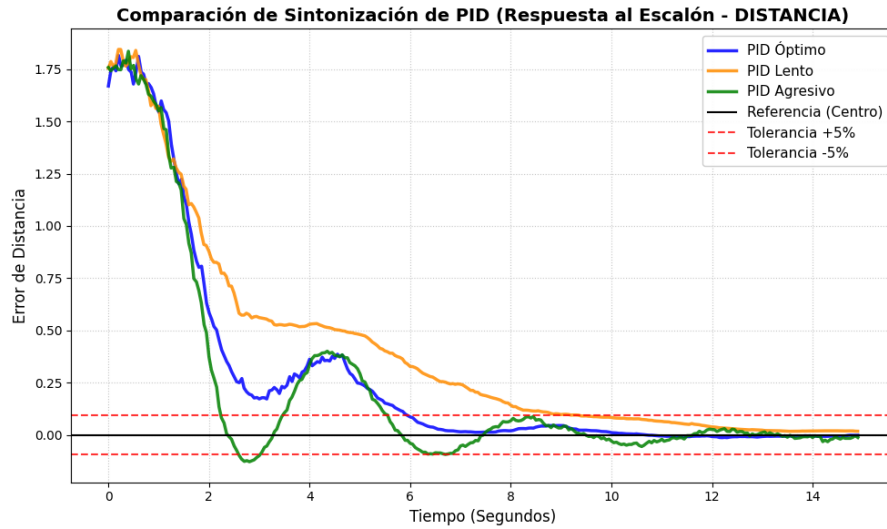


Figura 5.9: Tiempo de establecimiento del PID de distancia. (Imagen de Elaboración Propia)

El tiempo de establecimiento del PID de distancia fue de aproximadamente 6 s, notablemente superior al de los otros dos controladores. Este comportamiento se debe a que el avance longitudinal se ajustó de forma conservadora para no provocar una aproximación excesivamente agresiva al UAV objetivo. Además, debe tenerse en cuenta que este controlador PID está pensado para seguir a un UAV objetivo en movimiento, por lo que en una prueba estática aparece de forma más marcada la dinámica de aceleración y frenado frente a una referencia fija. Cuando el UAV cazador acelera para avanzar, tiende a inclinarse hacia delante; al frenar de forma brusca, realiza el movimiento contrario, generando un pequeño retroceso y variaciones en la estimación visual. Estas oscilaciones hacen que la señal tarde más en estabilizarse, pero también permiten mantener una maniobra más segura y reproducible, reduciendo el riesgo de colisión durante las pruebas.

Cuadro 5.3: Tiempos de establecimiento obtenidos para cada controlador PID.

Controlador	Condición inicial	T_s
PID de <i>yaw</i>	1 m lateral y 3 m de separación	0.65 s
PID de altura	1 m vertical y 3 m de separación	1.25 s
PID de distancia	3 m de separación longitudinal	6 s

En conjunto, estos resultados muestran que los controladores lateral y vertical alcanzan la referencia en tiempos reducidos, mientras que el con-

trol de distancia presenta una respuesta más lenta debido a las restricciones de seguridad y a la propia dinámica longitudinal del UAV. Esta diferencia resulta aceptable para el sistema propuesto, ya que en un escenario de seguimiento cercano es preferible priorizar una aproximación suave y repetible antes que una reducción demasiado rápida de la distancia.

Prueba conjunta de los tres PID activados en Simulación

Una vez validados los controladores de forma individual, se realizó una prueba conjunta con los tres PID activados simultáneamente. Esta prueba sirve para comprobar si las acciones de *yaw*, altura y distancia son compatibles cuando trabajan al mismo tiempo dentro del lazo de seguimiento visual.

Con este experimento se responde a la pregunta de si el sistema completo puede mantener errores acotados cuando el UAV objetivo genera desviaciones horizontales, verticales y longitudinales de manera simultánea. A diferencia de las pruebas anteriores, esta validación no aísla un único eje de control, sino que somete al sistema a una trayectoria tridimensional más exigente y cercana a una situación real de persecución.

El experimento consiste en activar los tres controladores y hacer que el UAV objetivo describa una trayectoria que combine movimiento circular en el plano horizontal con variaciones periódicas de altura. De esta forma, el UAV cazador debe corregir orientación, altura y avance de manera coordinada durante toda la maniobra.

La trayectoria del UAV objetivo se definió combinando un movimiento circular en el plano horizontal con una onda sinusoidal en altura. En el plano XY, el objetivo describe una circunferencia alrededor de un centro de referencia, utilizando una velocidad angular calculada a partir de la velocidad lineal y el radio de giro. Al mismo tiempo, la coordenada vertical se modifica mediante una onda de altura base $Z_0 = 7$ m, amplitud de 1 m y frecuencia de 0.1 Hz. De este modo, la altura del objetivo varía de forma suave alrededor de los 7 metros, con un periodo aproximado de 10 segundos.

Esta configuración permite que el UAV objetivo avance alrededor del cazador mientras asciende y desciende periódicamente. La posición se actualizó a 20 Hz, manteniendo una orientación fija durante la trayectoria, de forma que las variaciones observadas por el sistema provinieran principalmente del movimiento espacial del objetivo. Con ello se buscó evaluar la coordinación entre el PID de *yaw*, encargado del centrado horizontal, el PID de altura, responsable de corregir el error vertical, y el PID de distancia, encargado de mantener la separación longitudinal respecto al objetivo.

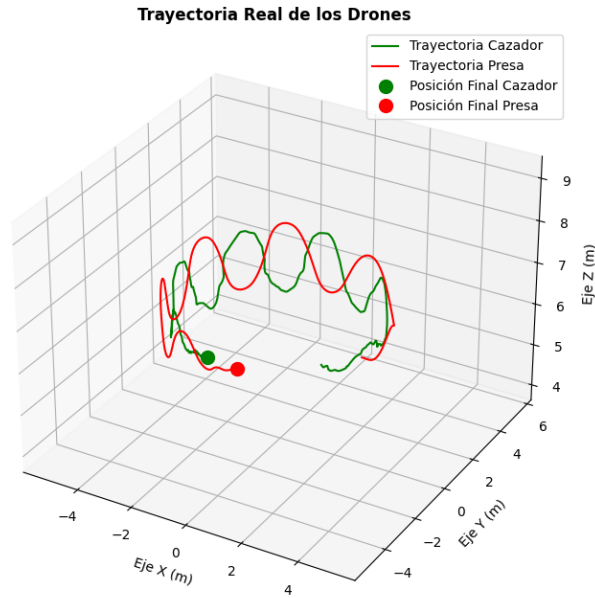


Figura 5.10: Trayectoria circular ondulada utilizada para la prueba conjunta de los tres PID. (Imagen de Elaboración Propia)

En la Figura 5.10 se muestra la trayectoria tridimensional utilizada en la prueba. Se observa que el UAV objetivo no solo se desplaza en círculo, sino que también modifica su altura de forma continua. Esta combinación obliga al UAV cazador a corregir simultáneamente su orientación, su posición vertical y su avance, por lo que resulta adecuada para comprobar el funcionamiento conjunto del sistema de control visual.

La respuesta obtenida durante la ejecución se representa en la Figura 5.11. En esta gráfica se analiza la evolución de las señales asociadas a los tres controladores cuando trabajan de manera simultánea. Una respuesta adecuada debe mantener los errores acotados, evitar oscilaciones crecientes y permitir que el UAV cazador conserve el seguimiento visual del objetivo a pesar de que la trayectoria introduce cambios continuos en los tres ejes.

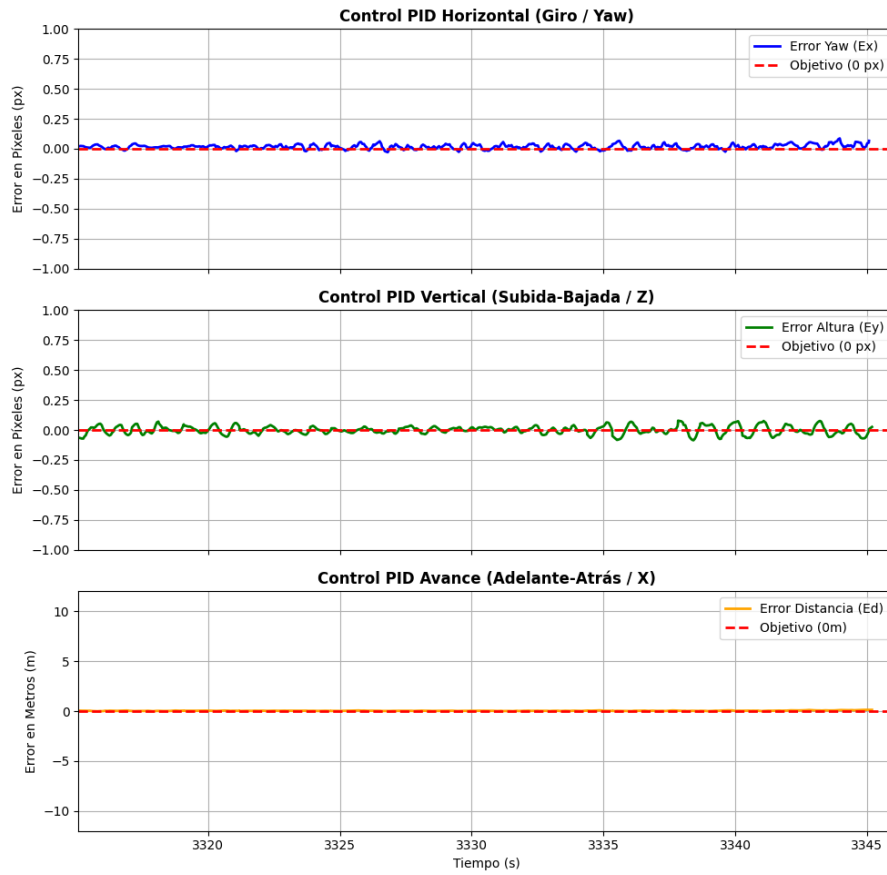


Figura 5.11: Respuesta conjunta de los tres PID durante la trayectoria circular ondulada. (Imagen de Elaboración Propia)

Los resultados muestran que el sistema es capaz de mantener una respuesta estable cuando los tres controladores actúan a la vez. El PID de *yaw* corrige el desplazamiento horizontal producido por el movimiento circular, el PID de altura compensa la variación sinusoidal de la coordenada vertical y el PID de distancia ajusta el avance del UAV cazador para conservar la separación de seguimiento. Aunque la trayectoria genera perturbaciones continuas y pequeñas oscilaciones en las señales de control, estas permanecen acotadas y no derivan en una pérdida del objetivo. Por tanto, esta prueba confirma que la integración de los tres PID permite abordar un escenario de seguimiento tridimensional más completo que las validaciones individuales.

5.2.5. Seguimiento del Objetivo en Simulación

Estas pruebas sirven para validar el comportamiento del sistema cuando el UAV presa realiza trayectorias completas y el UAV cazador debe mantener el seguimiento durante un vuelo continuo. A diferencia de las validaciones individuales de los PID, aquí no se evalúa un controlador aislado, sino la integración de percepción, estimación y control.

Con este bloque se responde a la pregunta de si el sistema completo puede conservar una referencia visual útil cuando el objetivo cambia de posición de forma dinámica. Por ello se analizan las métricas de visión, como la detección directa de YOLO, el apoyo del filtro de Kalman y la tasa de pérdida del objetivo, junto con métricas de control como el EPE de centrado y el RMSE de distancia.

Los experimentos consisten en ejecutar tres escenarios progresivos en simulación: una trayectoria en zigzag, una trayectoria circular ondulada y una ruta final definida por *waypoints*. En todos los casos se registran los frames analizados, el porcentaje de detección directa, el uso de Kalman, el TLR, el error de centrado y el error de distancia para, posteriormente, exponer los resultados y discutir las limitaciones observadas.

Prueba de seguimiento con trayectoria en zigzag

Esta prueba sirve para evaluar el seguimiento visual ante cambios laterales sucesivos. La trayectoria en zigzag obliga al UAV cazador a corregir alternancias en el error horizontal mientras intenta mantener la distancia de referencia y el centrado vertical del objetivo.

Con este experimento se responde a la pregunta de si el sistema puede mantener la referencia visual sin depender de una trayectoria suave o rectilínea. Es una prueba relevante porque los cambios de dirección pueden provocar desplazamientos rápidos de la caja detectada, variaciones en la estimación de distancia y posibles pérdidas temporales de detección.

El experimento consiste en hacer que el UAV presa realice un desplazamiento en zigzag mientras el UAV cazador intenta mantenerlo dentro del campo de visión. Durante el vuelo permanecen activos los tres controladores principales: el PID de *yaw*, encargado de corregir el error horizontal de la imagen; el PID de altura, responsable de reducir el error vertical; y el PID de distancia, utilizado para regular la separación longitudinal respecto al objetivo.

La trayectoria utilizada se muestra en la Figura 5.12. El movimiento alterna desplazamientos laterales a medida que el objetivo avanza, generando cambios de signo en el error horizontal. Esta característica permite compro-

bar si el controlador de *yaw* responde con suficiente rapidez y si el detector mantiene una localización estable del UAV presa durante los cambios de dirección.

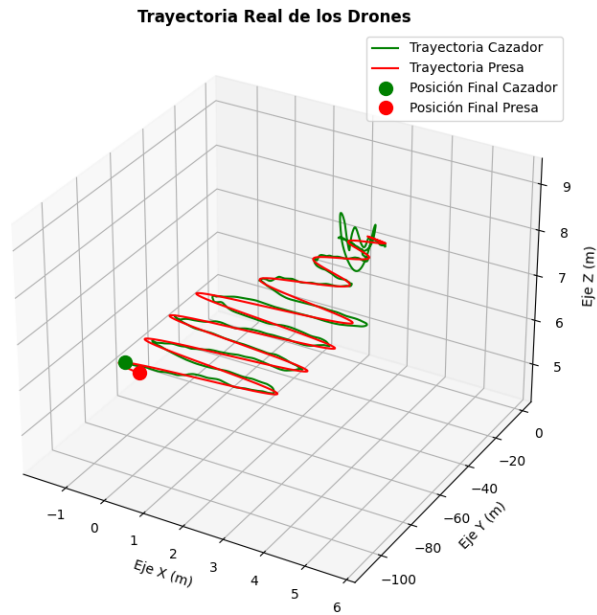


Figura 5.12: Trayectoria en zigzag utilizada para la prueba de seguimiento del objetivo. (Imagen de Elaboración Propia)

La Figura 5.13 muestra una vista del escenario de Gazebo durante esta prueba, con ambos UAV dentro del entorno de simulación utilizado para evaluar el seguimiento.

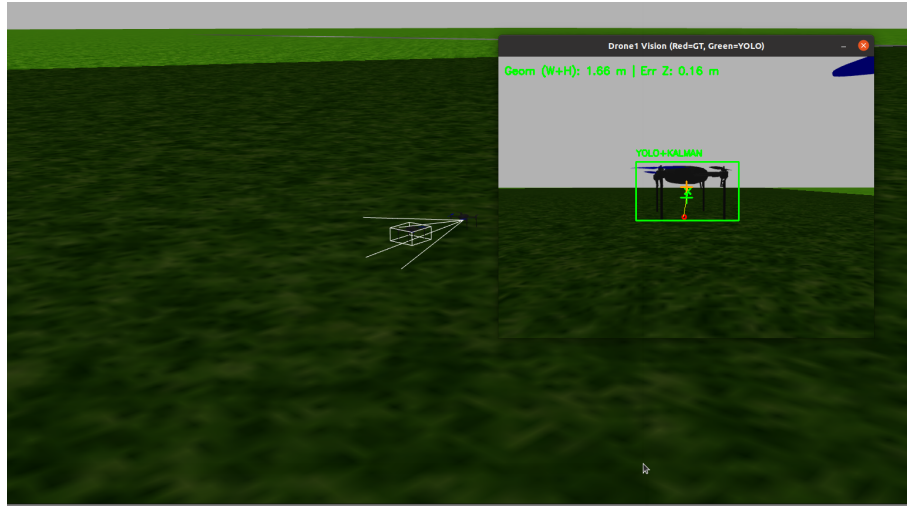


Figura 5.13: Vista del escenario de Gazebo y la camara durante la prueba de seguimiento en zigzag. (Imagen de Elaboración Propia)

Durante la ejecución se registraron las señales del evaluador de métricas del vuelo. El informe final obtenido se resume en la Tabla 5.4. En ella se recogen las métricas de visión, el EPE 2D de centrado en imagen y el RMSE de distancia, manteniendo separadas las pérdidas de percepción respecto al error de control.

Cuadro 5.4: Métricas EPE/RMSE/TLR obtenidas durante la prueba de seguimiento con trayectoria en zigzag.

Métrica	Valor
Frames analizados	3266
Visión activa con YOLO	100.00 %
Tracking ciego con Kalman	0.00 %
Target Loss Rate (TLR)	0.00 %
EPE durante YOLO	0.0550 normalizado
EPE durante Kalman	No aplicable
EPE medio total	0.0550 normalizado
RMSE de distancia (E_z)	1.255 m

Los resultados de visión muestran que el sistema mantuvo detección directa durante toda la maniobra. De los 3266 frames analizados, YOLO proporcionó una referencia válida en el 100.00 % de los casos, no fue necesario recurrir a la predicción de la caja mediante Kalman y el TLR se mantuvo en 0.00 %. Por tanto, no se produjo ninguna pérdida completa del objetivo durante el vuelo.

El EPE medio total fue de 0.0550 en magnitud normalizada, calculado íntegramente durante frames con detección YOLO. Este valor resume el error bidimensional de centrado en la imagen, combinando las componentes horizontal y vertical. Al no haberse usado Kalman, el EPE asociado a este modo se considera no aplicable, lo que refuerza que el seguimiento visual dependió de detecciones directas durante toda la prueba.

El RMSE de distancia fue de 1.255 m. Este valor indica que la separación longitudinal fue la componente más difícil de mantener durante la trayectoria en zigzag. Los cambios laterales sucesivos modifican la posición relativa entre ambos UAV y pueden introducir variaciones en la estimación de profundidad basada en la caja delimitadora, por lo que el error de distancia resulta mayor que los errores normalizados de centrado visual.

En conjunto, la prueba de zigzag confirma que el sistema es capaz de mantener la referencia visual del UAV presa en una trayectoria con cambios laterales sucesivos. Aunque el control de distancia presenta un error mayor que los ejes de centrado visual, la detección se mantiene estable durante toda la secuencia y no se produce pérdida del objetivo.

Prueba de seguimiento con trayectoria circular ondulada

Esta prueba sirve para evaluar el seguimiento cuando el objetivo genera errores simultáneos en *yaw*, altura y distancia. La trayectoria circular ondulada combina desplazamiento circular en el plano horizontal con variaciones sinusoidales de altura, por lo que representa una maniobra tridimensional más completa que el zigzag.

Con este experimento se responde a la pregunta de si la validación conjunta de los tres PID se mantiene también desde el punto de vista de la continuidad visual. Es decir, no solo se comprueba que los controladores puedan actuar de forma simultánea, sino también que la percepción proporcione una referencia suficientemente estable durante toda la maniobra.

El experimento consiste en reutilizar la trayectoria circular ondulada mostrada previamente en la Figura 5.10 y registrar métricas específicas de seguimiento. Para ello se recogieron las métricas EPE/RMSE/TLR del vuelo, incluyendo el error de centrado bidimensional, el error de distancia y los porcentajes de detección directa, seguimiento mediante Kalman y pérdida del objetivo.

La Figura 5.14 muestra la escena de Gazebo correspondiente a esta segunda prueba de seguimiento, donde el UAV cazador mantiene la referencia visual del UAV objetivo durante la maniobra circular ondulada.

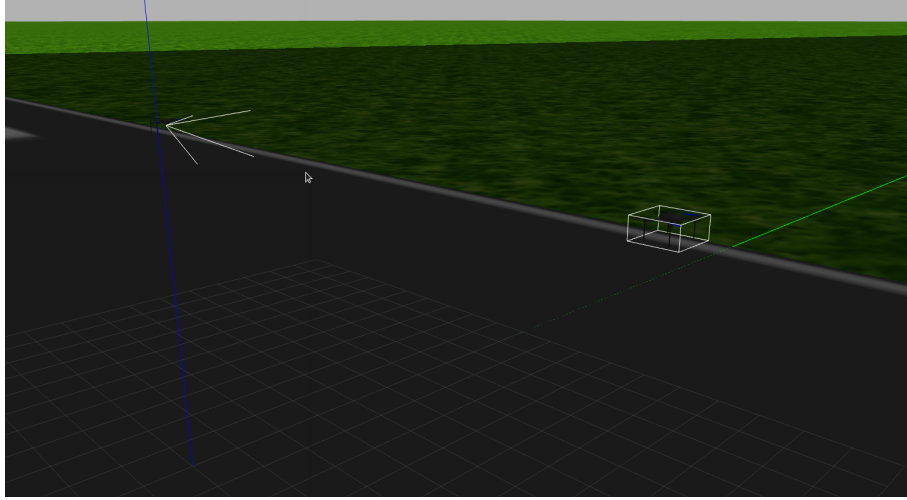


Figura 5.14: Vista del escenario inicial de Gazebo en la prueba de seguimiento con trayectoria circular ondulada. (Imagen de Elaboración Propia)

Los resultados obtenidos se resumen en la Tabla 5.5. Al igual que en la prueba anterior, los errores de imagen y el error de distancia se presentan por separado, ya que corresponden a magnitudes con unidades distintas.

Cuadro 5.5: Métricas EPE/RMSE/TLR obtenidas durante la prueba de seguimiento con trayectoria circular ondulada.

Métrica	Valor
Frames analizados	6499
Visión activa con YOLO	99.74 %
<i>Tracking</i> ciego con Kalman	0.26 %
<i>Target Loss Rate</i> (TLR)	0.00 %
EPE durante YOLO	0.0510 normalizado
EPE durante Kalman	0.1590 normalizado
EPE medio total	0.0513 normalizado
RMSE de distancia (E_z)	0.2200 m

Los resultados de seguimiento muestran un centrado preciso durante la trayectoria circular ondulada. El EPE medio total fue de 0.0513 en magnitud normalizada, con un valor de 0.0510 durante los frames con detección directa de YOLO. En los instantes en los que fue necesario recurrir al seguimiento ciego mediante Kalman, el EPE aumentó hasta 0.1590, lo que indica que la predicción mantiene una referencia útil, aunque con menor precisión que la detección directa.

Desde el punto de vista de la visión, el detector YOLO mantuvo de-

tección directa en el 99.74 % de los 6499 frames analizados. En el 0.26 % restante fue necesario recurrir al seguimiento predictivo mediante el filtro de Kalman, pero el TLR se mantuvo en 0.00 %. Esto indica que, aunque aparecen interrupciones muy puntuales de la detección directa, el sistema no pierde completamente la referencia del UAV objetivo durante la maniobra.

El RMSE de distancia fue de 0.2200 m, muy inferior al obtenido en la prueba de zigzag. Esto indica que, en esta trayectoria, el sistema mantuvo con mayor precisión la separación longitudinal respecto al UAV objetivo. En conjunto, la prueba confirma que la trayectoria circular ondulada no solo es útil para validar la actuación simultánea de los tres PID, sino también para medir la robustez del seguimiento visual en una situación tridimensional. La combinación de un TLR nulo, una detección directa superior al 99 % y errores EPE/RMSE reducidos demuestra que el sistema mantiene una referencia visual estable incluso cuando el objetivo introduce cambios simultáneos en el plano horizontal, la altura y la distancia.

Prueba de seguimiento con ruta final por *waypoints*

Esta prueba sirve como validación final del seguimiento en simulación. A diferencia de las trayectorias anteriores, la ruta por *waypoints* no sigue una forma periódica simple, sino que combina desplazamientos largos en el plano horizontal con cambios de altura entre puntos consecutivos.

Con este experimento se responde a la pregunta de si el sistema conserva la referencia visual en una maniobra menos regular y más representativa de un seguimiento general. En esta versión, la ruta aumenta el recorrido longitudinal y presenta un tramo final más prolongado, por lo que el UAV presa realiza una maniobra más exigente en la que el UAV cazador debe mantener el objetivo visible mientras cambian simultáneamente la posición lateral, longitudinal y vertical.

El experimento consiste en definir una secuencia de *waypoints* y hacer que el UAV presa los recorra dentro del entorno de simulación. Los puntos utilizados para generar la trayectoria se recogen en la Tabla 5.6; cada *waypoint* está definido mediante sus coordenadas (x, y, z) en metros.

Cuadro 5.6: *Waypoints* utilizados en la ruta final de seguimiento.

Punto	x (m)	y (m)	z (m)
WP0	5.0	-30.0	8.0
WP1	15.0	-20.0	5.0
WP2	35.0	-25.0	12.0
WP3	40.0	20.0	9.0
WP4	25.0	25.0	15.0
WP5	35.0	80.0	15.0

La trayectoria resultante se muestra en la Figura 5.15. La ruta obliga al sistema a adaptarse a varios tramos con cambios de dirección y altura, incluyendo un desplazamiento final largo hasta $y = 80$ m. Por ello, resulta adecuada para cerrar el bloque de pruebas de seguimiento con un escenario menos regular que el zigzag o la trayectoria circular ondulada.

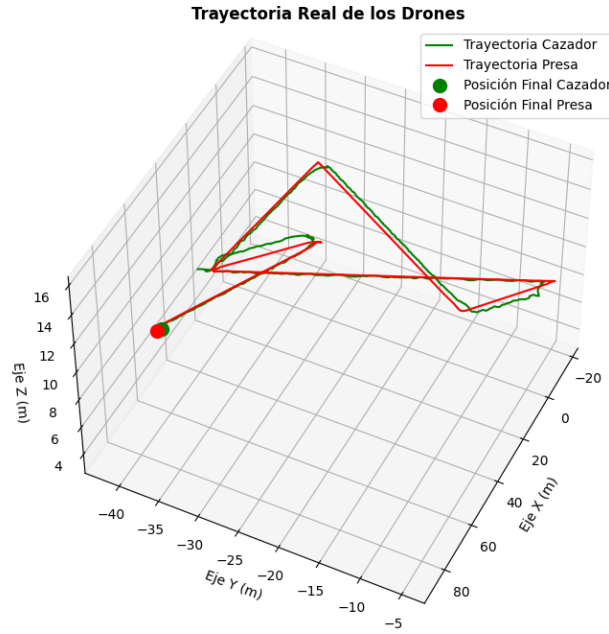


Figura 5.15: Trayectoria final por *waypoints* utilizada para la prueba de seguimiento. (Imagen de Elaboración Propia)

Durante la ejecución se registraron las métricas EPE/RMSE/TLR del vuelo. Los resultados obtenidos se resumen en la Tabla 5.7.

Cuadro 5.7: Métricas EPE/RMSE/TLR obtenidas durante la prueba de seguimiento con ruta final por *waypoints*.

Métrica	Valor
Frames analizados	17911
Visión activa con YOLO	99.98 %
<i>Tracking</i> ciego con Kalman	0.03 %
<i>Target Loss Rate</i> (TLR)	0.00 %
EPE durante YOLO	0.0221 normalizado
EPE durante Kalman	0.0584 normalizado
EPE medio total	0.0221 normalizado
RMSE de distancia (E_z)	0.6821 m

Los resultados de visión muestran que el sistema mantuvo una detección directa prácticamente continua durante la ruta final. De los 17911 frames analizados, YOLO proporcionó una referencia válida en el 99.98 % de los casos, mientras que el filtro de Kalman solo fue necesario en un porcentaje residual, del 0.03 %. Además, el TLR fue de 0.00 %, por lo que no se produjo ninguna pérdida completa del objetivo durante la trayectoria.

El EPE medio total fue de 0.0221 en magnitud normalizada, coincidiendo tras el redondeo con el valor obtenido durante detección directa de YOLO debido al uso casi exclusivo de detecciones observadas. Durante los instantes puntuales en los que se recurrió a Kalman, el EPE fue de 0.0584, lo que indica que la predicción mantuvo una referencia suficientemente precisa incluso cuando no existía una caja observada por el detector. Este resultado supone una mejora clara respecto a las pruebas anteriores en términos de centrado visual.

El RMSE de distancia fue de 0.6821 m. Este valor es mayor que el obtenido en la trayectoria circular ondulada, pero menor que el registrado en la prueba de zigzag. La diferencia se debe a que la ruta final introduce desplazamientos más largos entre *waypoints* y cambios de altura que dificultan mantener constante la separación longitudinal. Aun así, el error permanece acotado y no afecta a la continuidad del seguimiento visual.

En conjunto, la ruta final confirma que el sistema puede mantener el seguimiento del UAV presa en una trayectoria tridimensional compuesta por varios tramos, cambios de altura y un recorrido longitudinal amplio. La combinación de una detección YOLO prácticamente continua, un uso muy reducido de Kalman, un TLR nulo y un EPE medio total bajo indica que el sistema de percepción y control conserva una referencia visual robusta incluso en una maniobra menos regular y más cercana a un escenario de seguimiento general.

5.2.6. Prueba de Estimación de Distancia con Cámara Real

Esta prueba sirve como primera validación real del sistema de estimación de distancia utilizando la cámara física y el UAV objetivo real. A diferencia de las pruebas en simulación, aquí la estimación depende de la calibración real de la cámara, de las dimensiones medidas del dron objetivo y de las variaciones propias de una detección visual en un entorno no ideal.

Con este experimento se responde a la pregunta de si la distancia calculada a partir de la caja delimitadora mantiene una relación coherente con la separación real entre la cámara y el dron objetivo. También permite estudiar el límite práctico del sistema de percepción, identificando a partir de qué rango la estimación empieza a degradarse por el menor tamaño aparente del UAV en la imagen.

El experimento consiste en realizar capturas a distintas distancias conocidas, comenzando por separaciones cercanas, donde el UAV ocupa una parte significativa de la imagen, y aumentando progresivamente la separación hasta distancias mayores. La prueba se repitió cinco veces para obtener resultados más consistentes y evaluar la dispersión entre mediciones, ya que una variación de pocos píxeles en la caja delimitadora puede traducirse en un cambio significativo en la distancia, especialmente cuando el UAV objetivo se encuentra más alejado.

El montaje experimental empleado para esta prueba se muestra en la Figura 5.16. En él se situó el dron objetivo frente a la cámara del dron cazador y se utilizó un metro como referencia física para ir colocando el UAV a distintas separaciones conocidas. De esta forma fue posible comparar la distancia real medida manualmente con la distancia estimada por el sistema.



Figura 5.16: Montaje utilizado para la prueba real de estimación de distancia. (Imagen de Elaboración Propia)

Las Figuras 5.17 y 5.18 muestran dos ejemplos de la imagen observada por la cámara durante la prueba. En la primera se representa una distancia cercana de aproximadamente 1 m, donde el dron ocupa una región grande de la imagen y la caja delimitadora dispone de más píxeles para estimar la anchura. En la segunda se muestra una distancia mucho mayor, cercana a 5.75 m, donde el tamaño aparente del UAV se reduce de forma considerable y cualquier pequeña variación de la caja tiene más efecto sobre la estimación final.



Figura 5.17: Vista del UAV objetivo a una distancia aproximada de 1 m.
(Imagen de Elaboración Propia)

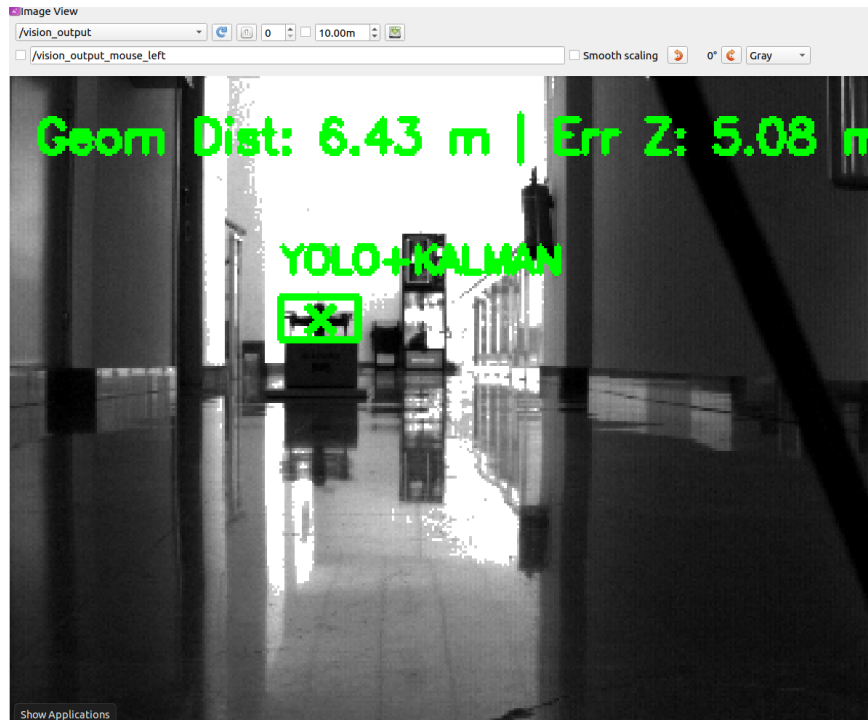


Figura 5.18: Vista del UAV objetivo a una distancia aproximada de 5.75 m. (Imagen de Elaboración Propia)

A partir de las cinco repeticiones realizadas se generaron las gráficas de error de las Figuras 5.19 y 5.20. En la primera se representa el error de estimación obtenido en cada una de las cinco pruebas, expresado en centímetros, frente a la distancia real en metros. En la segunda se muestra el error medio para cada distancia junto con la desviación típica, representada mediante una zona sombreada correspondiente a ± 1 desviación típica. De este modo, la primera gráfica permite comparar las repeticiones individuales, mientras que la segunda resume la tendencia general y la variabilidad del modelo.

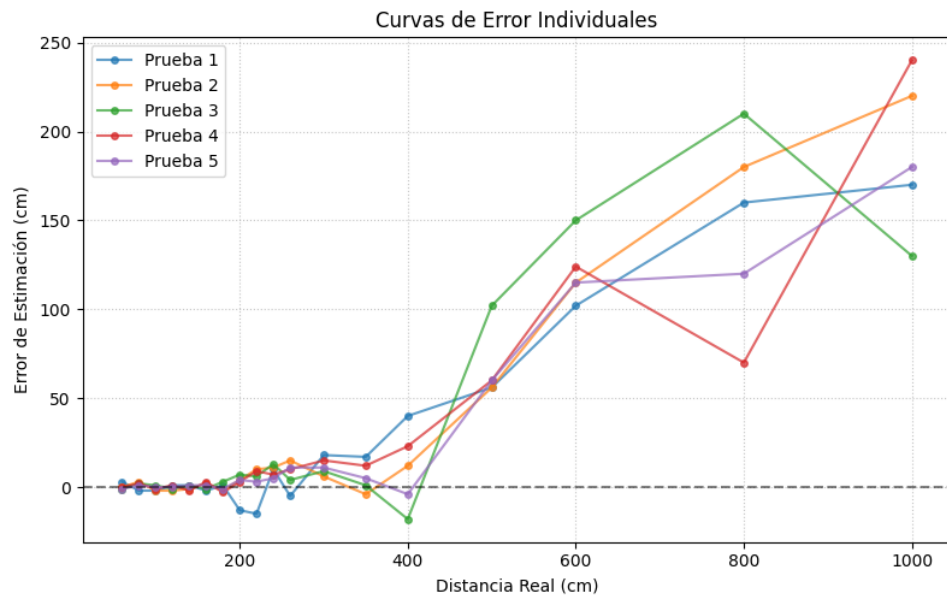


Figura 5.19: Errores de estimación de distancia obtenidos en las cinco pruebas realizadas con la cámara real y el UAV objetivo real. (Imagen de Elaboración Propia)

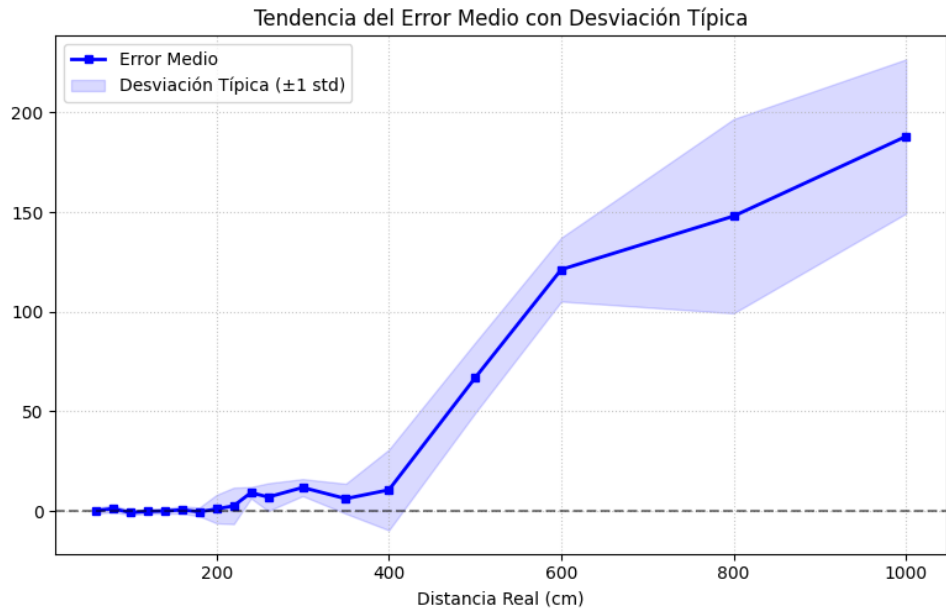


Figura 5.20: Error medio de estimación de distancia con la desviación típica sombreada para las cinco pruebas realizadas. (Imagen de Elaboración Propia)

Las métricas globales obtenidas para el modelo de distancia se resumen en la Tabla 5.8. El error medio absoluto fue de 33.68 cm y la desviación típica total alcanzó 58.48 cm. Estos valores reflejan que la estimación mantiene una relación útil con la distancia real, pero también que existe una variabilidad apreciable entre repeticiones, especialmente cuando el objetivo ocupa pocos píxeles en la imagen.

Cuadro 5.8: Métricas globales del modelo de estimación de distancia con cámara real.

Métrica	Valor
Error Medio Absoluto (MAE)	33.68 cm
Desviación típica total	58.48 cm

Los resultados muestran que, para distancias cortas y medias, el error se mantiene más acotado en comparación con las zonas de mayor separación. En la zona inicial de la gráfica aparecen desviaciones positivas y negativas de menor magnitud, lo que indica que el sistema puede sobreestimar o subestimar la distancia, pero sin perder una relación coherente entre el tamaño aparente del dron y la separación real. Este comportamiento es esperable, ya

que a corta distancia la caja delimitadora ocupa más píxeles y la estimación basada en anchura es menos sensible a pequeñas variaciones de la detección.

A medida que aumenta la distancia, el error y la dispersión entre repeticiones crecen de forma más clara. En particular, a partir de aproximadamente 3.5 m se observa que la estimación comienza a degradarse ligeramente y aparecen diferencias más apreciables entre pruebas. Esto se debe a que el UAV aparece cada vez más pequeño en la imagen: una diferencia de pocos píxeles en la anchura de la caja representa una variación mayor en la distancia calculada. Además, a distancias largas influyen más la resolución de la cámara, la estabilidad de la detección, la orientación del dron y la calidad de la calibración. Por este motivo, la desviación típica sombreada en la gráfica resulta especialmente útil para interpretar la fiabilidad de la estimación en cada rango de distancia.

En conjunto, la prueba confirma que el sistema de estimación resulta útil para trabajar en rangos cercanos y medios, que son los más relevantes para iniciar y mantener un seguimiento visual seguro. No obstante, también evidencia que la precisión empieza a verse afectada a partir de unos 3.5 m y que la variabilidad aumenta cuando el objetivo se encuentra más alejado y ocupa pocos píxeles en la imagen. Por tanto, para pruebas reales de persecución conviene tener en cuenta esta degradación progresiva y utilizar la distancia estimada como una referencia de control dentro del rango en el que la detección se mantiene suficientemente estable.

5.3. Análisis de Resultados

Los resultados obtenidos permiten valorar el comportamiento general del sistema desde cinco puntos de vista: la detección visual, la optimización de inferencia, el control del UAV cazador, el seguimiento en simulación y la estimación de distancia con cámara real. Esta lectura conjunta es importante, ya que el buen funcionamiento del sistema no depende de un único elemento, sino de la coordinación entre percepción y control.

La Tabla 5.9 resume el grado de cumplimiento de las pruebas realizadas. Se considera que un punto está cumplido cuando el resultado responde al objetivo principal de la prueba, y cumplido parcialmente cuando el sistema funciona, pero todavía muestra alguna limitación relevante.

Cuadro 5.9: Resumen del grado de cumplimiento de las pruebas realizadas.

Punto evaluado	Resultado	Justificación
Selección del detector visual	Cumplido	YOLOv8n ofrece un buen equilibrio entre precisión, tamaño y velocidad, con $mAP50$ de 0.992507, $mAP50-95$ de 0.733692 y 2.660333 ms de inferencia.
Optimización con TensorRT	Cumplido	La conversión a <code>.engine</code> FP16 reduce el tiempo total de 22.53 ms a 11.74 ms y eleva los FPS teóricos de 44.4 a 85.2, dejando margen suficiente para una cámara de 30 FPS.
Control de <i>yaw</i>	Cumplido	El error horizontal se corrige con rapidez, con un tiempo de establecimiento aproximado de 0.65 s.
Control de altura	Cumplido	El error vertical se reduce de forma estable, con un tiempo de establecimiento aproximado de 1.25 s.
Control de distancia	Cumplido parcialmente	par- Mantiene una separación segura, aunque responde más lento que los otros ejes, con $T_s \approx 6$ s.
Seguimiento visual en simulación	Cumplido	En las trayectorias evaluadas no se perdió el objetivo, con TLR del 0.00 % y detección YOLO igual o superior al 99.74 %.
Seguimiento en zigzag	Cumplido parcialmente	par- El objetivo se mantiene visible durante toda la prueba, aunque la distancia presenta más error, con RMSE de 1.255 m.
Trayectoria circular ondulada	Cumplido	Se obtiene seguimiento estable, con EPE medio de 0.0513 y RMSE de distancia de 0.2200 m.
Ruta final por <i>waypoints</i>	Cumplido	El sistema mantiene un TLR nulo, detección YOLO del 99.98 % y EPE medio total de 0.0221.
Estimación de distancia con cámara real	Cumplido parcialmente	par- La estimación es útil en distancias cortas y medias, aunque a partir de unos 3.5 m empieza a perder estabilidad; el MAE fue de 33.68 cm.
Seguimiento real completo	Pendiente	No se valida todavía una persecución real completa con ambos UAV en vuelo, sino una primera prueba real de estimación de distancia.

En percepción, el objetivo principal se considera cumplido. YOLOv8n fue seleccionado porque combina una precisión alta con un tiempo de inferencia bajo, lo que facilita su uso dentro del lazo de control. Además, la conversión a TensorRT redujo de forma notable la latencia del detector, pasando de 22.53 ms a 11.74 ms por frame, lo que incrementa el margen temporal disponible para el resto de procesos del sistema. En las pruebas de seguimiento, el detector mantuvo porcentajes de detección muy altos y no se produjo pérdida completa del UAV objetivo en simulación.

En cuanto al control, los resultados son claros en los ejes de *yaw* y altura. Ambos controladores corrigen el error de forma rápida y estable, permitiendo que el objetivo vuelva al centro de la imagen. El control de distancia también cumple su función principal, ya que mantiene una separación segura, aunque su respuesta es más lenta y muestra más dificultad en trayectorias como el zigzag.

Las pruebas de seguimiento en simulación muestran un comportamiento global positivo. El sistema fue capaz de seguir al UAV objetivo en trayectorias con cambios laterales, variaciones de altura y rutas por *waypoints*. Aunque el eje de distancia sigue siendo el más exigente, la referencia visual se mantuvo durante las pruebas y el seguimiento no se interrumpió.

En las pruebas reales, la estimación de distancia funcionó de forma coherente en rangos cercanos y medios. Sin embargo, a partir de aproximadamente 3.5 m el objetivo ocupa menos píxeles en la imagen y la estimación empieza a ser menos estable. Por ello, en ese rango conviene que el control actúe de forma más conservadora y se apoye en la continuidad de la detección visual.

En conjunto, los objetivos principales del capítulo quedan cubiertos: se selecciona un detector adecuado, se optimiza su ejecución mediante TensorRT, se valida el centrado visual, se comprueba el seguimiento en simulación y se realiza una primera prueba real de distancia. Las principales líneas de mejora se centran en ajustar el control longitudinal, mejorar la estimación de profundidad a distancias mayores y avanzar hacia una validación real completa con ambos UAV en vuelo.

Conclusiones

6.1. Revisión del cumplimiento de objetivos

En esta sección se revisa el grado de cumplimiento de los objetivos planteados al inicio del trabajo. Para cada uno de ellos se indica si se considera cumplido, cumplido parcialmente o pendiente, justificando la valoración a partir de los resultados obtenidos durante la implementación y la validación experimental.

Objetivo general: Desarrollar e implementar un sistema de control de navegación autónoma para seguimiento de UAVs

Grado de cumplimiento: Cumplido parcialmente, aproximadamente un 85 %.

El objetivo general se considera cumplido aproximadamente en un 85 %, ya que se ha desarrollado e integrado un sistema completo de seguimiento visual compuesto por percepción mediante YOLO, filtrado Kalman + EMA, estimación de distancia, compensación de actitud y control PID en tres ejes. El sistema ha sido validado en simulación con trayectorias dinámicas, manteniendo el UAV objetivo dentro del campo de visión y generando consignas de control coherentes para el UAV cazador.

No obstante, no se ha podido realizar de forma completa porque la validación real se ha limitado a la preparación del hardware, la calibración de la cámara y la comprobación de la estimación de distancia con el UAV objetivo físico. Por tanto, aunque se ha podido completar una parte significativa del trabajo, estimada en torno al 85 %, queda pendiente realizar una persecución real completa con ambos UAV en vuelo y demostrar la autonomía total del sistema en entorno real.

OE1: Evaluar y optimizar modelos de percepción visual para sistemas empotrados

Grado de cumplimiento: Cumplido.

Este objetivo se considera cumplido. Se entrenaron y compararon distintas arquitecturas de la familia YOLO, evaluando precisión, tamaño del modelo y tiempo de inferencia. A partir de esta comparación se seleccionó YOLOv8n como modelo final, no por ser necesariamente el detector con mayor precisión absoluta, sino por ofrecer el mejor equilibrio entre exactitud, rapidez y coste computacional para su integración en un lazo de control visual en tiempo real.

Los resultados obtenidos confirman esta elección, ya que el modelo alcanzó métricas de detección elevadas y mantuvo una inferencia suficientemente rápida. Además, durante las pruebas de seguimiento en simulación se observaron porcentajes de detección directa muy altos y una tasa de pérdida total del objetivo nula, lo que demuestra que el detector elegido proporciona una señal visual estable para alimentar el sistema de control.

OE2: Diseñar e implementar el lazo de control de navegación visual

Grado de cumplimiento: Cumplido.

Este objetivo se considera cumplido. Se ha diseñado e implementado el lazo de control completo, transformando las detecciones 2D de la cámara en errores visuales y longitudinales que alimentan tres controladores PID: *yaw*, altura y distancia. Los controladores de *yaw* y altura mostraron una respuesta estable y rápida, permitiendo mantener el objetivo centrado horizontal y verticalmente en la imagen.

La parte longitudinal también se implementó y permitió mantener una separación de seguimiento segura. Aunque su respuesta fue más lenta y presentó un error mayor que los ejes de centrado visual, este comportamiento resulta coherente con la función del controlador de distancia: no debe aproximar el UAV cazador de forma agresiva al objetivo, sino mantener un equilibrio entre reactividad y seguridad para evitar colisiones. Por tanto, el lazo de control cumple su objetivo, ya que permite seguir al UAV objetivo, mantenerlo centrado y conservar una distancia de seguridad durante el seguimiento.

OE3: Integrar la arquitectura software en un entorno de simulación realista

Grado de cumplimiento: Cumplido.

Este objetivo se considera cumplido. La arquitectura software fue integrada en un entorno basado en ROS, Gazebo, ArduPilot y MAVROS, permitiendo conectar los nodos de percepción, estimación del error y control con el autopiloto simulado mediante SITL. El sistema quedó organizado

como un flujo cerrado desde la imagen de entrada hasta la publicación de consignas de velocidad.

Además, la integración se verificó mediante los grafos de comunicación de ROS y mediante pruebas de funcionamiento con el sistema completo activo. La existencia de tópicos diferenciados para imagen, detección, error visual, estado del UAV y consignas de velocidad demuestra que la arquitectura quedó correctamente conectada y preparada para ejecutar ensayos de seguimiento en simulación.

OE4: Validar la robustez del sistema frente a maniobras evasivas

Grado de cumplimiento: Cumplido parcialmente.

Este objetivo se considera cumplido parcialmente. Se diseñaron y ejecutaron distintas pruebas dinámicas en simulación, incluyendo trayectorias en zigzag, trayectorias circulares con variación de altura y una ruta final por *waypoints*. En todas ellas el sistema mantuvo la referencia visual del objetivo, con una tasa de pérdida total nula, lo que confirma una robustez adecuada de la percepción y del centrado visual.

Sin embargo, algunas maniobras evidenciaron limitaciones en el eje longitudinal. En particular, la trayectoria en zigzag produjo un error de distancia superior al observado en otras pruebas, mostrando que el seguimiento de separación todavía es sensible a cambios rápidos de movimiento. Por ello, el objetivo se considera parcialmente cumplido: el sistema es robusto para mantener el objetivo visible y seguirlo en simulación, pero la respuesta de distancia ante maniobras exigentes aún puede mejorarse.

OE5: Desplegar y evaluar la viabilidad computacional en una plataforma empotrada real

Grado de cumplimiento: Cumplido parcialmente.

Este objetivo se considera cumplido parcialmente. Se avanzó en la preparación del sistema para pruebas reales mediante la calibración de la cámara, la medición física del UAV objetivo y la adaptación de los parámetros necesarios para emplear el sistema fuera del entorno simulado. También se validó la estimación de distancia con cámara real, comprobando que el método proporciona una referencia útil en distancias cortas y medias.

No obstante, no se completó una validación real del seguimiento autónomo en vuelo ni una evaluación exhaustiva del rendimiento del sistema empotrado durante una misión completa. Por tanto, aunque la viabilidad inicial queda apoyada por la selección de un modelo ligero y por las pruebas reales

de percepción, queda pendiente medir de forma definitiva la tasa de procesamiento, la latencia y la seguridad del sistema durante un despliegue real completo.

6.2. Futuros Trabajos

Como continuación del trabajo desarrollado, se proponen las siguientes líneas futuras:

- **Finalización y evaluación de las pruebas reales:** completar el seguimiento con ambos UAV en vuelo y analizar el rendimiento del sistema fuera de simulación. Estas pruebas permitirían comprobar su comportamiento ante iluminación variable, viento, movimiento real del objetivo y posibles pérdidas temporales de detección.
- **Integración de una cámara de eventos:** estudiar la incorporación de este tipo de sensor para detectar cambios rápidos en la escena con menor latencia. Esto podría ayudar a localizar antes al UAV objetivo y aumentar la capacidad de reacción del sistema ante maniobras bruscas.
- **Optimización del sistema:** reducir la latencia global del procesamiento, mejorar la estimación de distancia y ajustar los parámetros de control. Con ello se buscaría una respuesta más rápida, estable y adecuada para su funcionamiento en tiempo real.

$$Z_c = \frac{f_x \cdot X_c}{x}$$

Bibliografía

- [1] Robot Operating System, *Logotipo de ROS*, URL: <https://www.skyfilabs.com/blog/10-simple-ros-projects-for-beginners>.
- [2] Jobted, *Salario promedio para ingeniero de software en España*, 2025. URL: <https://www.jobted.es/salario/ingeniero-software>.
- [3] Dron de ala fija, *Foto de dron ala fija*, URL: <https://umilesgroup.com/dron-ala-fija/>.
- [4] Dron VTOL, *Foto de dron VTOL*, URL: <https://www.foxtechuav.com/es/inspection-vtol-drone-long-endurance-2500mm-wingspan-ayk-250.html>.
- [5] Dron multirrotores, *Foto de dron multirrotores*, URL: <https://www.unmannedsystemstechnology.com/es/expo/multirrotor-drones/>.
- [6] clasificación drones 1, *clasificación drones 1*, URL: <https://umilesgroup.com/tipos-de-drones/>.
- [7] clasificación drones 2, *clasificación drones 2*, URL: <https://www.embention.com/es/noticias/tipos-de-drones-y-sus-principales-ventajas/>.
- [8] Adrián Hernández, *Problema del desvanecimiento del gradiente (vanishing gradient problem)*, Meta-learning, 6 de mayo de 2018. URL: <https://stalearning.wordpress.com/2018/05/06/problema-de-desvanecimiento-del-gradiente-vanishing-gradient-problem/>.
- [9] ROS Wiki, *Instalación de ROS Noetic en Ubuntu*, URL: <https://wiki.ros.org/noetic/Installation/Ubuntu>.
- [10] Instituto Nacional de Técnica Aeroespacial (INTA), *Logotipo del INTA*, URL: <https://www.inta.es/INTA/es/index.html>.
- [11] Intelligent Quads, *Installing ROS on Ubuntu 20.04*, GitHub, guía de instalación. URL: https://github.com/Intelligent-Quads/iq-tutorials/blob/master/docs/installing_ros_20_04.md.

BIBLIOGRAFÍA

- [12] Intelligent Quads, *Installing Gazebo ArduPilot Plugin*, GitHub, guía de instalación. URL: https://github.com/Intelligent-Quads/iq_tutorials/blob/master/docs/installing_gazebo_ardupugin.md.
- [13] Intelligent Quads, *github de iqsim*: https://github.com/Intelligent-Quads/iq_sim.
- [14] I. Goodfellow, Y. Bengio y A. Courville, *Deep Learning*, MIT Press, 2016. URL: <https://www.deeplearningbook.org/>.
- [15] Nanobaly, *Introducción a la detección de objetos en visión artificial [2025]*, Innovatiana, publicado el 2024-01-26. URL: <https://www.innovatiana.com/es/post/object-detection-our-guide>.
- [16] Imagen Dron monitoreando cultivos, *Imagen Dron monitoreando cultivos*, URL: <https://uss.com.ar/corporativo/aplicaciones-de-drones-para-monitoreo-y-vigilancia-de-grandes-areas/>.
- [17] Web sobre Dron en la agricultura, *Web sobre Dron en la agricultura*, URL: <https://www.muginuav.com/es/drones-in-agriculture-overview/>.
- [18] Imagen Dron Rescate Marítimo, *Imagen Dron Rescate Marítimo*, URL: <https://www.drone-school.es/contratar-pilotos-y-servicios-con-drones/servicios-con-drone-para-salvamento-y-busqueda/>.
- [19] Web el uso del dron en emergencias y búsqueda, *Web el uso del dron en emergencias y búsqueda*, URL: <https://aerocamaras.es/servicios-drones-profesionales/emergencias-y-rescates-con-drones/>.
- [20] Imagen Dron transportando paquetes, *Imagen Dron transportando paquetes*, URL: <https://easy-trans.es/el-actual-uso-de-los-drones-en-el-transporte/>.
- [21] Web el uso del dron en logística, transporte, *Web el uso del dron en logística, transporte*, URL: <https://idc.apddrones.com/transporte/uso-de-drones-en-transporte-y-logistica/>.
- [22] Imagen Dron uso en defensa, *Imagen Dron uso en defensa*, URL: <https://geopol21.com/como-ha-evolucionado-la-guerra-con-drones-autonomos/>.
- [23] Web el uso del dron en defensa, *Web el uso del dron en defensa*, URL: <https://sentrycs.com/es/the-counter-drone-blog/empowering-special-forces-with-counter-drone-solutions/>.

BIBLIOGRAFÍA

- [24] K. Chávez y O. Swed, *Emulating underdogs: Tactical drones in the Russia-Ukraine war*, 2024. URL: <https://www.tandfonline.com/doi/full/10.1080/13523260.2023.2257964>.
- [25] Wikipedia, *Visual servoing*, URL: https://en.wikipedia.org/wiki/Visual_servoing.
- [26] F. Chaumette y S. Hutchinson, *Visual servo control, part I: Basic approaches*, IEEE Robotics & Automation Magazine, vol. 13, n.º 4, 2006. URL: <https://inria.hal.science/inria-00350283>.
- [27] K. Yang, C. Bai, Z. She y Q. Quan, *High-speed interception multicop-ter control by image-based visual servoing*, IEEE Trans. Control Syst. Technol., 2024. URL: <http://arxiv.org/abs/2404.08296>.
- [28] H. Yan, K. Yang, Y. Cheng, Z. Wang y D. Li, *Precise interception flight targets by image-based visual servoing of multicopter*, sep. 2024. URL: <http://arxiv.org/abs/2409.17497>.
- [29] P. M. Wyder et al., *Autonomous drone hunter operating by deep learning and all-onboard computations in GPS-denied environments*, PLOS ONE, vol. 14, 2019. URL: <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0225092>.
- [30] Z. W. Lee, W. H. Chin y H. W. Ho, *Air-to-air micro air vehicle interceptor with an embedded mechanism and deep learning*, Aerospace Science and Technology, vol. 135, 2023. URL: <https://www.sciencedirect.com/science/article/pii/S1270963823000895>.
- [31] A. Rodriguez-Ramos, A. Alvarez-Fernandez, H. Bavle, J. Rodriguez-Vazquez, L. Lu, M. Fernandez-Cortizas, R. A. Suarez Fernandez, A. Rodelgo, C. Santos, M. Molina, L. Merino, F. Caballero y P. Campoy, *Autonomous Aerial Robot for High-Speed Search and Intercept Applications*, arXiv preprint arXiv:2112.05465, 2021. URL: <https://arxiv.org/abs/2112.05465>.
- [32] C.A.G. van der Aa, *Design of an Image-Based Visual Servoing System for Autonomous Quadcopter Interception*, Master Thesis, Delft University of Technology, 2024. URL: <https://resolver.tudelft.nl/uuid:4d8f5709-c20e-4ed2-9f62-26ce3ecde223>.
- [33] H. Weber, *Autonomous Drone Navigation Using Reinforcement Learning Algorithms*, Scientific Journal of Artificial Intelligence and Blockchain Technologies, vol. 2, n.º 3, 2025. DOI: <https://doi.org/10.63345/sjaibt.v2.i3.303>, URL: https://www.researchgate.net/publication/396807493_Autonomous_Drone_Navigation_Using_Reinforcement_Learning_Algorithms.

BIBLIOGRAFÍA

- [34] A. V. R. Katkuri, H. Madan, N. Khatri, A. S. H. Abdul-Qawy y K. S. Patnaik, *Autonomous UAV navigation using deep learning-based computer vision frameworks: A systematic literature review*, Array, vol. 23, 2024, artículo 100361. DOI: <https://doi.org/10.1016/j.array.2024.100361>, URL: <https://www.sciencedirect.com/science/article/pii/S2590005624000274>.
- [35] M. Guazzaloca, *Learning-Based Control for Nano-Drones Flight: From Simulation to Reality*, Master Thesis, Alma Mater Studiorum – University of Bologna, Academic Year 2024/2025. URL: <https://amslaurea.unibo.it/id/eprint/36244/1/main.pdf>.
- [36] SharkYun, *Computer Vision — Object Detection, One-Stage vs Two-Stage detectors*, 2024. URL: <https://sharkyun.medium.com/computer-vision-object-detection-one-stage-vs-two-stage-b05dbff88195>.
- [37] Abirami Vina, *¿Qué es R-CNN? Una descripción general rápida*. Ultralytics, 7 de junio de 2024. URL: <https://www.ultralytics.com/es/blog/what-is-r-cnn-a-quick-overview>.
- [38] P. F. Felzenszwalb, R. B. Girshick, D. McAllester y D. Ramanan, *Object Detection with Discriminatively Trained Part-Based Models*, IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 32, n.º 9, 2010, pp. 1627–1645. URL: <https://cs.brown.edu/people/pfelzens/papers/lsvm-pami.pdf>.
- [39] J. Terven et al., *A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS*, 2023. URL: <https://www.mdpi.com/2504-4990/5/4/83>.
- [40] AIknow.io, *Introducción a YOLO*, 2024. URL: <https://www.aiknow.io/es/introduccion-a-yolo/>.
- [41] Y. Zheng et al., *Air-to-air visual detection of micro-UAVs: An experimental evaluation of deep learning*, IEEE Robotics and Automation Letters, 2021. URL: <https://ieeexplore.ieee.org/document/9343737>.
- [42] R. E. Kalman, *A new approach to linear filtering and prediction problems*, 1960. URL: <https://asmedigitalcollection.asme.org/fluidsengineering/article/82/1/35/397706/A-New-Approach-to-Linear-Filtering-and-Prediction>.
- [43] KalmanFilter.net, *Filtro de Kalman explicado mediante ejemplos*, consultado el 20 de abril de 2026. URL: https://kalmanfilter.net/ES/default_es.aspx.

BIBLIOGRAFÍA

- [44] Wikipedia, *Filtro de Kalman*, consultado el 20 de abril de 2026. URL: https://es.wikipedia.org/wiki/Filtro_de_Kalman.
- [45] Industrial Monitor Direct, *Analysis of an Exponential Moving Average (EMA) Filter in Ladder Logic for Load Cell Signal Conditioning*, consultado el 20 de abril de 2026. URL: <https://industrialmonitordirect.com/blogs/knowledgebase/analysis-of-an-exponential-moving-average-ema-filter-in-ladder-logic-for-load->
- [46] Wikipedia, *Suavizamiento exponencial*, consultado el 20 de abril de 2026. URL: https://es.wikipedia.org/wiki/Suavizamiento_exponencial.
- [47] MCI Electronics, *Filtro exponencial EMA (Exponential Moving Average)*, consultado el 20 de abril de 2026. URL: <https://cursos.mcielectronics.cl/2025/07/24/filtro-exponencial-ema-exponential-moving-average/>.
- [48] H. Jabbari Asl, G. Oriolo y H. Bolandi, *An adaptive scheme for image-based visual servoing of an underactuated UAV*, Int. Journal of Robotics and Automation, vol. 29, 2014. DOI: https://www.google.com/url?sa=t&source=web&rct=j&opi=89978449&url=https://www.researchgate.net/publication/260173544_An_adaptive_scheme_for_image-based_visual_servoing_of_an_underactuated_uav&ved=2ahUKEwiCisPGve2TAxVnxgIHHTv8DRkQFnoECBkQAQ&usg=A0vVaw23lbIYBAzu8TNIdwwOtQ4n.
- [49] ACTA Press, *An adaptive scheme for image-based visual servoing of an underactuated UAV*, fuente de la imagen de compensación de actitud. URL: <https://www.actapress.com/PaperInfo.aspx?paperId=43448>.
- [50] OpenCV, *Camera Calibration and 3D Reconstruction*, URL: https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html.
- [51] Wikipedia, *Cámara estenopeica*, consultado el 4 de mayo de 2026. URL: https://es.wikipedia.org/wiki/C%C3%A1mara_estenopeica.
- [52] openMVG library, *cameras*, documentación del modelo de cámara *pinhole*, de donde se obtiene el esquema utilizado en esta memoria. Consultado el 4 de mayo de 2026. URL: <https://openmvg.readthedocs.io/en/latest/openMVG/cameras/cameras/>.
- [53] ROS Wiki, *Monocular Camera Calibration*, tutorial utilizado para la calibración monocular con el paquete `camera_calibration`, consultado el 8 de junio de 2026. URL: https://wiki.ros.org/camera_calibration/Tutorials/MonocularCalibration.

BIBLIOGRAFÍA

- [54] CameraModels, *Camera Models*, URL: https://web.stanford.edu/class/cs231a/course_notes/01-camera-models.pdf.
- [55] F. Bergamasco, *Pinhole Camera*, Computer Vision, curso 2018/2019. URL: https://www.dsi.unive.it/~bergamasco/teachingfiles/cvslides2019/13_pinhole_camera.pdf.
- [56] R. van den Boomgaard, *The Pinhole Camera*, Lecture Notes on Image Processing and Computer Vision, University of Amsterdam. URL: <https://staff.fnwi.uva.nl/r.vandenboomgaard/IPCV20162017/LectureNotes/CV/PinholeCamera/PinholeCamera.html>.
- [57] Distorsiones de una cámara pinhole, *Distorsiones de una cámara pinhole*, URL: https://www.researchgate.net/figure/A-pinhole-camera-shows-no-distortion-Images-of-a-rectangular-object-screen-shows-distortion-fig2_316300680.
- [58] Esquema del PID, *Esquema del PID*, URL: <https://controlautomaticoeducacion.com/control-de-procesos/control-ipd/>.
- [59] Dewesoft, *¿Qué es un controlador PID?*, URL: <https://dewesoft.com/es/blog/que-es-un-controlador-pid>.
- [60] Wikipedia, *Controlador PID*, URL: https://es.wikipedia.org/wiki/Controlador_PID.
- [61] NVIDIA, *NVIDIA Jetson Orin*, especificaciones oficiales de la familia Jetson Orin, incluyendo rangos de potencia configurable y rendimiento máximo en TOPS. URL: <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/>.
- [62] Wikipedia, *Arduino Uno*, fuente de referencia para la imagen de la placa Arduino Uno. URL: https://es.wikipedia.org/wiki/Arduino_Uno.
- [63] RS Online, *Raspberry Pi*, fuente de referencia para la imagen de la placa Raspberry Pi. URL: <https://es.rs-online.com/web/p/raspberry-pi/0219252>.
- [64] Amazon, *NVIDIA Jetson Orin Nano*, fuente de referencia para la imagen de la plataforma NVIDIA Jetson Orin Nano. URL: <https://www.amazon.es/Nvidia-Jetson-Orin-Nano-GHz/dp/B09GKX1G16>.
- [65] NVIDIA, *NVIDIA TensorRT SDK*, documentación oficial del SDK para optimización y aceleración de inferencia de modelos de *Deep Learning* sobre GPUs NVIDIA. URL: <https://developer.nvidia.com/tensorrt>.

- [66] Ultralytics, *TensorRT*, glosario técnico sobre TensorRT, sus aplicaciones en el mundo real y su uso con modelos YOLO. URL: <https://www.ultralytics.com/es/glossary/tensorrt#aplicaciones-en-el-mundo-real>.
- [67] Microsoft, *ONNX Runtime*, documentación oficial del motor de inferencia multiplataforma para modelos ONNX. URL: <https://onnxruntime.ai/>.